Contents

Overview

The aim of this project is to provide a set of convenient and performant tools for performing worldsheet string theory calculations. The project is split into the following parts:

- CFT-Code is the kernel for performing worldsheet CFT calculations. It defines local operators, their OPE and correlation functions (on various Riemann surfaces).
- Tensor-Code defines tensor structures and provides utilities for their manipulation and generation.
- On-Shell-Code defines string amplitudes in the on-shell formalism.
- SFT-Code defines string field theory vertices and brackets given an abstract set of local coordinate data. It provides utilities for the local coordinate data construction of open-closed $SL(2)$ vertices.
- NLSM-Code defines utilities for quantizing nonlinear sigma models on weakly curved target spacetimes. As such it defines the Polyakov path integral in a suitable regularization scheme. Composite operators are defined by inserting fields into the regularized path integral.
- Pure-Spinor-Code defines utilities for quantizing the $AdS_5 \times S^5$ pure spinor worldsheet theory by defining its path integral in a suitable regularization scheme. Composite operators are defined by inserting fields into the regularized path integral.

For each part, we have a few physical goals in mind, guiding the implementation requirements:

- CFT-Code

  - Covariantly solve R-sector OPE and correlation functions for GSO-projected free fermion + ghosts theories. By solving, we mean to all mass levels and at practical speeds. This provides the basic ingredients for the treatment of RR backgrounds in SFT and for computing elusive high-point on-shell amplitudes involving the R-sector.

- Tensor-Code
  - For many applications (beyond the worldsheet), one needs to quickly generate independent invariant tensors contained in some tensor product of representations. Current Lie algebra packages are severely lacking in this regarding, and we aim to fill this gap.

- On-Shell-Code
  - Compute higher-point string amplitudes. A good testing ground is to reproduce known results regarding the SUSY completion of R^4 term of Type IIB superstring, and to extend it further. This will require very efficient handling of tensor structures on both the string theory and the EFT side. For example, we expect there to be a 16-dilatino term and a 16-dilatino amplitude should have about 80 million independent invariant tensors in it.
  - As a check, we also aim to compute the RR and NSNS amplitudes at four-point tree-level and one-loop and check they are related by SUSY [1]. This is nontrivial, the R-sector amplitudes involve excited spin fields upon picture-raising.

- SFT-Code
  - Compute the anomalous scaling dimension of the Konishi string in the large radius $AdS_5 \times S^5$ regime. To begin with, at tree-level, but more ambitiously also incorporate string loops, which go beyond integrability.
  - Carry out loop calculations on Calabi-Yau orientifolds, expressing the answers in terms of the 2d $(2,2)$ SCFT conformal data.

- NLSM-Code
  - Given numeric Calabi-Yau metrics, obtain the 2d $(2,2)$ SCFT conformal data in a large volume expansion. A good test-case is the IIA quintic as it only has a single volume modulus.

- Pure-Spinor-Code
  - Systematically quantize the $AdS_5 \times S^5$ pure spinor $PSU(2,2|4)$ sigma model and use it to compute the pure spinor BRST cohomology at mass level 0 and 1.

In a previously written Mathematica prototype, we have encountered that there are a few major bottlenecks towards achieving these goals.

- Free field calculations fundamentally lead to a huge combinatorial blow-up of the number of terms involved. When scaling up, it is infeasible to store the intermediate expressions in RAM. Rather, we construct a stream of data that emits the various expressions and introduce filters along the way that extract the desired result. We are then only limited by the size of that result.

- R-sector OPE was severely limited by the speed at which we could construct invariant tensors. Our algorithm was to construct a largely overcomplete set of invariants and then statistically (by plugging in random components) determine an independent subset. We must now leverage representation theory better to do less computation.

- We have not succeeded in writing the code abstractly enough so that adding new CFTs, combining CFTs to form a valid string worldsheet and adding boundaries, orientifolds could be done without large interference with the code. We must now design the CFT-Code kernel abstractly enough to avoid these issues.

Due to the large expressions involved, each part has a core written in a performant programming language that handles memory explicitly. We expose a well-defined API that allows one to easily write wrappers for say Python, Mathematica, ... backing a DSL. Also note that having such a concise API to carry out worldsheet calculations is useful for agentic coding as it saves a large amount of context.

String-Code

The root module exposes the two build-checked kernels. Higher string modules depend on these through their public APIs.

API

Constants

`tensor = @import("tensor-code")`
tensor exposes tensor-code data structures and tensor handles.

`cft = @import("cft-code")`
cft exposes the selected worldsheet CFT public API.

```
/// tensor exposes tensor-code data structures and tensor handles.
pub const tensor = @import("tensor-code");
/// cft exposes the selected worldsheet CFT public API.
pub const cft = @import("cft-code");
```

CFT-Code

The following module is the worldsheet CFT core of the project. It defines symbolic Expressions used in worldsheet CFT calculations i.e. local Operators multiplied by nontrivial Coefficients and computes their OPE and Correlators. It can also generate a basis of local operator with a given set of quantum numbers via Basis-Generation. The data of a CFT or BCFT is specified by generated preset constructors and executed by CFT Kernel.

As far as dependencies go, this module only depends on Tensor-Code, which forms an integral part of the definition of Coefficients and determines the properties of various quantum numbers of Operators. The public API follows Tensor-Code: implementation nodes such as Presets, Theory, and CFT Kernel organize the code, but clients import this root module and use the selected declarations below.

API

Types

`Handle = presets.Handle`
Handle groups compact ids used by generated preset operator builders.

`FreeBoson = presets.FreeBoson`
FreeBoson groups the public free-boson preset constructors and configs.

`Bc = presets.Bc`
: Bc groups the public bc-ghost preset constructors and configs.

Constants

`scalars = presets.scalars`
: scalars exposes scalar atoms and exact scalar helpers used in convention choices.

`product = presets.product`
: product builds the generated preset type for a product of independent bulk presets.

`boundary = presets.boundary`
: boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.

```
const presets = @import("presets.zig");

/// Handle groups compact ids used by generated preset operator builders.
pub const Handle = presets.Handle;
/// scalars exposes scalar atoms and exact scalar helpers used in convention choices.
pub const scalars = presets.scalars;

/// FreeBoson groups the public free-boson preset constructors and configs.
pub const FreeBoson = presets.FreeBoson;
/// Bc groups the public bc-ghost preset constructors and configs.
pub const Bc = presets.Bc;
/// product builds the generated preset type for a product of independent bulk presets.
pub const product = presets.product;
/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
↪  extensions.
pub const boundary = presets.boundary;
```

This root block is the public CFT API for now. It intentionally groups each CFT family behind one compact object and does not export raw Wick-rule data, correlator configs, generated-kernel internals, or the preset implementation namespace. Application modules such as on-shell code, SFT code, NLSM code, and pure-spinor code should build theories through the constructors above; a later public user-defined-CFT DSL should be one deliberate export rather than a leak of the preset runtime.

1 Expressions

Interface

Types

`Expr (struct)`
: A symbolic expression encountered in worldsheet calculations

`ExprSummand (struct)`
: Each summand is a local operator with a coefficient

```
const coefficient = @import("coefficient.zig");
const operators = @import("operators.zig");
```

The symbolic expressions we work with are simply Coefficients multiplied by local Operators.

```
/// A symbolic expression encountered in worldsheet calculations
pub const Expr = struct {
    summands: []const ExprSummand
```

```
};
/// Each summand is a local operator with a coefficient
pub const ExprSummand = struct {
    coeff: coefficient.Coeff,
    operators: operators.Operator
};
```

1.1 Operators

Interface

Types

`Operator (struct)`
: A local operator inserted at a point

`OperatorBodyId = u32`
: An identifier for operator body

`OperatorBody (union)`
: An operator body is either atomic or normal-ordered

`AtomicBody (struct)`
: An atomic operator body comes in different kinds and can carry extra information

`OperatorKindId = u32`
: An identifier for operator kind

`OperatorExtraId = u32`
: An identifier for operator extra information

`OperatorKind (struct)`
: Defines the various kinds of operators

`Holomorphicity (enum)`
: Determines whether an operator is holomorphic/antiholomorphic or both

`ConformalWeights (union)`
: ConformalWeights stores fixed weights or a rule for computing them.

`FixedConformalWeights (struct)`
: FixedConformalWeights stores optional left and right conformal weights.

`WeightRuleId = u16`
: WeightRuleId names a rule that computes conformal weights from labels.

`CorrelatorTactic (enum)`
: A tactic for the the evaluation of correlators

`OperatorInsertion (union)`
: Operator insertion data are specified by a position and a number of derivatives

```
const coefficient = @import("coefficient.zig");
const tensor = @import("tensor-code");
```

A local operator can be inserted at a point and carry extra information captured in the code by its body.

```
/// A local operator inserted at a point
pub const Operator = struct {
    insertion: OperatorInsertion,
    body: OperatorBodyId,
};
```

```
/// An identifier for operator body
pub const OperatorBodyId = u32;
```

An operator body can either be that of the identity, a generic atomic operator or a normal-ordered product of operator bodies (in free field theory).

```
/// An operator body is either atomic or normal-ordered
pub const OperatorBody = union(enum) {
    identity,
    atomic: AtomicBody,
    normal_ordered: []const OperatorBodyId,
};
```

An atomic operator body can come in different kinds specified when defining the Theory and can carry extra runtime information.

```
/// An atomic operator body comes in different kinds and can carry extra information
pub const AtomicBody = struct {
    kind: OperatorKindId,
    extra: OperatorExtraId,
};

/// An identifier for operator kind
pub const OperatorKindId = u32;
/// An identifier for operator extra information
pub const OperatorExtraId = u32;
```

An operator kind contains what is required to define a local operator at the CFT level.

```
/// Defines the various kinds of operators
pub const OperatorKind = struct {
    holomorphicity: Holomorphicity,
    conformal_weights: ConformalWeights,
    quantum_numbers: tensor.QuantumNumbers,
    supported_correlator_tactics: []const CorrelatorTactic,
};
```

In the case of interest, that means defininining whether the operator is holomorphic/antiholomorphic or both,

```
/// Determines whether an operator is holomorphic/antiholomorphic or both
pub const Holomorphicity = enum {
    holomorphic,
    antiholomorphic,
    both,
};
```

its conformal weight(s) that can either be fixed at compile time or computed at runtime

```
/// ConformalWeights stores fixed weights or a rule for computing them.
pub const ConformalWeights = union(enum) {
    fixed: FixedConformalWeights,
    computed: WeightRuleId,
};

/// FixedConformalWeights stores optional left and right conformal weights.
pub const FixedConformalWeights = struct {
    h: ?f16,
    hbar: ?f16,
};

/// WeightRuleId names a rule that computes conformal weights from labels.
```

```
pub const WeightRuleId = u16;
```

and lastly what quantum numbers supported by Tensor-Code it carries and what tactics of correlator computation it supports.

Currently, the supported correlator tactics are to either keep the correlator abstract (only return zero if selection rule is no met, otherwise keep in abstract form), evaluate it via Wick contractions of input operators or bosonize the input operators first and then evaluate via Wick contractions.

```
/// A tactic for the the evaluation of correlators
pub const CorrelatorTactic = enum {
    abstract,
    wick,
    bosonize,
};
```

The operator insertion point be denoted by a single point such as z or $\bar{z}$, or both $z, \bar{z}$, which are typed as variables from Coefficient. We bundle the derivatives of a local operator with the insertion point.

```
/// Operator insertion data are specified by a position and a number of derivatives
pub const OperatorInsertion = union(enum) {
    single: struct {
        position: coefficient.Variable,
        derivatives: u8,
        },
    pair: struct {
        holomorphic_position: coefficient.Variable,
        antiholomorphic_position: coefficient.Variable,
        holomorphic_derivatives: u8,
        antiholomorphic_derivatives: u8,
    },
};
```

1.2 Coefficient

Interface

Functions

`rational(numerator: i64, denominator: i64) ?Rational`
rational constructs a normalized exact rational number.

`integer(value: i64) Rational`
integer constructs an exact rational integer.

Types

`Rational (struct)`
Rational stores one normalized exact rational number.

`CoordinateDifference (struct)`
CoordinateDifference stores one ordered coordinate difference.

`CoordinateKernel (union)`
CoordinateKernel stores one coordinate-dependent kernel factor.

`CoordinateFactor (struct)`

CoordinateFactor stores a coordinate kernel with its rational derivative multiplier.

`Coeff (struct)`

A coefficient is a sum of rational functions multiplied by a tensor structure

`Variable = u32`

Variables are internally assigned numeric identifiers

```
const tensor = @import("tensor-code");
```

A coefficient is a rational function multiplied by tensor structures from Tensor-Code. The full expression store will grow later, but Wick evaluation already needs a small exact vocabulary for scalar multipliers and coordinate kernels. These records are deliberately plain values so primitive contractions can pass them around without allocation.

```
/// Rational stores one normalized exact rational number.
pub const Rational = struct {
    numerator: i64,
    denominator: i64,
};

fn absInt(value: i64) u64 {
    if (value == @import("std").math.minInt(i64)) return @as(u64, 1) << 63;
    return if (value < 0) @intCast(-value) else @intCast(value);
}

fn gcd(first: u64, second: u64) u64 {
    var a = first;
    var b = second;
    while (b != 0) {
        const rem = a % b;
        a = b;
        b = rem;
    }
    return if (a == 0) 1 else a;
}

/// rational constructs a normalized exact rational number.
pub fn rational(numerator: i64, denominator: i64) ?Rational {
    if (denominator == 0) return null;
    var num = numerator;
    var den = denominator;
    if (den < 0) {
        num = -num;
        den = -den;
    }
    const factor: i64 = @intCast(gcd(absInt(num), absInt(den)));
    return .{
        .numerator = @divExact(num, factor),
        .denominator = @divExact(den, factor),
    };
}

/// integer constructs an exact rational integer.
pub fn integer(value: i64) Rational {
    return .{ .numerator = value, .denominator = 1 };
}
```

Coordinate kernels are the coordinate-dependent pieces produced by Wick contractions. They are not a simplifier; they are the compact output of a single primitive contraction after insertion derivatives have acted where the Wick rule requested them.

```
/// CoordinateDifference stores one ordered coordinate difference.
pub const CoordinateDifference = struct {
    left: Variable,
    right: Variable,
};

/// CoordinateKernel stores one coordinate-dependent kernel factor.
pub const CoordinateKernel = union(enum) {
    difference_power: struct { coordinate: CoordinateDifference, exponent: i16 },
    logarithm: CoordinateDifference,
    green_kernel: struct { name: []const u8, coordinate: CoordinateDifference,
    ↪  left_derivatives: u8 = 0, right_derivatives: u8 = 0 },
    green_exponential: CoordinateDifference,
};

/// CoordinateFactor stores a coordinate kernel with its rational derivative multiplier.
pub const CoordinateFactor = struct {
    scalar: Rational = .{ .numerator = 1, .denominator = 1 },
    kernel: CoordinateKernel,
};
```

The older coefficient shell is kept because higher layers already refer to a coefficient as a sum of tensor-weighted rational functions. Its internals remain private until the real simplifier exists.

```
/// A coefficient is a sum of rational functions multiplied by a tensor structure
pub const Coeff = struct {
    summands: []const CoeffSummand,
};

const CoeffSummand = struct {
    rational_function: RationalFunction,
    tensor_structure: tensor.TensorStructure,
};
```

A rational function can be written as a product of polynomial factors raised to a power.

```
const RationalFunction = struct {
    factors: []const Factor,
};

const Factor = struct {
    base: Polynomial,
    exponent: i16,
};
```

A polynomial is a sum of monomial terms.

```
const Polynomial = struct {
    terms: []const PolynomialTerm,
};

const PolynomialTerm = struct {
    monomial_coefficient: Scalar,
    monomial: Monomial,
};

const Scalar = f32;

const Monomial = struct {
    terms: []const MonomialTerm,
};

const MonomialTerm = struct {
```

```
    variable: Variable,
    power: u16,
};
```

Variables are internally assigned a numeric identifier.

```
/// Variables are internally assigned numeric identifiers
pub const Variable = u32;
```

2 CFT Kernel

The kernel is the runtime executor for a CFT preset. It does not define what a CFT is; that belongs to Theory. The kernel receives a theory declaration at `comptime`, builds a generated type from it, and exposes only the small runtime operations that Correlators, OPE, and Basis-Generation need.

```
const operators = @import("expressions/operators.zig");
const theory = @import("theory/theory.zig");
const tensor = @import("tensor-code");
```

INTERFACE

FUNCTIONS

`CFTKernel(comptime spec: theory.Spec.CFT) type`
: CFTKernel returns the generated kernel type for a bulk CFT preset.

`BCFTKernel(comptime Bulk: type, comptime bcft: theory.Spec.BCFT) type`
: BCFTKernel returns the generated kernel type for a BCFT extension.

TYPES

`Stream (struct)`
: Stream groups coefficient and projected-local-term records emitted by kernels.

`Call (struct)`
: Call groups compact input records passed to generated kernels.

2.1 Streamed Output

A correlator or OPE should emit terms as soon as possible instead of materializing a large Expression. The stream namespace holds the small output records used by sinks.

```
/// Stream groups coefficient and projected-local-term records emitted by kernels.
pub const Stream = struct {
    /// Scalar is the numeric coefficient type used by the kernel skeleton.
    pub const Scalar = f64;
    /// CoordinateFactor names a coordinate factor in a coefficient store.
    pub const CoordinateFactor = u32;

    /// CoeffTerm is one streamed scalar-coordinate-tensor coefficient term.
    pub const CoeffTerm = struct {
        scalar: Scalar,
        coordinate_factor: CoordinateFactor,
        tensor_structure: ?tensor.TensorExprId,
    };

    /// LocalTerm is one streamed projected local OPE term.
```

```
    pub const LocalTerm = struct {
        coeff: CoeffTerm,
        insertion: operators.OperatorInsertion,
        body: operators.OperatorBodyId,
    };
};
```

2.2 Runtime Calls

The public OPE and correlator accept any number of local Operators through a `MultiOp`. At runtime a local operator is only its insertion, operator kind, and a span into an immutable label store. The typed preset builder will create these spans; the kernel only needs compact data for fast Wick and zero-mode passes.

Projection and basis queries are call data; Theory owns the quantum-number, weight, domain, and mode ids they refer to.

```
/// Call groups compact input records passed to generated kernels.
pub const Call = struct {
    /// RationalLabel stores one exact rational operator label.
    pub const RationalLabel = struct {
        numerator: i64,
        denominator: i64,
    };

    /// LabelValue stores one compact runtime operator label.
    pub const LabelValue = union(enum) {
        integer: i64,
        rational: RationalLabel,
        tensor: tensor.TensorExprId,
        symbol: u32,
    };

    /// LabelStore is the immutable label arena carried by a MultiOp.
    pub const LabelStore = struct {
        values: []const LabelValue,
    };

    /// LocalOp is one runtime local operator with labels stored out-of-line.
    pub const LocalOp = struct {
        insertion: operators.OperatorInsertion,
        kind: operators.OperatorKindId,
        labels: theory.Id.LabelSpan,
    };

    /// MultiOp is an ordered collection of local operators and their label store.
    pub const MultiOp = struct {
        operators: []const LocalOp,
        labels: *const LabelStore,
    };

    /// OpeProjection stores the target and filters for projected OPE output.
    pub const OpeProjection = struct {
        target: OpeTarget,
        target_weights: ?theory.Runtime.ConformalWeights,
        quantum_filters: []const theory.Label.QuantumFilter,
        max_level: ?theory.Runtime.LevelBound,
    };

    /// OpeTarget tells the OPE where the projected local term is inserted.
    pub const OpeTarget = union(enum) {
        first_insertion,
```

```
        explicit: operators.OperatorInsertion,
    };

    /// BasisQuery stores filters for streamed operator or state generation.
    pub const BasisQuery = struct {
        weights: ?theory.Runtime.ConformalWeights,
        quantum_filters: []const theory.Label.QuantumFilter,
        domain: ?theory.Runtime.OperatorDomain,
        support: ?theory.Runtime.ChiralSupport,
        max_level: ?theory.Runtime.LevelBound,
        finite_projections: []const theory.Id.FiniteProjection,
        descendants: DescendantFilter,
    };

    /// DescendantFilter limits which mode descendants basis generation may use.
    pub const DescendantFilter = struct {
        mode_algebras: []const theory.Id.ModeAlgebra,
        max_level: ?theory.Runtime.LevelBound,
    };

    /// BasisState is one streamed candidate body with computed weights.
    pub const BasisState = struct {
        body: operators.OperatorBodyId,
        weights: theory.Runtime.ConformalWeights,
    };
};
```

2.3 Body Construction

The body store is generated from a theory operator schema. Later implementations will validate labels and canonicalize normal products here; the first version only build-checks the boundary.

```
const Body = struct {
    const Schema = struct {
        bulk_kinds: []const operators.OperatorKind,
        boundary_kind_ids: []const operators.OperatorKindId = &.{},
        boundary_changing_kinds: []const theory.Boundary.ChangingKind = &.{},
    };

    const CanonicalResult = struct {
        coeff: Stream.Scalar,
        body: ?operators.OperatorBodyId,
    };

    fn Store(comptime schema: Schema) type {
        return struct {
            const operator_schema = schema;

            /// atomic constructs an atomic operator body handle.
            pub fn atomic(kind: operators.OperatorKindId, labels: theory.Id.LabelSpan)
            ↪  !operators.OperatorBodyId {
                _ = labels;
                return @as(operators.OperatorBodyId, kind);
            }

            /// normalOrdered constructs a canonical normal-product body handle.
            pub fn normalOrdered(factors: []const operators.OperatorBodyId) !CanonicalResult {
                if (factors.len == 0) {
                    return .{ .coeff = 1, .body = null };
                }
                return .{ .coeff = 1, .body = factors[0] };
            }
```

```
        };
    }
};
```

The store is a generated type because its schema is preset data. This is the right use of `comptime`: it specializes a hot construction boundary without moving runtime operator labels or generated basis states to compile time.

2.4 Generated Kernels

The generated bulk kernel owns no boundary data. It receives a bulk `Spec.CFT`, exposes public runtime operations, and keeps preset tables private to the generated type.

```
/// CFTKernel returns the generated kernel type for a bulk CFT preset.
pub fn CFTKernel(comptime spec: theory.Spec.CFT) type {
    return struct {
        const cft_spec = spec;
        const bulk_operator_kinds = spec.bulk_operator_kinds;

        /// Bodies constructs operator bodies using this preset's schema.
        pub const Bodies = Body.Store(.{
            .bulk_kinds = spec.bulk_operator_kinds,
        });

        /// ope computes the projected OPE of any number of local operators.
        pub fn ope(input: Call.MultiOp, projection: Call.OpeProjection, sink: anytype) !void {
            _ = input;
            _ = projection;
            _ = sink;
            return error.NotImplemented;
        }

        /// correlator evaluates any number of local operators with a config.
        pub fn correlator(config: *const theory.Runtime.CorrelatorConfig, input: Call.MultiOp,
        ↪  sink: anytype) !void {
            _ = config;
            _ = input;
            _ = sink;
            return error.NotImplemented;
        }

        /// basis streams operator or state candidates satisfying a query.
        pub fn basis(query: Call.BasisQuery, sink: anytype) !void {
            _ = query;
            _ = sink;
            return error.NotImplemented;
        }
    };
}
```

The BCFT generated kernel extends an already generated bulk kernel. Boundary data enters only through the `Spec.BCFT` supplied here.

```
/// BCFTKernel returns the generated kernel type for a BCFT extension.
pub fn BCFTKernel(comptime Bulk: type, comptime bcft: theory.Spec.BCFT) type {
    return struct {
        const bulk = Bulk;
        const bcft_spec = bcft;

        /// Bodies constructs bulk, boundary, and boundary-changing bodies.
        pub const Bodies = Body.Store(.{
```

```
            .bulk_kinds = Bulk.bulk_operator_kinds,
            .boundary_kind_ids = bcft.boundary_operator_kinds,
            .boundary_changing_kinds = bcft.boundary_changing_kinds,
        });

        /// ope computes the projected OPE for this BCFT extension.
        pub fn ope(input: Call.MultiOp, projection: Call.OpeProjection, sink: anytype) !void {
            _ = input;
            _ = projection;
            _ = sink;
            return error.NotImplemented;
        }

        /// correlator evaluates bulk and boundary local operators with a config.
        pub fn correlator(config: *const theory.Runtime.CorrelatorConfig, input: Call.MultiOp,
        ↪  sink: anytype) !void {
            _ = config;
            _ = input;
            _ = sink;
            return error.NotImplemented;
        }

        /// basis streams bulk, boundary, or boundary-changing candidates.
        pub fn basis(query: Call.BasisQuery, sink: anytype) !void {
            _ = query;
            _ = sink;
            return error.NotImplemented;
        }
    };
}
```

3 Presets

Interface

Types

`Handle = shared.Handle`
: Handle groups compact ids used by preset operator builders.

`FreeBoson (struct)`
: FreeBoson groups the free-boson preset constructors and configs.

`Bc (struct)`
: Bc groups the bc-ghost preset constructors and configs.

Constants

`scalars = shared.scalars`
: scalars exposes basic algebra for compact structured rule scalars.

`product = composition.product`
: product builds the generated preset type for a product of independent bulk presets.

`boundary = composition.boundary`
: boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.

Presets are the physics-facing constructors for common worldsheet CFT data. They sit between Theory and CFT Kernel. A user asks for a free boson, a *bc* system, a product theory, or a boundary extension, and receives a generated type with a small operator namespace and named correlator configs.

The implementation is split by audit target:

- Preset Shared Runtime defines compact handles, local-operator input, Wick templates, zero-mode templates, and the streaming correlator boundary.
- Free Boson Preset defines the free-boson bulk and Neumann/Dirichlet disk data.
- bc Preset defines the *bc* sphere and disk data.
- Preset Composition joins bulk sectors and boundary extensions without introducing a new runtime operator representation.

This node is an internal preset bridge used by CFT-Code. It is not the user API. It re-exports only the declarations the root CFT API needs to build compact family objects such as `FreeBoson` and `Bc`. Raw Wick-rule data, correlator config records, and runtime resolver types stay in Preset Shared Runtime for implementation modules.

```
const shared = @import("presets/shared.zig");
const free_boson = @import("presets/free_boson.zig");
const bc = @import("presets/bc.zig");
const composition = @import("presets/composition.zig");

const freeBoson = free_boson.freeBoson;
const bcSphere = bc.bcSphere;
const freeBosonBoundary = free_boson.boundaryExtension;
const bcDiskBoundary = bc.boundaryExtension;
```

3.1 Shared Bridge

Only handle construction and scalar convention helpers cross from the shared runtime into the root public API.

```
/// Handle groups compact ids used by preset operator builders.
pub const Handle = shared.Handle;
/// scalars exposes basic algebra for compact structured rule scalars.
pub const scalars = shared.scalars;
```

3.2 Preset Families

The constructor bridge is grouped by CFT family. The root public API can re-export these family objects without proliferating flat constructor and config names.

```
/// FreeBoson groups the free-boson preset constructors and configs.
pub const FreeBoson = struct {
    /// Config declares a noncompact D-dimensional free-boson preset.
    pub const Config = free_boson.FreeBosonConfig;
    /// BoundaryConfig declares Neumann and Dirichlet data for a brane.
    pub const BoundaryConfig = free_boson.FreeBosonBoundaryConfig;

    /// make builds the generated preset type for a noncompact free-boson CFT.
    pub const make = free_boson.freeBoson;
    /// boundary declares the Neumann/Dirichlet free-boson boundary extension.
    pub const boundary = free_boson.boundaryExtension;
};

/// Bc groups the bc-ghost preset constructors and configs.
pub const Bc = struct {
    /// SphereConfig declares the holomorphic bc system and optional antiholomorphic copy.
    pub const SphereConfig = bc.BcSphereConfig;
```

```
    /// DiskConfig declares the disk boundary extension for the bc ghost system.
    pub const DiskConfig = bc.BcDiskConfig;

    /// sphere builds the generated preset type for the sphere bc ghost CFT.
    pub const sphere = bc.bcSphere;
    /// diskBoundary declares boundary and mixed bulk-boundary bc rules on the disk.
    pub const diskBoundary = bc.boundaryExtension;
};

/// product builds the generated preset type for a product of independent bulk presets.
pub const product = composition.product;
/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
↪  extensions.
pub const boundary = composition.boundary;
```

3.3 Preset Shared Runtime

Interface

Functions

`profileHandle(comptime presentation: Handle.ProfilePresentation, payload: u32) Handle.Profile`
profileHandle packs a profile presentation tag with a compact payload id.

`profilePresentation(profile: Handle.Profile) Handle.ProfilePresentation`
profilePresentation reads the presentation tag from a profile handle.

`profilePayload(profile: Handle.Profile) u32`
profilePayload reads the presentation-local payload id from a profile handle.

`configRef(comptime namespace: []const u8, comptime name: []const u8) ConfigRef`
configRef derives a stable id for correlator configuration data.

`familyKind(comptime namespace: []const u8, comptime family: anytype) operators.OperatorKindId`
familyKind derives a compact operator kind id from a preset namespace and local family enum.

`sectorId(comptime namespace: []const u8, comptime sector: anytype) u32`
sectorId derives a compact zero-mode sector id from a preset namespace and local sector enum.

`extensionKind(comptime namespace: []const u8) u32`
extensionKind derives a compact boundary-extension id from a preset namespace.

`wickRuleIndexStorage(comptime rules: []const wick.Rule) [wickRuleIndexEntryCount(rules)]WickRu`
wickRuleIndexStorage builds a sorted lookup table for primitive Wick rules.

`emitZeroModeBaseCase(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, residual_i`
emitZeroModeBaseCase consumes residual operators with configured zero-mode rules.

`emitWickPairTerms(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, left_index: us`
emitWickPairTerms emits complete primitive Wick terms for two input positions.

`streamCorrelator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink: anytype)`
streamCorrelator emits pair Wick events and then tries the zero-mode base case.

Types

`Handle (struct)`
Handle groups compact ids used by preset operator builders.

`ScalarAtom = u32`
ScalarAtom names one scalar generator.

`ConfigRef = u32`
: ConfigRef names one entry in a correlator configuration payload.

`RationalScalar (struct)`
: RationalScalar stores one exact rational scalar multiplier.

`ScalarMonomial (struct)`
: ScalarMonomial stores a compact rational times i-power times one scalar atom power.

`RuleScalar (union)`
: RuleScalar stores small structured scalar factors used by preset builders.

`Local (struct)`
: Local builds typed handles, local operators, and the MultiOp label store.

`WickRuleIndexEntry (struct)`
: WickRuleIndexEntry maps an ordered operator-kind pair to one primitive Wick rule.

`CorrelatorConfig (struct)`
: CorrelatorConfig selects actual Wick and zero-mode rule declarations.

`ConfigValue (union)`
: ConfigValue stores one typed value referenced by Wick or zero-mode rules.

`ConfigEntry (struct)`
: ConfigEntry binds one compact config id to one typed value.

`BoundaryExtension (struct)`
: BoundaryExtension carries one preset-authored boundary operator namespace and rule slices.

Constants

`scalars = ScalarDsl`
: scalars exposes basic algebra for compact structured rule scalars.

`wick = WickDsl`
: wick exposes compact preset-author helpers for primitive Wick rule templates.

`zero_mode = ZeroModeDsl`
: zero_mode exposes compact preset-author helpers for zero-mode rules.

This node contains the small runtime vocabulary shared by the preset constructors in Free Boson Preset, bc Preset, and Preset Composition. It deliberately does not define a specific CFT. It gives preset authors compact handles, local-operator construction, Wick rule templates, zero-mode rule templates, and the first streaming correlator driver.

The data path is:

1. A user creates typed handles with `Local`.
2. Preset-owned operator builders choose their own operator kind ids and store only compact labels plus insertion data in a CFT Kernel `MultiOp`.
3. A preset supplies a `CorrelatorConfig` containing Wick and zero-mode rules.
4. The streaming driver emits matched rules to a sink without building the full correlator expression in memory.

```
const std = @import("std");
const operators = @import("../expressions/operators.zig");
const coefficient = @import("../expressions/coefficient.zig");
const kernel = @import("../kernel.zig");
const theory = @import("../theory/theory.zig");
```

3.3.1 Handles

Handles are compact integer-backed tokens. The user sees physics names such as `momentum("k")` or `index("mu")`, but the kernel receives only small ids. This keeps operator input cheap to copy and easy to stream.

```
/// Handle groups compact ids used by preset operator builders.
pub const Handle = struct {
    /// Coord names a holomorphic or antiholomorphic coordinate variable.
    pub const Coord = enum(u32) { _ };
    /// BoundaryCoord names a real boundary coordinate variable.
    pub const BoundaryCoord = enum(u32) { _ };
    /// Index names a target-space vector index.
    pub const Index = enum(u32) { _ };
    /// Momentum names a target-space momentum expression.
    pub const Momentum = enum(u32) { _ };
    /// Profile names a selected presentation of a target-space profile.
    pub const Profile = enum(u32) { _ };
    /// TargetFunction names a position-space target profile.
    pub const TargetFunction = enum(u32) { _ };
    /// FourierTransform names a Fourier-space target profile.
    pub const FourierTransform = enum(u32) { _ };
    /// TargetPoint names a point in target space.
    pub const TargetPoint = enum(u32) { _ };
    /// ProfileCoefficient names one coefficient in a polynomial profile.
    pub const ProfileCoefficient = enum(u32) { _ };
    /// TensorProjector names a tensor-code projector expression.
    pub const TensorProjector = enum(u32) { _ };
    /// BoundaryStack names stacked boundary-condition Chan-Paton data.
    pub const BoundaryStack = enum(u32) { _ };
    /// PolynomialProfileTerm stores one finite polynomial profile term.
    pub const PolynomialProfileTerm = struct {
        coefficient: ProfileCoefficient,
        power: u16,
    };
    /// ProfilePresentation records how a profile handle should be lowered.
    pub const ProfilePresentation = enum(u2) {
        position_space,
        fourier,
        polynomial_rnc,
    };
};
```

The deterministic token helpers are for compile-time preset data such as target-space projectors. Runtime handles come from `Local` and are fresh within the multi-local insertion being built.

```
fn rawToken(value: anytype) u32 {
    return @intFromEnum(value);
}

fn token(comptime T: type, id: u32) T {
    return @enumFromInt(if (id == 0) 1 else id);
}

const profile_tag_shift = 30;
const profile_payload_mask: u32 = 0x3fff_ffff;

/// profileHandle packs a profile presentation tag with a compact payload id.
pub fn profileHandle(comptime presentation: Handle.ProfilePresentation, payload: u32)
↪   Handle.Profile {
    const tag = @as(u32, @intFromEnum(presentation)) << profile_tag_shift;
    return token(Handle.Profile, tag | (payload & profile_payload_mask));
```

```
}

/// profilePresentation reads the presentation tag from a profile handle.
pub fn profilePresentation(profile: Handle.Profile) Handle.ProfilePresentation {
    return @enumFromInt(@intFromEnum(profile) >> profile_tag_shift);
}

/// profilePayload reads the presentation-local payload id from a profile handle.
pub fn profilePayload(profile: Handle.Profile) u32 {
    return @intFromEnum(profile) & profile_payload_mask;
}

fn stableId(comptime namespace: []const u8, comptime name: []const u8) u32 {
    var hash: u32 = 2166136261;
    inline for (namespace) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    hash = (hash ^ @as(u32, ':')) *% 16777619;
    inline for (name) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

/// configRef derives a stable id for correlator configuration data.
pub fn configRef(comptime namespace: []const u8, comptime name: []const u8) ConfigRef {
    return stableId(namespace, name);
}
```

3.3.2 Rule Scalars

Preset Wick rules should not encode scalar constants as compound strings. A scalar factor is a small monomial: a rational number, an optional power of i, and an optional named scalar atom such as α'. The atom id is stable and compact; the coefficient evaluator can attach display names or numerical values later.

```
/// ScalarAtom names one scalar generator.
pub const ScalarAtom = u32;

/// ConfigRef names one entry in a correlator configuration payload.
pub const ConfigRef = u32;

/// RationalScalar stores one exact rational scalar multiplier.
pub const RationalScalar = struct {
    numerator: i64,
    denominator: i64,
};

/// ScalarMonomial stores a compact rational times i-power times one scalar atom power.
pub const ScalarMonomial = struct {
    rational: RationalScalar = .{ .numerator = 1, .denominator = 1 },
    imaginary_power: u2 = 0,
    atom: ?ScalarAtom = null,
    atom_power: i8 = 0,
};

/// RuleScalar stores small structured scalar factors used by preset builders.
pub const RuleScalar = union(enum) {
    one,
    rational: RationalScalar,
    monomial: ScalarMonomial,
};
```

```
/// scalars exposes basic algebra for compact structured rule scalars.
pub const scalars = ScalarDsl;

const ScalarDsl = struct {
    fn absInt(comptime value: i64) u64 {
        if (value == std.math.minInt(i64)) @compileError("cannot normalize minInt(i64)
        ↪  rational component");
        return if (value < 0) @intCast(-value) else @intCast(value);
    }

    fn gcd(comptime first: u64, comptime second: u64) u64 {
        var a = first;
        var b = second;
        while (b != 0) {
            const rem = a % b;
            a = b;
            b = rem;
        }
        return if (a == 0) 1 else a;
    }

    fn rationalValue(comptime numerator: i64, comptime denominator: i64) RationalScalar {
        if (denominator == 0) @compileError("rational denominator cannot be zero");
        var num = numerator;
        var den = denominator;
        if (den < 0) {
            num = -num;
            den = -den;
        }
        const factor: i64 = @intCast(gcd(absInt(num), absInt(den)));
        return .{
            .numerator = @divExact(num, factor),
            .denominator = @divExact(den, factor),
        };
    }

    /// atom derives a stable scalar atom id from a preset namespace and atom name.
    pub fn atom(comptime namespace: []const u8, comptime name: []const u8) ScalarAtom {
        return stableId(namespace, name);
    }

    /// one constructs the multiplicative unit scalar.
    pub fn one() RuleScalar {
        return .one;
    }

    /// rational constructs an exact rational scalar.
    pub fn rational(comptime numerator: i64, comptime denominator: i64) RuleScalar {
        return .{ .rational = rationalValue(numerator, denominator) };
    }

    /// atomScalar constructs a scalar consisting of one named atom.
    pub fn atomScalar(comptime atom_id: ScalarAtom) RuleScalar {
        return monomial(1, 1, 0, atom_id, 1);
    }

    /// monomial constructs a rational times i-power times atom-power scalar.
    pub fn monomial(comptime numerator: i64, comptime denominator: i64, comptime
    ↪  imaginary_power: u2, comptime atom_id: ?ScalarAtom, comptime atom_power: i8)
    ↪  RuleScalar {
        return .{ .monomial = .{
            .rational = rationalValue(numerator, denominator),
            .imaginary_power = imaginary_power,
```

```
            .atom = atom_id,
            .atom_power = atom_power,
        } };
    }

    fn monomialFrom(comptime value: RuleScalar) ScalarMonomial {
        return switch (value) {
            .one => .{},
            .rational => |r| .{ .rational = r },
            .monomial => |m| m,
        };
    }

    /// scale multiplies a scalar by an exact rational number.
    pub fn scale(comptime value: RuleScalar, comptime numerator: i64, comptime denominator:
    ↪  i64) RuleScalar {
        var factor = monomialFrom(value);
        factor.rational = rationalValue(factor.rational.numerator * numerator,
        ↪  factor.rational.denominator * denominator);
        return .{ .monomial = factor };
    }

    /// neg negates a scalar.
    pub fn neg(comptime value: RuleScalar) RuleScalar {
        return scale(value, -1, 1);
    }

    /// div divides a scalar by an integer denominator.
    pub fn div(comptime value: RuleScalar, comptime denominator: i64) RuleScalar {
        return scale(value, 1, denominator);
    }

    /// i multiplies a scalar by the imaginary unit.
    pub fn i(comptime value: RuleScalar) RuleScalar {
        var factor = monomialFrom(value);
        factor.imaginary_power +%= 1;
        return .{ .monomial = factor };
    }
};
```

3.3.3 Local Operators

`Local` is only a builder for one `MultiOp`. It is not a theory context and it does not own Wick rules. Its job is to place labels out-of-line and keep local operators fixed-width enough for the hot correlator path.

Operator families are not decided here. Each preset owns its physics vocabulary, for example free-boson or ghost fields, and calls `familyKind` to derive compact kind ids from that preset-local enum. Shared code only sees the resulting `OperatorKindId`.

```
/// familyKind derives a compact operator kind id from a preset namespace and local family
↪  enum.
pub fn familyKind(comptime namespace: []const u8, comptime family: anytype)
↪  operators.OperatorKindId {
    return stableId(namespace, @tagName(family));
}

/// sectorId derives a compact zero-mode sector id from a preset namespace and local sector
↪  enum.
pub fn sectorId(comptime namespace: []const u8, comptime sector: anytype) u32 {
    return stableId(namespace, @tagName(sector));
}
```

```
/// extensionKind derives a compact boundary-extension id from a preset namespace.
pub fn extensionKind(comptime namespace: []const u8) u32 {
    return stableId("boundary_extension", namespace);
}

/// Local builds typed handles, local operators, and the MultiOp label store.
pub const Local = struct {
    allocator: std.mem.Allocator,
    labels: std.ArrayList(kernel.Call.LabelValue) = .empty,
    operators_list: std.ArrayList(kernel.Call.LocalOp) = .empty,
    owned_labels: []kernel.Call.LabelValue = &.{},
    owned_operators: []kernel.Call.LocalOp = &.{},
    label_store: kernel.Call.LabelStore = .{ .values = &.{} },
    next_symbol: u32 = 1,
    finished: bool = false,

    /// init constructs an empty local-operator builder.
    pub fn init(allocator: std.mem.Allocator) Local {
        return .{ .allocator = allocator };
    }

    /// deinit releases memory owned by this local builder and its frozen MultiOp.
    pub fn deinit(self: *Local) void {
        if (self.finished) {
            self.allocator.free(self.owned_labels);
            self.allocator.free(self.owned_operators);
        } else {
            self.labels.deinit(self.allocator);
            self.operators_list.deinit(self.allocator);
        }
        self.* = Local.init(self.allocator);
    }

    fn next(self: *Local) u32 {
        const value = self.next_symbol;
        self.next_symbol += 1;
        return value;
    }

    /// coord creates a named bulk coordinate token.
    pub fn coord(self: *Local, name: []const u8) !Handle.Coord {
        _ = name;
        return token(Handle.Coord, self.next());
    }

    /// boundaryCoord creates a named boundary coordinate token.
    pub fn boundaryCoord(self: *Local, name: []const u8) !Handle.BoundaryCoord {
        _ = name;
        return token(Handle.BoundaryCoord, self.next());
    }

    /// index creates a target-space index token.
    pub fn index(self: *Local, name: []const u8) !Handle.Index {
        _ = name;
        return token(Handle.Index, self.next());
    }

    /// momentum creates a target-space momentum token.
    pub fn momentum(self: *Local, name: []const u8) !Handle.Momentum {
        _ = name;
        return token(Handle.Momentum, self.next());
    }

    /// targetFunction creates a position-space target-profile token.
```

```
    pub fn targetFunction(self: *Local, name: []const u8) !Handle.TargetFunction {
        _ = name;
        return token(Handle.TargetFunction, self.next());
    }

    /// fourierTransform creates a Fourier-space profile token.
    pub fn fourierTransform(self: *Local, name: []const u8) !Handle.FourierTransform {
        _ = name;
        return token(Handle.FourierTransform, self.next());
    }

    /// targetPoint creates a target-space point token.
    pub fn targetPoint(self: *Local, name: []const u8) !Handle.TargetPoint {
        _ = name;
        return token(Handle.TargetPoint, self.next());
    }

    /// profileCoefficient creates a polynomial-profile coefficient token.
    pub fn profileCoefficient(self: *Local, name: []const u8) !Handle.ProfileCoefficient {
        _ = name;
        return token(Handle.ProfileCoefficient, self.next());
    }

    fn labelSpan(self: *Local, values: []const kernel.Call.LabelValue) !theory.Id.LabelSpan {
        const start = self.labels.items.len;
        try self.labels.appendSlice(self.allocator, values);
        return @intCast(start);
    }

    /// symbol stores an enum-backed handle as a compact label value.
    pub fn symbol(value: anytype) kernel.Call.LabelValue {
        return .{ .symbol = rawToken(value) };
    }

    /// op builds one generic local operator from a preset-owned kind id and label slice.
    pub fn op(self: *Local, kind: operators.OperatorKindId, insertion:
    ↪  operators.OperatorInsertion, values: []const kernel.Call.LabelValue)
    ↪  !kernel.Call.LocalOp {
        return .{
            .insertion = insertion,
            .kind = kind,
            .labels = try self.labelSpan(values),
        };
    }

    /// add appends one local operator to this builder's MultiOp.
    pub fn add(self: *Local, item: kernel.Call.LocalOp) !void {
        if (self.finished) return error.LocalAlreadyFinished;
        try self.operators_list.append(self.allocator, item);
    }

    /// ops freezes a tuple of local operators into a MultiOp.
    pub fn ops(self: *Local, items: anytype) !kernel.Call.MultiOp {
        inline for (items) |item| {
            try self.add(item);
        }
        return self.finish();
    }

    /// finish freezes the accumulated local operators and labels into a MultiOp.
    pub fn finish(self: *Local) !kernel.Call.MultiOp {
        if (self.finished) return error.LocalAlreadyFinished;
        self.owned_operators = try self.operators_list.toOwnedSlice(self.allocator);
        self.owned_labels = try self.labels.toOwnedSlice(self.allocator);
```

```
        self.label_store = .{ .values = self.owned_labels };
        self.finished = true;
        return .{ .operators = self.owned_operators, .labels = &self.label_store };
    }
};
```

3.3.4 Wick Templates

A Wick rule is written as a pair of operator families and an expression template. The template is not a final coefficient; it is the small symbolic recipe a resolver plugs into two concrete local operators. This keeps operator matching separate from later coefficient algebra.

```
/// wick exposes compact preset-author helpers for primitive Wick rule templates.
pub const wick = WickDsl;

const WickDsl = struct {
    /// Support tells the evaluator which insertion coordinate slots it reads.
    pub const Support = enum {
        holomorphic,
        antiholomorphic,
        bulk_pair,
        boundary,
    };

    /// Side selects the left or right operator in a Wick rule expression.
    pub const Side = enum { left, right };

    /// CoordinateSlot selects the coordinate component of a matched insertion.
    pub const CoordinateSlot = enum { position, holomorphic, antiholomorphic };

    /// CoordinateRef refers to one coordinate of one side of a Wick rule.
    pub const CoordinateRef = struct {
        side: Side,
        slot: CoordinateSlot,
    };

    /// LabelRef refers to one runtime label of one side of a Wick rule.
    pub const LabelRef = struct {
        side: Side,
        slot: u8,
    };

    /// ScalarFactor describes one non-coordinate scalar multiplier.
    pub const ScalarFactor = union(enum) {
        value: RuleScalar,
        label_bilinear_phase: struct { left: LabelRef, right: LabelRef, form: []const u8 },
        cocycle: struct { table: []const u8, left: LabelRef, right: LabelRef },
        spin_structure: []const u8,
    };

    /// TensorRef identifies tensor data coming from labels or selected config data.
    pub const TensorRef = union(enum) {
        label: LabelRef,
        config: ConfigRef,
    };

    /// TensorFactor describes one tensor multiplier in a Wick coefficient.
    pub const TensorFactor = union(enum) {
        none,
        metric: struct { left: LabelRef, right: LabelRef },
        momentum_index: struct { momentum: LabelRef, index: LabelRef },
        momentum_pair: struct { left: LabelRef, right: LabelRef },
        projector_metric: struct { projector: TensorRef, left: LabelRef, right: LabelRef },
```

```
    projector_momentum_index: struct { projector: TensorRef, momentum: LabelRef, index:
    ↪ LabelRef },
    projector_momentum_pair: struct { projector: TensorRef, left: LabelRef, right:
    ↪ LabelRef },
};

/// ActionFactor describes a non-multiplicative action produced by a Wick contraction.
pub const ActionFactor = union(enum) {
    profile_derivative: struct { profile: LabelRef, index: LabelRef },
    projected_profile_derivative: struct { projector: TensorRef, profile: LabelRef, index:
    ↪ LabelRef },
};

/// CoordinateDifference is the coordinate difference read by a Wick kernel.
pub const CoordinateDifference = struct {
    left: CoordinateRef,
    right: CoordinateRef,
};

/// DerivativeAction tells resolution how insertion derivative labels act on a coordinate
↪ factor.
pub const DerivativeAction = struct {
    include_left: bool = false,
    include_right: bool = false,
};

/// CoordinateFactor describes one coordinate-dependent multiplier.
pub const CoordinateFactor = union(enum) {
    difference_power: struct { coordinate: CoordinateDifference, exponent: i16,
    ↪ derivatives: DerivativeAction = .{} },
    logarithm: struct { coordinate: CoordinateDifference, derivatives: DerivativeAction =
    ↪ .{} },
    green_kernel: struct { name: []const u8, coordinate: CoordinateDifference,
    ↪ derivatives: DerivativeAction = .{} },
    green_exponential: CoordinateDifference,
};

/// Term is one product of scalar, coordinate, tensor, and action factors.
pub const Term = struct {
    scalars: []const ScalarFactor,
    coordinates: []const CoordinateFactor,
    tensors: []const TensorFactor,
    actions: []const ActionFactor = &.{},
};

/// Expr is a sum of Wick coefficient terms.
pub const Expr = struct {
    terms: []const Term,
};

const PoleExpr = struct {
    scalar: ScalarFactor,
    numerator: TensorFactor,
    coordinate: CoordinateDifference,
    exponent: i16,
};

const GreenExponentialExpr = struct {
    scalar: ScalarFactor,
    momentum_pair: TensorFactor,
    coordinate: CoordinateDifference,
};

/// Pattern is one side of a primitive Wick rule.
```

```
    pub const Pattern = struct {
        kind: operators.OperatorKindId,
        support: Support,
    };

    /// Rule is one preset-authored primitive Wick rule.
    pub const Rule = struct {
        left: Pattern,
        right: Pattern,
        expr: Expr,
    };

    /// rule constructs one primitive Wick rule template.
    pub fn rule(left: Pattern, right: Pattern, expression: Expr) Rule {
        return .{ .left = left, .right = right, .expr = expression };
    }

    /// pattern constructs one Wick-rule side from a preset-owned kind id.
    pub fn pattern(comptime kind: operators.OperatorKindId, comptime support: Support) Pattern
    ↪ {
        return .{ .kind = kind, .support = support };
    }

    /// coord refers to one coordinate slot of one Wick-rule side.
    pub fn coord(side: Side, slot: CoordinateSlot) CoordinateRef {
        return .{ .side = side, .slot = slot };
    }

    /// label refers to one label slot of one Wick-rule side.
    pub fn label(side: Side, slot: u8) LabelRef {
        return .{ .side = side, .slot = slot };
    }

    /// difference constructs a coordinate difference.
    pub fn difference(left: CoordinateRef, right: CoordinateRef) CoordinateDifference {
        return .{ .left = left, .right = right };
    }

    /// scalar constructs a scalar factor from a rule scalar.
    pub fn scalar(comptime value: RuleScalar) ScalarFactor {
        return .{ .value = value };
    }

    /// term constructs one product term in a Wick expression.
    pub fn term(comptime scalar_factors: []const ScalarFactor, comptime coordinates: []const
    ↪  CoordinateFactor, comptime tensors: []const TensorFactor) Term {
        return .{ .scalars = scalar_factors, .coordinates = coordinates, .tensors = tensors };
    }

    /// termWithActions constructs one Wick term that also carries operator actions.
    pub fn termWithActions(comptime scalar_factors: []const ScalarFactor, comptime
    ↪  coordinates: []const CoordinateFactor, comptime tensors: []const TensorFactor,
    ↪  comptime actions: []const ActionFactor) Term {
        return .{ .scalars = scalar_factors, .coordinates = coordinates, .tensors = tensors,
        ↪  .actions = actions };
    }

    /// expr constructs a Wick expression from one or more terms.
    pub fn expr(comptime terms: []const Term) Expr {
        return .{ .terms = terms };
    }

    /// profileDerivative constructs the action of a target derivative on a profile label.
```

```
    pub fn profileDerivative(comptime profile: LabelRef, comptime index: LabelRef)
    ↪  ActionFactor {
        return .{ .profile_derivative = .{ .profile = profile, .index = index } };
    }

    /// projectedProfileDerivative constructs a projected target derivative on a profile
    ↪  label.
    pub fn projectedProfileDerivative(comptime projector: TensorRef, comptime profile:
    ↪  LabelRef, comptime index: LabelRef) ActionFactor {
        return .{ .projected_profile_derivative = .{ .projector = projector, .profile =
        ↪  profile, .index = index } };
    }

    /// differencePower constructs a power of a coordinate difference.
    pub fn differencePower(comptime coordinate: CoordinateDifference, comptime exponent: i16)
    ↪  CoordinateFactor {
        return .{ .difference_power = .{ .coordinate = coordinate, .exponent = exponent } };
    }

    /// differentiatedPower applies matched insertion derivative labels to a coordinate power.
    pub fn differentiatedPower(comptime coordinate: CoordinateDifference, comptime exponent:
    ↪  i16, comptime derivatives: DerivativeAction) CoordinateFactor {
        return .{ .difference_power = .{ .coordinate = coordinate, .exponent = exponent,
        ↪  .derivatives = derivatives } };
    }

    /// logarithm constructs a logarithm of a coordinate difference.
    pub fn logarithm(comptime coordinate: CoordinateDifference) CoordinateFactor {
        return .{ .logarithm = .{ .coordinate = coordinate } };
    }

    /// differentiatedLogarithm applies matched insertion derivative labels to a logarithm.
    pub fn differentiatedLogarithm(comptime coordinate: CoordinateDifference, comptime
    ↪  derivatives: DerivativeAction) CoordinateFactor {
        return .{ .logarithm = .{ .coordinate = coordinate, .derivatives = derivatives } };
    }

    /// greenKernel constructs a named Green-kernel coordinate factor.
    pub fn greenKernel(comptime name: []const u8, comptime coordinate: CoordinateDifference)
    ↪  CoordinateFactor {
        return .{ .green_kernel = .{ .name = name, .coordinate = coordinate } };
    }

    /// differentiatedGreenKernel records derivative labels acting on a named Green kernel.
    pub fn differentiatedGreenKernel(comptime name: []const u8, comptime coordinate:
    ↪  CoordinateDifference, comptime derivatives: DerivativeAction) CoordinateFactor {
        return .{ .green_kernel = .{ .name = name, .coordinate = coordinate, .derivatives =
        ↪  derivatives } };
    }

    /// pole constructs a primitive pole coefficient.
    pub fn pole(comptime scalar_value: RuleScalar, comptime numerator: TensorFactor, comptime
    ↪  coordinate: CoordinateDifference, comptime exponent: i16) Expr {
        const shape = PoleExpr{
            .scalar = scalar(scalar_value),
            .numerator = numerator,
            .coordinate = coordinate,
            .exponent = exponent,
        };
        return expr(&.{term(&.{shape.scalar}, &.{differencePower(shape.coordinate,
        ↪  shape.exponent)}, &.{shape.numerator})});
    }

    /// differentiatedPole constructs a pole acted on by matched insertion derivatives.
```

```
    pub fn differentiatedPole(comptime scalar_value: RuleScalar, comptime numerator:
    ↪  TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent: i16,
    ↪  comptime derivatives: DerivativeAction) Expr {
        const scalar_factor = scalar(scalar_value);
        return expr(&.{term(&.{scalar_factor}, &.{differentiatedPower(coordinate, exponent,
        ↪  derivatives)}, &.{numerator})});
    }

    /// differentiatedActionPole constructs a differentiated pole carrying profile or custom
    ↪  actions.
    pub fn differentiatedActionPole(comptime scalar_value: RuleScalar, comptime numerator:
    ↪  TensorFactor, comptime coordinate: CoordinateDifference, comptime exponent: i16,
    ↪  comptime derivatives: DerivativeAction, comptime actions: []const ActionFactor) Expr {
        const scalar_factor = scalar(scalar_value);
        return expr(&.{termWithActions(&.{scalar_factor}, &.{differentiatedPower(coordinate,
        ↪  exponent, derivatives)}, &.{numerator}, actions)});
    }

    /// greenExponential constructs a plane-wave Green-kernel factor.
    pub fn greenExponential(comptime scalar_value: RuleScalar, comptime momentum_pair:
    ↪  TensorFactor, comptime coordinate: CoordinateDifference) Expr {
        const shape = GreenExponentialExpr{
            .scalar = scalar(scalar_value),
            .momentum_pair = momentum_pair,
            .coordinate = coordinate,
        };
        return expr(&.{term(&.{shape.scalar}, &.{.{ .green_exponential = shape.coordinate }},
        ↪  &.{shape.momentum_pair})});
    }

};
```

3.3.5 Zero Modes

Zero-mode data is separate from Wick data. A Wick rule knows how to contract two local operators. A zero-mode rule is a base-case consumer: after Wick recursion leaves a residual operator view, each zero-mode evaluator consumes the residual operators it understands and emits concrete coefficient factors. If anything remains unconsumed, the branch is zero.

The rule payloads stay small. The *bc* evaluator is finite-dimensional CKV saturation. The free-boson evaluator emits momentum deltas, Dirichlet phases, and presentation-specific profile objects without attaching string coupling normalizations.

```
/// zero_mode exposes compact preset-author helpers for zero-mode rules.
pub const zero_mode = ZeroModeDsl;

const ZeroModeDsl = struct {
    /// Sector names a preset-owned zero-mode sector.
    pub const Sector = u32;

    /// BcSupport selects the finite c-zero-mode basis used by the bc evaluator.
    pub const BcSupport = enum {
        sphere_holomorphic,
        sphere_antiholomorphic,
        disk_doubled,
    };

    /// BcTopForm declares one top-form ghost zero-mode saturation rule.
    pub const BcTopForm = struct {
        support: BcSupport,
        c_kind_ids: []const operators.OperatorKindId,
        normalization: RuleScalar = .one,
    };
```

```
    /// RankSource records where a power of 2pi gets its exponent.
    pub const RankSource = union(enum) {
        none,
        target_dimension: ConfigRef,
        projector_rank: ConfigRef,
    };

    /// Normalization records the CFT normalization attached to a zero-mode rule.
    pub const Normalization = struct {
        scalar: RuleScalar = .one,
        two_pi_power: RankSource = .none,
    };

    /// FreeBosonConstantMode declares the free-boson constant-mode base case.
    pub const FreeBosonConstantMode = struct {
        integration_projector: ?ConfigRef = null,
        fixed_projector: ?ConfigRef = null,
        fixed_position: ?ConfigRef = null,
        exp_kind_ids: []const operators.OperatorKindId,
        profile_kind_ids: []const operators.OperatorKindId,
        normalization: Normalization = .{},
    };

    /// Expr selects the zero-mode evaluator and payload.
    pub const Expr = union(enum) {
        bc_top_form: BcTopForm,
        free_boson_constant_mode: FreeBosonConstantMode,
    };

    /// Rule is one preset-authored zero-mode rule.
    pub const Rule = struct {
        sector: Sector,
        expr: Expr,
    };

    /// rule constructs a zero-mode rule.
    pub fn rule(sector: Sector, expr: Expr) Rule {
        return .{ .sector = sector, .expr = expr };
    }
};

/// WickRuleIndexEntry maps an ordered operator-kind pair to one primitive Wick rule.
pub const WickRuleIndexEntry = struct {
    key: u64,
    rule_index: u32,
    reversed: bool,
};

/// CorrelatorConfig selects actual Wick and zero-mode rule declarations.
pub const CorrelatorConfig = struct {
    wick_rules: []const wick.Rule,
    wick_rule_index: []const WickRuleIndexEntry = &.{},
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry = &.{},
};

/// ConfigValue stores one typed value referenced by Wick or zero-mode rules.
pub const ConfigValue = union(enum) {
    tensor_projector: Handle.TensorProjector,
    target_point: Handle.TargetPoint,
    target_dimension: u16,
    boundary_stack: Handle.BoundaryStack,
};
```

```
/// ConfigEntry binds one compact config id to one typed value.
pub const ConfigEntry = struct {
    id: ConfigRef,
    value: ConfigValue,
};

/// BoundaryExtension carries one preset-authored boundary operator namespace and rule slices.
pub const BoundaryExtension = struct {
    kind: u32,
    op: type,
    wick_rules: []const wick.Rule,
    zero_modes: []const zero_mode.Rule,
    config_entries: []const ConfigEntry = &.{},
};

fn wickRuleKey(left_kind: operators.OperatorKindId, right_kind: operators.OperatorKindId) u64
↪ {
    return (@as(u64, left_kind) << 32) | @as(u64, right_kind);
}

fn wickRuleIndexEntryCount(comptime rules: []const wick.Rule) usize {
    var count: usize = 0;
    for (rules) |rule| {
        count += 1;
        if (rule.left.kind != rule.right.kind) count += 1;
    }
    return count;
}

fn sortWickRuleIndex(entries: []WickRuleIndexEntry) void {
    var index: usize = 1;
    while (index < entries.len) : (index += 1) {
        const value = entries[index];
        var hole = index;
        while (hole > 0 and entries[hole - 1].key > value.key) : (hole -= 1) {
            entries[hole] = entries[hole - 1];
        }
        entries[hole] = value;
    }
}

/// wickRuleIndexStorage builds a sorted lookup table for primitive Wick rules.
pub fn wickRuleIndexStorage(comptime rules: []const wick.Rule)
↪ [wickRuleIndexEntryCount(rules)]WickRuleIndexEntry {
    var entries: [wickRuleIndexEntryCount(rules)]WickRuleIndexEntry = undefined;
    var entry_index: usize = 0;

    for (rules, 0..) |rule, rule_index| {
        if (rule_index > std.math.maxInt(u32)) @compileError("too many Wick rules for compact
        ↪ rule index");
        entries[entry_index] = .{
            .key = wickRuleKey(rule.left.kind, rule.right.kind),
            .rule_index = @intCast(rule_index),
            .reversed = false,
        };
        entry_index += 1;

        if (rule.left.kind != rule.right.kind) {
            entries[entry_index] = .{
                .key = wickRuleKey(rule.right.kind, rule.left.kind),
                .rule_index = @intCast(rule_index),
                .reversed = true,
            };
```

```
            entry_index += 1;
        }
    }

    sortWickRuleIndex(entries[0..]);
    return entries;
}
```

3.3.6 Streaming Driver

This is not the final correlator algorithm. The pair-level boundary below is the actual primitive Wick step: given a multi-local input and two operator indices, it finds matching primitive rules and emits complete primitive Wick terms to a sink. The correlator does not inspect Wick-rule internals, resolved labels, tensor refs, or coordinate-derivative bookkeeping. It only chooses pairings and asks this layer to multiply a primitive Wick term into the current accumulator.

Full Wick recursion, signs, zero-mode evaluation, and simplification filters remain downstream.

The Wick sink contract is intentionally push-based. A correlator accumulator must provide:

- `emitWickTermStart(event)`, where `event` only records the input operator positions and term index,
- `emitWickScalar(factor)` for each resolved scalar factor,
- `emitWickCoordinate(factor)` for each coordinate kernel after universal insertion derivatives have acted,
- `emitWickTensor(factor)` for each resolved tensor or projector factor,
- `emitWickAction(factor)` for profile-derivative or projected-profile actions,
- `emitWickTermEnd()` after the primitive term has been emitted.

This is the boundary the full correlator should use. It never receives `wick.Rule`, `WickPair`, resolved label references, config references, or coordinate derivative bookkeeping.

Generated configs also carry a compact Wick-rule index. The hot pair path does not scan all rules for every candidate pair; it looks up the ordered operator-kind pair and streams the matching primitive rules in index order.

The zero-mode sink is also push-based but smaller. Base-case evaluation emits `emitZeroModeFactor(factor)` for each surviving residual factor and then `emitZeroModeBaseEnd()`. If configured zero-mode rules cannot consume every residual operator, the base-case procedure returns `false` and emits no end marker.

```
const Correlator = struct {
    /// WickPair is one oriented primitive Wick contraction between two concrete local
    ↪  operators.
    const WickPair = struct {
        rule: *const wick.Rule,
        config: *const CorrelatorConfig,
        left: kernel.Call.LocalOp,
        right: kernel.Call.LocalOp,
        labels: *const kernel.Call.LabelStore,
        reversed: bool,

        fn sideOp(self: WickPair, side: wick.Side) kernel.Call.LocalOp {
            return switch (side) {
                .left => if (self.reversed) self.right else self.left,
                .right => if (self.reversed) self.left else self.right,
            };
        }
```

```
    fn coordinate(self: WickPair, coordinate_ref: wick.CoordinateRef)
    ↪  ?coefficient.Variable {
        const op = self.sideOp(coordinate_ref.side);
        return switch (op.insertion) {
            .single => |single| switch (coordinate_ref.slot) {
                .position, .holomorphic => single.position,
                .antiholomorphic => null,
            },
            .pair => |pair| switch (coordinate_ref.slot) {
                .position, .holomorphic => pair.holomorphic_position,
                .antiholomorphic => pair.antiholomorphic_position,
            },
        };
    }

    fn derivativeCount(self: WickPair, coordinate_ref: wick.CoordinateRef) ?u8 {
        const op = self.sideOp(coordinate_ref.side);
        return switch (op.insertion) {
            .single => |single| switch (coordinate_ref.slot) {
                .position, .holomorphic => single.derivatives,
                .antiholomorphic => null,
            },
            .pair => |pair| switch (coordinate_ref.slot) {
                .position, .holomorphic => pair.holomorphic_derivatives,
                .antiholomorphic => pair.antiholomorphic_derivatives,
            },
        };
    }

    fn label(self: WickPair, label_ref: wick.LabelRef) ?kernel.Call.LabelValue {
        const op = self.sideOp(label_ref.side);
        const index: usize = @as(usize, @intCast(op.labels)) + label_ref.slot;
        if (index >= self.labels.values.len) return null;
        return self.labels.values[index];
    }

    fn configValue(self: WickPair, config_ref: ConfigRef) ?ConfigValue {
        for (self.config.config_entries) |entry| {
            if (entry.id == config_ref) return entry.value;
        }
        return null;
    }
};

/// ResolvedCoordinateRef is one concrete coordinate variable with its derivative count.
const ResolvedCoordinateRef = struct {
    variable: coefficient.Variable,
    derivatives: u8,
};

/// ResolvedCoordinateDifference is a concrete coordinate difference read by a Wick
↪  factor.
const ResolvedCoordinateDifference = struct {
    left: ResolvedCoordinateRef,
    right: ResolvedCoordinateRef,
};

/// ResolvedDerivativeAction records the derivative counts requested by one coordinate
↪  factor.
const ResolvedDerivativeAction = struct {
    left: ?u8 = null,
    right: ?u8 = null,
};
```

```
/// ResolvedTensorRef is tensor data resolved either from an operator label or config
↪   entry.
const ResolvedTensorRef = union(enum) {
    label: kernel.Call.LabelValue,
    config: ConfigValue,
};

/// ResolvedScalarFactor is a scalar factor with all label references plugged in.
const ResolvedScalarFactor = union(enum) {
    value: RuleScalar,
    label_bilinear_phase: struct { left: kernel.Call.LabelValue, right:
    ↪   kernel.Call.LabelValue, form: []const u8 },
    cocycle: struct { table: []const u8, left: kernel.Call.LabelValue, right:
    ↪   kernel.Call.LabelValue },
    spin_structure: []const u8,
};

/// ResolvedTensorFactor is a tensor multiplier with concrete labels and config values.
const ResolvedTensorFactor = union(enum) {
    none,
    metric: struct { left: kernel.Call.LabelValue, right: kernel.Call.LabelValue },
    momentum_index: struct { momentum: kernel.Call.LabelValue, index:
    ↪   kernel.Call.LabelValue },
    momentum_pair: struct { left: kernel.Call.LabelValue, right: kernel.Call.LabelValue },
    projector_metric: struct { projector: ResolvedTensorRef, left: kernel.Call.LabelValue,
    ↪   right: kernel.Call.LabelValue },
    projector_momentum_index: struct { projector: ResolvedTensorRef, momentum:
    ↪   kernel.Call.LabelValue, index: kernel.Call.LabelValue },
    projector_momentum_pair: struct { projector: ResolvedTensorRef, left:
    ↪   kernel.Call.LabelValue, right: kernel.Call.LabelValue },
};

/// ResolvedActionFactor is a Wick action with concrete profile, index, and projector
↪   data.
const ResolvedActionFactor = union(enum) {
    profile_derivative: struct { profile: kernel.Call.LabelValue, index:
    ↪   kernel.Call.LabelValue },
    projected_profile_derivative: struct { projector: ResolvedTensorRef, profile:
    ↪   kernel.Call.LabelValue, index: kernel.Call.LabelValue },
};

/// ResolvedCoordinateFactor is a coordinate kernel with concrete coordinates and
↪   derivative data.
const ResolvedCoordinateFactor = union(enum) {
    difference_power: struct { coordinate: ResolvedCoordinateDifference, exponent: i16,
    ↪   derivatives: ResolvedDerivativeAction },
    logarithm: struct { coordinate: ResolvedCoordinateDifference, derivatives:
    ↪   ResolvedDerivativeAction },
    green_kernel: struct { name: []const u8, coordinate: ResolvedCoordinateDifference,
    ↪   derivatives: ResolvedDerivativeAction },
    green_exponential: ResolvedCoordinateDifference,
};

/// ResolvedTerm is one Wick term whose factors can be resolved without allocation.
const ResolvedTerm = struct {
    pair: WickPair,
    source: *const wick.Term,

    fn scalarCount(self: ResolvedTerm) usize {
        return self.source.scalars.len;
    }

    fn coordinateCount(self: ResolvedTerm) usize {
```

```
        return self.source.coordinates.len;
    }

    fn tensorCount(self: ResolvedTerm) usize {
        return self.source.tensors.len;
    }

    fn actionCount(self: ResolvedTerm) usize {
        return self.source.actions.len;
    }

    fn scalar(self: ResolvedTerm, index: usize) ?ResolvedScalarFactor {
        if (index >= self.source.scalars.len) return null;
        return resolveScalarFactor(self.pair, self.source.scalars[index]);
    }

    fn coordinate(self: ResolvedTerm, index: usize) ?ResolvedCoordinateFactor {
        if (index >= self.source.coordinates.len) return null;
        return resolveCoordinateFactor(self.pair, self.source.coordinates[index]);
    }

    fn tensor(self: ResolvedTerm, index: usize) ?ResolvedTensorFactor {
        if (index >= self.source.tensors.len) return null;
        return resolveTensorFactor(self.pair, self.source.tensors[index]);
    }

    fn action(self: ResolvedTerm, index: usize) ?ResolvedActionFactor {
        if (index >= self.source.actions.len) return null;
        return resolveActionFactor(self.pair, self.source.actions[index]);
    }
};

/// ResolvedWickPair gives allocation-free access to concrete Wick terms for one pair.
const ResolvedWickPair = struct {
    pair: WickPair,

    fn termCount(self: ResolvedWickPair) usize {
        return self.pair.rule.expr.terms.len;
    }

    fn term(self: ResolvedWickPair, index: usize) ?ResolvedTerm {
        if (index >= self.pair.rule.expr.terms.len) return null;
        return .{ .pair = self.pair, .source = &self.pair.rule.expr.terms[index] };
    }
};

/// PrimitiveWickTerm is one complete primitive Wick contribution without later
↪  simplification.
const PrimitiveWickTerm = struct {
    resolved: ResolvedTerm,

    fn scalarCount(self: PrimitiveWickTerm) usize {
        return self.resolved.scalarCount();
    }

    fn coordinateCount(self: PrimitiveWickTerm) usize {
        return self.resolved.coordinateCount();
    }

    fn tensorCount(self: PrimitiveWickTerm) usize {
        return self.resolved.tensorCount();
    }

    fn actionCount(self: PrimitiveWickTerm) usize {
```

```
        return self.resolved.actionCount();
    }

    fn scalar(self: PrimitiveWickTerm, index: usize) ?ResolvedScalarFactor {
        return self.resolved.scalar(index);
    }

    fn coordinate(self: PrimitiveWickTerm, index: usize) ?coefficient.CoordinateFactor {
        const factor = self.resolved.coordinate(index) orelse return null;
        return evaluateCoordinateFactor(factor);
    }

    fn tensor(self: PrimitiveWickTerm, index: usize) ?ResolvedTensorFactor {
        return self.resolved.tensor(index);
    }

    fn action(self: PrimitiveWickTerm, index: usize) ?ResolvedActionFactor {
        return self.resolved.action(index);
    }
};

/// PrimitiveWickPair gives allocation-free complete terms for one primitive Wick pair.
const PrimitiveWickPair = struct {
    pair: WickPair,

    fn termCount(self: PrimitiveWickPair) usize {
        return self.pair.rule.expr.terms.len;
    }

    fn term(self: PrimitiveWickPair, index: usize) ?PrimitiveWickTerm {
        const resolved = (ResolvedWickPair{ .pair = self.pair }).term(index) orelse return
        ↪  null;
        return .{ .resolved = resolved };
    }
};

fn resolveCoordinateRef(pair: WickPair, coordinate_ref: wick.CoordinateRef)
↪  ?ResolvedCoordinateRef {
    return .{
        .variable = pair.coordinate(coordinate_ref) orelse return null,
        .derivatives = pair.derivativeCount(coordinate_ref) orelse return null,
    };
}

fn resolveCoordinateDifference(pair: WickPair, difference: wick.CoordinateDifference)
↪  ?ResolvedCoordinateDifference {
    return .{
        .left = resolveCoordinateRef(pair, difference.left) orelse return null,
        .right = resolveCoordinateRef(pair, difference.right) orelse return null,
    };
}

fn resolveDerivativeAction(pair: WickPair, coordinate: wick.CoordinateDifference, action:
↪  wick.DerivativeAction) ?ResolvedDerivativeAction {
    return .{
        .left = if (action.include_left) pair.derivativeCount(coordinate.left) orelse
        ↪  return null else null,
        .right = if (action.include_right) pair.derivativeCount(coordinate.right) orelse
        ↪  return null else null,
    };
}

fn resolveTensorRef(pair: WickPair, tensor_ref: wick.TensorRef) ?ResolvedTensorRef {
    return switch (tensor_ref) {
```

```
            .label => |label_ref| .{ .label = pair.label(label_ref) orelse return null },
            .config => |config_ref| .{ .config = pair.configValue(config_ref) orelse return
            ↪  null },
        };
    }

    fn resolveScalarFactor(pair: WickPair, factor: wick.ScalarFactor) ?ResolvedScalarFactor {
        return switch (factor) {
            .value => |value| .{ .value = value },
            .label_bilinear_phase => |phase| .{ .label_bilinear_phase = .{
                .left = pair.label(phase.left) orelse return null,
                .right = pair.label(phase.right) orelse return null,
                .form = phase.form,
            } },
            .cocycle => |cocycle| .{ .cocycle = .{
                .table = cocycle.table,
                .left = pair.label(cocycle.left) orelse return null,
                .right = pair.label(cocycle.right) orelse return null,
            } },
            .spin_structure => |spin_structure| .{ .spin_structure = spin_structure },
        };
    }

    fn resolveTensorFactor(pair: WickPair, factor: wick.TensorFactor) ?ResolvedTensorFactor {
        return switch (factor) {
            .none => .none,
            .metric => |metric| .{ .metric = .{
                .left = pair.label(metric.left) orelse return null,
                .right = pair.label(metric.right) orelse return null,
            } },
            .momentum_index => |item| .{ .momentum_index = .{
                .momentum = pair.label(item.momentum) orelse return null,
                .index = pair.label(item.index) orelse return null,
            } },
            .momentum_pair => |item| .{ .momentum_pair = .{
                .left = pair.label(item.left) orelse return null,
                .right = pair.label(item.right) orelse return null,
            } },
            .projector_metric => |item| .{ .projector_metric = .{
                .projector = resolveTensorRef(pair, item.projector) orelse return null,
                .left = pair.label(item.left) orelse return null,
                .right = pair.label(item.right) orelse return null,
            } },
            .projector_momentum_index => |item| .{ .projector_momentum_index = .{
                .projector = resolveTensorRef(pair, item.projector) orelse return null,
                .momentum = pair.label(item.momentum) orelse return null,
                .index = pair.label(item.index) orelse return null,
            } },
            .projector_momentum_pair => |item| .{ .projector_momentum_pair = .{
                .projector = resolveTensorRef(pair, item.projector) orelse return null,
                .left = pair.label(item.left) orelse return null,
                .right = pair.label(item.right) orelse return null,
            } },
        };
    }

    fn resolveActionFactor(pair: WickPair, factor: wick.ActionFactor) ?ResolvedActionFactor {
        return switch (factor) {
            .profile_derivative => |item| .{ .profile_derivative = .{
                .profile = pair.label(item.profile) orelse return null,
                .index = pair.label(item.index) orelse return null,
            } },
            .projected_profile_derivative => |item| .{ .projected_profile_derivative = .{
                .projector = resolveTensorRef(pair, item.projector) orelse return null,
```

```
            .profile = pair.label(item.profile) orelse return null,
            .index = pair.label(item.index) orelse return null,
        } },
    };
}

fn resolveCoordinateFactor(pair: WickPair, factor: wick.CoordinateFactor)
↪  ?ResolvedCoordinateFactor {
    return switch (factor) {
        .difference_power => |item| .{ .difference_power = .{
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↪  null,
            .exponent = item.exponent,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↪  item.derivatives) orelse return null,
        } },
        .logarithm => |item| .{ .logarithm = .{
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↪  null,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↪  item.derivatives) orelse return null,
        } },
        .green_kernel => |item| .{ .green_kernel = .{
            .name = item.name,
            .coordinate = resolveCoordinateDifference(pair, item.coordinate) orelse return
            ↪  null,
            .derivatives = resolveDerivativeAction(pair, item.coordinate,
            ↪  item.derivatives) orelse return null,
        } },
        .green_exponential => |coordinate| .{ .green_exponential =
        ↪  resolveCoordinateDifference(pair, coordinate) orelse return null },
    };
}

fn coordinateDifference(coordinate: ResolvedCoordinateDifference)
↪  coefficient.CoordinateDifference {
    return .{
        .left = coordinate.left.variable,
        .right = coordinate.right.variable,
    };
}

fn selectedDerivativeCount(action: ResolvedDerivativeAction) u16 {
    const left: u16 = if (action.left) |count| count else 0;
    const right: u16 = if (action.right) |count| count else 0;
    return left + right;
}

fn selectedRightDerivativeCount(action: ResolvedDerivativeAction) u16 {
    return if (action.right) |count| count else 0;
}

fn checkedMulInt(left: i64, right: i64) ?i64 {
    const result = @mulWithOverflow(left, right);
    if (result[1] != 0) return null;
    return result[0];
}

fn checkedAddExponent(exponent: i16, derivative_count: u16) ?i16 {
    const next = @as(i32, exponent) - @as(i32, derivative_count);
    if (next < std.math.minInt(i16) or next > std.math.maxInt(i16)) return null;
    return @intCast(next);
}
```

```
    fn fallingPower(exponent: i16, derivative_count: u16) ?i64 {
        var value: i64 = 1;
        var current: i64 = exponent;
        var index: u16 = 0;
        while (index < derivative_count) : (index += 1) {
            value = checkedMulInt(value, current) orelse return null;
            current -= 1;
        }
        return value;
    }

    fn factorial(value: u16) ?i64 {
        var result: i64 = 1;
        var current: u16 = 2;
        while (current <= value) : (current += 1) {
            result = checkedMulInt(result, current) orelse return null;
        }
        return result;
    }

    fn signFromParity(power: u16) i64 {
        return if ((power & 1) == 0) 1 else -1;
    }

    fn evaluateDifferencePower(item: anytype) ?coefficient.CoordinateFactor {
        const derivative_count = selectedDerivativeCount(item.derivatives);
        const right_count = selectedRightDerivativeCount(item.derivatives);
        const falling = fallingPower(item.exponent, derivative_count) orelse return null;
        const signed = checkedMulInt(signFromParity(right_count), falling) orelse return null;
        return .{
            .scalar = coefficient.rational(signed, 1) orelse return null,
            .kernel = .{ .difference_power = .{
                .coordinate = coordinateDifference(item.coordinate),
                .exponent = checkedAddExponent(item.exponent, derivative_count) orelse return
                ↪  null,
            } },
        };
    }

    fn evaluateLogarithm(item: anytype) ?coefficient.CoordinateFactor {
        const derivative_count = selectedDerivativeCount(item.derivatives);
        if (derivative_count == 0) {
            return .{
                .scalar = coefficient.integer(1),
                .kernel = .{ .logarithm = coordinateDifference(item.coordinate) },
            };
        }

        const right_count = selectedRightDerivativeCount(item.derivatives);
        const base = factorial(derivative_count - 1) orelse return null;
        const signed = checkedMulInt(signFromParity(right_count + derivative_count - 1), base)
        ↪  orelse return null;
        return .{
            .scalar = coefficient.rational(signed, 1) orelse return null,
            .kernel = .{ .difference_power = .{
                .coordinate = coordinateDifference(item.coordinate),
                .exponent = -@as(i16, @intCast(derivative_count)),
            } },
        };
    }

    fn evaluateGreenKernel(item: anytype) coefficient.CoordinateFactor {
        return .{
            .scalar = coefficient.integer(1),
```

```
            .kernel = .{ .green_kernel = .{
                .name = item.name,
                .coordinate = coordinateDifference(item.coordinate),
                .left_derivatives = if (item.derivatives.left) |count| count else 0,
                .right_derivatives = if (item.derivatives.right) |count| count else 0,
            } },
        };
    }

    fn evaluateCoordinateFactor(factor: ResolvedCoordinateFactor)
    ↪  ?coefficient.CoordinateFactor {
        return switch (factor) {
            .difference_power => |item| evaluateDifferencePower(item),
            .logarithm => |item| evaluateLogarithm(item),
            .green_kernel => |item| evaluateGreenKernel(item),
            .green_exponential => |coordinate| .{
                .scalar = coefficient.integer(1),
                .kernel = .{ .green_exponential = coordinateDifference(coordinate) },
            },
        };
    }

    const WickTermEvent = struct {
        left_index: usize,
        right_index: usize,
        term_index: usize,
    };

};

const max_zero_mode_residuals = 128;
const max_zero_mode_items = 32;

const ZeroModeRuntime = struct {
    const ResidualCursor = struct {
        ops: kernel.Call.MultiOp,
        indices: ?[]const usize,

        fn len(self: ResidualCursor) usize {
            return if (self.indices) |indices| indices.len else self.ops.operators.len;
        }

        fn opIndex(self: ResidualCursor, residual_index: usize) usize {
            return if (self.indices) |indices| indices[residual_index] else residual_index;
        }

        fn op(self: ResidualCursor, residual_index: usize) kernel.Call.LocalOp {
            return self.ops.operators[self.opIndex(residual_index)];
        }
    };

    const CJet = struct {
        coordinate: coefficient.Variable,
        derivative_order: u8,
    };

    const MomentumDelta = struct {
        projector: ?ConfigValue,
        momenta: []const kernel.Call.LabelValue,
        two_pi_power: u16,
        scalar: RuleScalar,
    };

    const DirichletPhase = struct {
```

```
        projector: ConfigValue,
        position: ConfigValue,
        momenta: []const kernel.Call.LabelValue,
    };

    const ProfileFactor = struct {
        profile: kernel.Call.LabelValue,
        coordinate: coefficient.Variable,
    };

    const ZeroModeFactor = union(enum) {
        bc_top_form: struct {
            support: zero_mode.BcSupport,
            normalization: RuleScalar,
            jets: [3]CJet,
        },
        momentum_delta: MomentumDelta,
        dirichlet_phase: DirichletPhase,
        profile_fourier_integral: ProfileFactor,
        profile_polynomial_integral: ProfileFactor,
        profile_gaussian_differential: ProfileFactor,
    };
};

fn residualConsumed(mask: u128, index: usize) bool {
    return (mask & (@as(u128, 1) << @intCast(index))) != 0;
}

fn markResidualConsumed(mask: *u128, index: usize) void {
    mask.* |= @as(u128, 1) << @intCast(index);
}

fn kindIn(kind: operators.OperatorKindId, candidates: []const operators.OperatorKindId) bool {
    for (candidates) |candidate| {
        if (candidate == kind) return true;
    }
    return false;
}

fn localPosition(op: kernel.Call.LocalOp) ?coefficient.Variable {
    return switch (op.insertion) {
        .single => |single| single.position,
        .pair => |pair| pair.holomorphic_position,
    };
}

fn localDerivativeOrder(op: kernel.Call.LocalOp) u8 {
    return switch (op.insertion) {
        .single => |single| single.derivatives,
        .pair => |pair| pair.holomorphic_derivatives,
    };
}

fn firstLabel(ops: kernel.Call.MultiOp, op: kernel.Call.LocalOp) ?kernel.Call.LabelValue {
    const index: usize = @intCast(op.labels);
    if (index >= ops.labels.values.len) return null;
    return ops.labels.values[index];
}

fn configValue(config_ptr: *const CorrelatorConfig, config_ref: ConfigRef) ?ConfigValue {
    for (config_ptr.config_entries) |entry| {
        if (entry.id == config_ref) return entry.value;
    }
    return null;
```

```
}

fn projectorRank(value: ConfigValue) ?u16 {
    return switch (value) {
        .tensor_projector => |projector| @intCast(@intFromEnum(projector) >> 24),
        else => null,
    };
}

fn rankValue(config_ptr: *const CorrelatorConfig, source: zero_mode.RankSource) ?u16 {
    return switch (source) {
        .none => 0,
        .target_dimension => |config_ref| switch (configValue(config_ptr, config_ref) orelse
        ↪  return null) {
            .target_dimension => |dimension| dimension,
            else => null,
        },
        .projector_rank => |config_ref| projectorRank(configValue(config_ptr, config_ref)
        ↪  orelse return null),
    };
}

fn hasRankSource(source: zero_mode.RankSource) bool {
    return switch (source) {
        .none => false,
        else => true,
    };
}

fn emitBcTopForm(rule: zero_mode.BcTopForm, residual: ZeroModeRuntime.ResidualCursor,
↪  consumed: *u128, sink: anytype) !void {
    var jets: [3]ZeroModeRuntime.CJet = undefined;
    var residual_ids: [3]usize = undefined;
    var jet_count: usize = 0;
    var overflow = false;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (!kindIn(op.kind, rule.c_kind_ids)) continue;
        if (jet_count >= jets.len) {
            overflow = true;
            continue;
        }
        jets[jet_count] = .{
            .coordinate = localPosition(op) orelse return error.InvalidZeroModeOperator,
            .derivative_order = localDerivativeOrder(op),
        };
        residual_ids[jet_count] = residual_index;
        jet_count += 1;
    }

    if (overflow or jet_count != 3) return;
    for (residual_ids) |residual_id| {
        markResidualConsumed(consumed, residual_id);
    }
    try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .bc_top_form = .{
        .support = rule.support,
        .normalization = rule.normalization,
        .jets = jets,
    } });
}
```

```
fn emitFreeBosonConstantMode(config_ptr: *const CorrelatorConfig, rule:
↪  zero_mode.FreeBosonConstantMode, residual: ZeroModeRuntime.ResidualCursor, consumed:
↪  *u128, sink: anytype) !void {
    var momenta: [max_zero_mode_items]kernel.Call.LabelValue = undefined;
    var momentum_count: usize = 0;

    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (residualConsumed(consumed.*, residual_index)) continue;
        const op = residual.op(residual_index);
        if (kindIn(op.kind, rule.exp_kind_ids)) {
            if (momentum_count >= momenta.len) return error.TooManyZeroModeItems;
            momenta[momentum_count] = firstLabel(residual.ops, op) orelse return
            ↪  error.InvalidZeroModeOperator;
            momentum_count += 1;
            markResidualConsumed(consumed, residual_index);
            continue;
        }

        if (kindIn(op.kind, rule.profile_kind_ids)) {
            const label = firstLabel(residual.ops, op) orelse return
            ↪  error.InvalidZeroModeOperator;
            const profile = switch (label) {
                .symbol => |value| @as(Handle.Profile, @enumFromInt(value)),
                else => return error.InvalidZeroModeOperator,
            };
            const factor = ZeroModeRuntime.ProfileFactor{
                .profile = label,
                .coordinate = localPosition(op) orelse return error.InvalidZeroModeOperator,
            };
            switch (profilePresentation(profile)) {
                .position_space => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
                ↪  .profile_gaussian_differential = factor }),
                .fourier => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
                ↪  .profile_fourier_integral = factor }),
                .polynomial_rnc => try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{
                ↪  .profile_polynomial_integral = factor }),
            }
            markResidualConsumed(consumed, residual_index);
        }
    }

    if (momentum_count != 0 or hasRankSource(rule.normalization.two_pi_power)) {
        const projector = if (rule.integration_projector) |config_ref| configValue(config_ptr,
        ↪  config_ref) orelse return error.InvalidZeroModeConfig else null;
        try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .momentum_delta = .{
            .projector = projector,
            .momenta = momenta[0..momentum_count],
            .two_pi_power = rankValue(config_ptr, rule.normalization.two_pi_power) orelse
            ↪  return error.InvalidZeroModeConfig,
            .scalar = rule.normalization.scalar,
        } });
    }

    if (rule.fixed_projector) |projector_ref| {
        if (momentum_count == 0) return;
        const position_ref = rule.fixed_position orelse return error.InvalidZeroModeConfig;
        try sink.emitZeroModeFactor(ZeroModeRuntime.ZeroModeFactor{ .dirichlet_phase = .{
            .projector = configValue(config_ptr, projector_ref) orelse return
            ↪  error.InvalidZeroModeConfig,
            .position = configValue(config_ptr, position_ref) orelse return
            ↪  error.InvalidZeroModeConfig,
            .momenta = momenta[0..momentum_count],
        } });
```

```
    }
}

fn allResidualsConsumed(residual: ZeroModeRuntime.ResidualCursor, consumed: u128) bool {
    var residual_index: usize = 0;
    while (residual_index < residual.len()) : (residual_index += 1) {
        if (!residualConsumed(consumed, residual_index)) return false;
    }
    return true;
}

/// emitZeroModeBaseCase consumes residual operators with configured zero-mode rules.
pub fn emitZeroModeBaseCase(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp,
↪  residual_indices: ?[]const usize, sink: anytype) !bool {
    const residual = ZeroModeRuntime.ResidualCursor{ .ops = ops, .indices = residual_indices
    ↪  };
    if (residual.len() > max_zero_mode_residuals) return error.TooManyZeroModeResiduals;

    var consumed: u128 = 0;
    for (config_ptr.zero_modes) |rule| {
        switch (rule.expr) {
            .bc_top_form => |payload| try emitBcTopForm(payload, residual, &consumed, sink),
            .free_boson_constant_mode => |payload| try emitFreeBosonConstantMode(config_ptr,
            ↪  payload, residual, &consumed, sink),
        }
    }

    if (!allResidualsConsumed(residual, consumed)) return false;
    try sink.emitZeroModeBaseEnd();
    return true;
}

fn ruleMatches(rule: wick.Rule, left: kernel.Call.LocalOp, right: kernel.Call.LocalOp) ?bool {
    if (rule.left.kind == left.kind and rule.right.kind == right.kind) {
        return false;
    }
    if (rule.left.kind == right.kind and rule.right.kind == left.kind) {
        return true;
    }
    return null;
}

fn matchRule(config_ptr: *const CorrelatorConfig, rule: *const wick.Rule, left:
↪  kernel.Call.LocalOp, right: kernel.Call.LocalOp, labels: *const kernel.Call.LabelStore)
↪  ?Correlator.WickPair {
    const reversed = ruleMatches(rule.*, left, right) orelse return null;
    return .{
        .rule = rule,
        .config = config_ptr,
        .left = left,
        .right = right,
        .labels = labels,
        .reversed = reversed,
    };
}

fn lowerBoundWickRuleIndex(entries: []const WickRuleIndexEntry, key: u64) usize {
    var lo: usize = 0;
    var hi: usize = entries.len;
    while (lo < hi) {
        const mid = lo + ((hi - lo) / 2);
        if (entries[mid].key < key) {
            lo = mid + 1;
        } else {
```

```
            hi = mid;
        }
    }
    return lo;
}

fn emitPrimitiveWickTerm(pair: Correlator.WickPair, left_index: usize, right_index: usize,
↪  term_index: usize, sink: anytype) !void {
    const primitive = (Correlator.PrimitiveWickPair{ .pair = pair }).term(term_index) orelse
    ↪  return error.InvalidWickTerm;

    const event = Correlator.WickTermEvent{
        .left_index = left_index,
        .right_index = right_index,
        .term_index = term_index,
    };
    try sink.emitWickTermStart(event);

    var scalar_index: usize = 0;
    while (scalar_index < primitive.scalarCount()) : (scalar_index += 1) {
        try sink.emitWickScalar(primitive.scalar(scalar_index) orelse return
        ↪  error.InvalidWickFactor);
    }

    var coordinate_index: usize = 0;
    while (coordinate_index < primitive.coordinateCount()) : (coordinate_index += 1) {
        try sink.emitWickCoordinate(primitive.coordinate(coordinate_index) orelse return
        ↪  error.InvalidWickFactor);
    }

    var tensor_index: usize = 0;
    while (tensor_index < primitive.tensorCount()) : (tensor_index += 1) {
        try sink.emitWickTensor(primitive.tensor(tensor_index) orelse return
        ↪  error.InvalidWickFactor);
    }

    var action_index: usize = 0;
    while (action_index < primitive.actionCount()) : (action_index += 1) {
        try sink.emitWickAction(primitive.action(action_index) orelse return
        ↪  error.InvalidWickFactor);
    }

    try sink.emitWickTermEnd();
}

fn emitIndexedWickPairTerms(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp,
↪  left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↪  kernel.Call.LocalOp, sink: anytype) !usize {
    const key = wickRuleKey(left.kind, right.kind);
    var entry_index = lowerBoundWickRuleIndex(config_ptr.wick_rule_index, key);
    var emitted: usize = 0;

    while (entry_index < config_ptr.wick_rule_index.len and
    ↪  config_ptr.wick_rule_index[entry_index].key == key) : (entry_index += 1) {
        const entry = config_ptr.wick_rule_index[entry_index];
        const rule_index: usize = @intCast(entry.rule_index);
        if (rule_index >= config_ptr.wick_rules.len) return error.InvalidWickRuleIndex;
        const rule = &config_ptr.wick_rules[rule_index];
        const pair = Correlator.WickPair{
            .rule = rule,
            .config = config_ptr,
            .left = left,
            .right = right,
            .labels = ops.labels,
```

```
            .reversed = entry.reversed,
        };

        var term_index: usize = 0;
        while (term_index < pair.rule.expr.terms.len) : (term_index += 1) {
            try emitPrimitiveWickTerm(pair, left_index, right_index, term_index, sink);
            emitted += 1;
        }
    }

    return emitted;
}

fn emitScannedWickPairTerms(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp,
↪  left_index: usize, right_index: usize, left: kernel.Call.LocalOp, right:
↪  kernel.Call.LocalOp, sink: anytype) !usize {
    var emitted: usize = 0;

    for (config_ptr.wick_rules) |*rule| {
        if (matchRule(config_ptr, rule, left, right, ops.labels)) |pair| {
            var term_index: usize = 0;
            while (term_index < pair.rule.expr.terms.len) : (term_index += 1) {
                try emitPrimitiveWickTerm(pair, left_index, right_index, term_index, sink);
                emitted += 1;
            }
        }
    }

    return emitted;
}

/// emitWickPairTerms emits complete primitive Wick terms for two input positions.
pub fn emitWickPairTerms(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp,
↪  left_index: usize, right_index: usize, sink: anytype) !usize {
    if (left_index >= ops.operators.len or right_index >= ops.operators.len or left_index ==
    ↪  right_index) return error.InvalidOperatorIndex;

    const left = ops.operators[left_index];
    const right = ops.operators[right_index];
    if (config_ptr.wick_rule_index.len != 0) {
        return emitIndexedWickPairTerms(config_ptr, ops, left_index, right_index, left, right,
        ↪  sink);
    }

    return emitScannedWickPairTerms(config_ptr, ops, left_index, right_index, left, right,
    ↪  sink);
}

/// streamCorrelator emits pair Wick events and then tries the zero-mode base case.
pub fn streamCorrelator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink:
↪  anytype) !void {
    for (ops.operators, 0..) |_, left_index| {
        var right_index = left_index + 1;
        while (right_index < ops.operators.len) : (right_index += 1) {
            _ = try emitWickPairTerms(config_ptr, ops, left_index, right_index, sink);
        }
    }

    _ = try emitZeroModeBaseCase(config_ptr, ops, null, sink);
}
```

3.4 Free Boson Preset

Interface

Functions

`freeBoson(comptime cfg: FreeBosonConfig) type`
freeBoson builds the generated preset type for a noncompact free-boson CFT.

`boundaryExtension(comptime cfg: FreeBosonBoundaryConfig) BoundaryExtension`
boundaryExtension declares the Neumann/Dirichlet free-boson boundary extension.

Types

`FreeBosonConfig (struct)`
FreeBosonConfig declares a noncompact D-dimensional free boson preset.

`FreeBosonBoundaryConfig (struct)`
FreeBosonBoundaryConfig declares Neumann and Dirichlet data for a brane.

This node defines the free-boson preset on the sphere and its Neumann/Dirichlet boundary extension. It uses the compact vocabulary from Preset Shared Runtime and is exported through Presets.

The preset does not perform tensor algebra. A D-dimensional target space only fixes which $SO(D)$-type labels are expected; later coefficient evaluation uses Tensor-Code to evaluate metrics, momenta, and projectors.

```
const std = @import("std");
const shared = @import("shared.zig");

const operators = @import("../expressions/operators.zig");
const kernel = @import("../kernel.zig");
const Handle = shared.Handle;
const RuleScalar = shared.RuleScalar;
const CorrelatorConfig = shared.CorrelatorConfig;
const BoundaryExtension = shared.BoundaryExtension;
const Local = shared.Local;
const wick = shared.wick;
const zero_mode = shared.zero_mode;
```

The preset owns the free-boson operator vocabulary. The enum below is physics data; Preset Shared Runtime only receives the compact kind ids derived from it. The base separates this preset's ids from other implemented presets, while individual ids still come from enum order rather than handwritten numbers.

```
const namespace = "free_boson";
const scalars = shared.scalars;
const alpha_prime_atom = scalars.atom(namespace, "alpha_prime");
const k_disk_atom = scalars.atom(namespace, "K_D2");
const neumann_projector_ref = shared.configRef(namespace, "neumann_projector");
const dirichlet_projector_ref = shared.configRef(namespace, "dirichlet_projector");
const bulk_boundary_projector_ref = shared.configRef(namespace, "bulk_boundary_projector");
const dirichlet_position_ref = shared.configRef(namespace, "dirichlet_position");
const chan_paton_ref = shared.configRef(namespace, "chan_paton");
const target_dimension_ref = shared.configRef(namespace, "target_dimension");

const Family = enum {
    x,
    d_x,
```

```
    d_xt,
    exp_x,
    profile_x,
    d_x_boundary,
    exp_x_boundary,
    profile_x_boundary,
};

const ZeroSector = enum {
    bulk,
    boundary,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return shared.familyKind(namespace, family);
}

fn zeroSector(comptime sector: ZeroSector) zero_mode.Sector {
    return shared.sectorId(namespace, sector);
}

fn rawToken(value: anytype) u32 {
    return @intFromEnum(value);
}

fn token(comptime T: type, id: u32) T {
    return @enumFromInt(if (id == 0) 1 else id);
}

fn coordVariable(coord: Handle.Coord) u32 {
    return rawToken(coord);
}

fn singleInsertion(z: Handle.Coord, derivatives: u8) operators.OperatorInsertion {
    return .{ .single = .{
        .position = coordVariable(z),
        .derivatives = derivatives,
    } };
}

fn pairInsertion(z: Handle.Coord, zbar: Handle.Coord) operators.OperatorInsertion {
    return .{ .pair = .{
        .holomorphic_position = coordVariable(z),
        .antiholomorphic_position = coordVariable(zbar),
        .holomorphic_derivatives = 0,
        .antiholomorphic_derivatives = 0,
    } };
}

fn boundaryCoord(y: Handle.BoundaryCoord) Handle.Coord {
    return token(Handle.Coord, rawToken(y));
}

fn freeBosonPattern(comptime family: Family, comptime support: wick.Support) wick.Pattern {
    return wick.pattern(kind(family), support);
}

fn dXPattern() wick.Pattern {
    return freeBosonPattern(.d_x, .holomorphic);
}

fn dXtPattern() wick.Pattern {
    return freeBosonPattern(.d_xt, .antiholomorphic);
}
```

```
fn expXPattern() wick.Pattern {
    return freeBosonPattern(.exp_x, .bulk_pair);
}

fn profilePattern() wick.Pattern {
    return freeBosonPattern(.profile_x, .bulk_pair);
}

fn dXBoundaryPattern() wick.Pattern {
    return freeBosonPattern(.d_x_boundary, .boundary);
}

fn expXBoundaryPattern() wick.Pattern {
    return freeBosonPattern(.exp_x_boundary, .boundary);
}

fn profileBoundaryPattern() wick.Pattern {
    return freeBosonPattern(.profile_x_boundary, .boundary);
}

fn stableNameId(comptime name: []const u8) u32 {
    var hash: u32 = 2166136261;
    inline for (name) |byte| {
        hash = (hash ^ @as(u32, byte)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

fn stableSubspaceId(comptime dimensions: []const u16) u32 {
    var hash: u32 = 2166136261;
    inline for (dimensions) |dimension| {
        hash = (hash ^ @as(u32, dimension & 0xff)) *% 16777619;
        hash = (hash ^ @as(u32, dimension >> 8)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

fn projectorId(comptime rank: u8, hash: u32) u32 {
    return (@as(u32, rank) << 24) | (hash & 0x00ff_ffff);
}

fn polynomialProfileId(point: Handle.TargetPoint, comptime terms: []const
↪  Handle.PolynomialProfileTerm) u32 {
    var hash = rawToken(point) ^ 0x9e37_79b9;
    inline for (terms) |term| {
        hash = (hash ^ rawToken(term.coefficient)) *% 16777619;
        hash = (hash ^ @as(u32, term.power)) *% 16777619;
    }
    return if (hash == 0) 1 else hash;
}

const TargetSpace = struct {
    /// subspace names a target-space projector onto a coordinate subspace.
    pub fn subspace(comptime dimensions: []const u16) Handle.TensorProjector {
        return token(Handle.TensorProjector, projectorId(@intCast(dimensions.len),
        ↪  stableSubspaceId(dimensions)));
    }

    /// complement names the complementary target-space projector.
    pub fn complement(comptime dimensions: []const u16) Handle.TensorProjector {
        return token(Handle.TensorProjector, projectorId(0, stableSubspaceId(dimensions) ^
        ↪  0xa5a5_a5a5));
    }
```

```
    /// point names a target-space point used by boundary conditions.
    pub fn point(comptime name: []const u8) Handle.TargetPoint {
        return token(Handle.TargetPoint, stableNameId(name));
    }

    /// boundaryStack names Chan-Paton data for a stacked boundary condition.
    pub fn boundaryStack(comptime name: []const u8) Handle.BoundaryStack {
        return token(Handle.BoundaryStack, stableNameId(name));
    }
};
```

3.4.1 Bulk Preset

The default normalization is the stringbook free-field normalization

$$T^X = -\frac{1}{\alpha'}\,\partial X^\mu \partial X_\mu$$

[2]. The rule scalar remains symbolic here because different applications set α' and ∂X normalizations differently.

```
/// FreeBosonConfig declares a noncompact D-dimensional free boson preset.
pub const FreeBosonConfig = struct {
    dimension: u16,
    alpha_prime: shared.ScalarAtom = alpha_prime_atom,
    zero_mode_normalization: RuleScalar = .one,
};

/// freeBoson builds the generated preset type for a noncompact free-boson CFT.
pub fn freeBoson(comptime cfg: FreeBosonConfig) type {
    return struct {
        /// is_free_boson_preset identifies this generated type for composition.
        pub const is_free_boson_preset = true;
        /// op exposes free-boson local operator builders.
        pub const op = FreeBosonOp;
        /// profile exposes profile-presentation builders.
        pub const profile = ProfileOp;
        /// target exposes deterministic target-space tokens for preset data.
        pub const target = TargetSpace;
        /// config exposes named correlator configs for this preset.
        pub const config = FreeBosonCorrelatorConfig(cfg);
        /// rules exposes the preset-authored Wick rule templates.
        pub const rules = FreeBosonRules(cfg);

        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) Local {
            return Local.init(allocator);
        }

        /// correlator streams rule matches for free-boson insertions.
        pub fn correlator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink:
        ↪  anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}
```

The derivative meaning is $dX(\mu, n, z) = \partial^{n+1} X^\mu(z)$. The explicit X body is kept because NLSM expansions and position-space profiles produce logarithmic Green kernels rather than only plane-wave contractions.

```
const FreeBosonOp = struct {
    /// X builds an explicit bulk free-boson field insertion.
    pub fn X(local: *Local, mu: Handle.Index, z: Handle.Coord, zbar: Handle.Coord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.x), pairInsertion(z, zbar), &.{Local.symbol(mu)});
    }

    /// dX builds a holomorphic derivative field insertion.
    pub fn dX(local: *Local, mu: Handle.Index, n: u8, z: Handle.Coord) !kernel.Call.LocalOp {
        return local.op(kind(.d_x), singleInsertion(z, n), &.{Local.symbol(mu)});
    }

    /// dXt builds an antiholomorphic derivative field insertion.
    pub fn dXt(local: *Local, mu: Handle.Index, n: u8, zbar: Handle.Coord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.d_xt), singleInsertion(zbar, n), &.{Local.symbol(mu)});
    }

    /// expX builds a normal-ordered plane-wave insertion.
    pub fn expX(local: *Local, k: Handle.Momentum, z: Handle.Coord, zbar: Handle.Coord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.exp_x), pairInsertion(z, zbar), &.{Local.symbol(k)});
    }

    /// profile builds a target-space profile insertion.
    pub fn profile(local: *Local, f: Handle.Profile, z: Handle.Coord, zbar: Handle.Coord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.profile_x), pairInsertion(z, zbar), &.{Local.symbol(f)});
    }

    /// profileVector builds a vector-valued target-space profile insertion.
    pub fn profileVector(local: *Local, f: Handle.Profile, nu: Handle.Index, z: Handle.Coord,
    ↪  zbar: Handle.Coord) !kernel.Call.LocalOp {
        return local.op(kind(.profile_x), pairInsertion(z, zbar), &.{ Local.symbol(f),
        ↪  Local.symbol(nu) });
    }
};
```

Profiles are handles to a chosen presentation. Fourier profiles resolve to plane waves and momentum conservation can partially evaluate the inverse Fourier transform. Position-space profiles keep the zero-mode integral explicit. Polynomial profiles support Riemann-normal-coordinate expansions.

```
const ProfileOp = struct {
    /// positionSpace selects the exact residual position-space profile presentation.
    pub fn positionSpace(f: Handle.TargetFunction) Handle.Profile {
        return shared.profileHandle(.position_space, rawToken(f));
    }

    /// fourier selects the Fourier-space profile presentation.
    pub fn fourier(f_hat: Handle.FourierTransform) Handle.Profile {
        return shared.profileHandle(.fourier, rawToken(f_hat));
    }

    /// polynomialRnc selects a finite polynomial profile presentation.
    pub fn polynomialRnc(point: Handle.TargetPoint, comptime terms: []const
    ↪  Handle.PolynomialProfileTerm) Handle.Profile {
        return shared.profileHandle(.polynomial_rnc, polynomialProfileId(point, terms));
    }
};
```

3.4.2 Sphere Rules

On the sphere the primitive free-boson rules include same-chirality $\partial X\,\partial X$ contractions, ∂X contractions with plane waves and target-space profiles, and the plane-wave Green-kernel factor. Labels supply indices, momenta, and profile handles at runtime.

```
fn holomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn bulkHolomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .holomorphic), wick.coord(.right, .holomorphic));
}

fn bulkToBoundaryHolomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .holomorphic), wick.coord(.right, .position));
}

fn bulkToBoundaryAntiholomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .antiholomorphic), wick.coord(.right,
    ↪  .position));
}

fn freeBosonDxDx(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(alpha, 2), .{ .metric = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, holomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn freeBosonDxtDxt(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(alpha, 2), .{ .metric = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, antiholomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn freeBosonDxExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(scalars.i(alpha)), 2), .{
    ↪  .momentum_index = .{
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, holomorphicDifference(), -1, .{ .include_left = true });
}

fn freeBosonDxProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPole(scalars.div(scalars.neg(alpha), 2), .none,
    ↪  holomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.profileDerivative(wick.label(.right, 0), wick.label(.left, 0)),
    });
}

fn freeBosonDxtExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(scalars.i(alpha)), 2), .{
    ↪  .momentum_index = .{
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, antiholomorphicDifference(), -1, .{ .include_left = true });
}
```

```
fn freeBosonDxtProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPole(scalars.div(scalars.neg(alpha), 2), .none,
    ↪  antiholomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.profileDerivative(wick.label(.right, 0), wick.label(.left, 0)),
    });
}

fn freeBosonExpXExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.greenExponential(scalars.div(alpha, 2), .{ .momentum_pair = .{
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, bulkHolomorphicDifference());
}

fn sphereWickStorage(comptime alpha: RuleScalar) [7]wick.Rule {
    return [_]wick.Rule{
        wick.rule(dXPattern(), dXPattern(), freeBosonDxDx(alpha)),
        wick.rule(dXtPattern(), dXtPattern(), freeBosonDxtDxt(alpha)),
        wick.rule(dXPattern(), expXPattern(), freeBosonDxExpX(alpha)),
        wick.rule(dXtPattern(), expXPattern(), freeBosonDxtExpX(alpha)),
        wick.rule(dXPattern(), profilePattern(), freeBosonDxProfile(alpha)),
        wick.rule(dXtPattern(), profilePattern(), freeBosonDxtProfile(alpha)),
        wick.rule(expXPattern(), expXPattern(), freeBosonExpXExpX(alpha)),
    };
}

fn sphereZeroStorage(comptime cfg: FreeBosonConfig) [1]zero_mode.Rule {
    return [_]zero_mode.Rule{
        zero_mode.rule(zeroSector(.bulk), .{ .free_boson_constant_mode = .{
            .exp_kind_ids = &.{kind(.exp_x)},
            .profile_kind_ids = &.{kind(.profile_x)},
            .normalization = .{
                .scalar = cfg.zero_mode_normalization,
                .two_pi_power = .{ .target_dimension = target_dimension_ref },
            },
        } }),
    };
}

fn FreeBosonRules(comptime cfg: FreeBosonConfig) type {
    const alpha = scalars.atomScalar(cfg.alpha_prime);
    return struct {
        const sphere_wick_storage = sphereWickStorage(alpha);

        /// sphere_wick lists the primitive free-boson Wick rules on the sphere.
        pub const sphere_wick: []const wick.Rule = &sphere_wick_storage;
    };
}

fn FreeBosonCorrelatorConfig(comptime cfg: FreeBosonConfig) type {
    const rules = FreeBosonRules(cfg);
    return struct {
        const sphere_zero_storage = sphereZeroStorage(cfg);
        const sphere_wick_index_storage = shared.wickRuleIndexStorage(rules.sphere_wick);

        /// sphere selects free-boson sphere Wick and zero-mode rules.
        pub const sphere = CorrelatorConfig{
            .wick_rules = rules.sphere_wick,
            .wick_rule_index = &sphere_wick_index_storage,
            .zero_modes = &sphere_zero_storage,
            .config_entries = &.{
                .{ .id = target_dimension_ref, .value = .{ .target_dimension = cfg.dimension }
                ↪  },
            },
```

```
        };
    };
}
```

3.4.3 Boundary Extension

The boundary extension is separate from the bulk free boson. It adds Neumann/Dirichlet data, optional Chan-Paton stack data, boundary-local operators, mixed bulk-boundary rules, and disk zero-mode rules. The doubling choice is already reflected in these rule templates before runtime.

```
/// FreeBosonBoundaryConfig declares Neumann and Dirichlet data for a brane.
pub const FreeBosonBoundaryConfig = struct {
    neumann: Handle.TensorProjector,
    dirichlet: Handle.TensorProjector,
    dirichlet_position: Handle.TargetPoint,
    alpha_prime: shared.ScalarAtom = alpha_prime_atom,
    zero_mode_normalization: RuleScalar = scalars.atomScalar(k_disk_atom),
    chan_paton: ?Handle.BoundaryStack = null,
};

const FreeBosonBoundaryOp = struct {
    /// dXBoundary builds stringbook's holomorphic boundary limit of dX.
    pub fn dXBoundary(local: *Local, mu: Handle.Index, n: u8, y: Handle.BoundaryCoord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.d_x_boundary), singleInsertion(boundaryCoord(y), n),
        ↪  &.{Local.symbol(mu)});
    }

    /// expXBoundary builds a Neumann boundary plane-wave insertion.
    pub fn expXBoundary(local: *Local, k: Handle.Momentum, y: Handle.BoundaryCoord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.exp_x_boundary), singleInsertion(boundaryCoord(y), 0),
        ↪  &.{Local.symbol(k)});
    }

    /// profileBoundary builds a boundary profile insertion.
    pub fn profileBoundary(local: *Local, f: Handle.Profile, y: Handle.BoundaryCoord)
    ↪  !kernel.Call.LocalOp {
        return local.op(kind(.profile_x_boundary), singleInsertion(boundaryCoord(y), 0),
        ↪  &.{Local.symbol(f)});
    }

    /// profileBoundaryVector builds a vector-valued boundary profile insertion.
    pub fn profileBoundaryVector(local: *Local, f: Handle.Profile, nu: Handle.Index, y:
    ↪  Handle.BoundaryCoord) !kernel.Call.LocalOp {
        return local.op(kind(.profile_x_boundary), singleInsertion(boundaryCoord(y), 0), &.{
        ↪  Local.symbol(f), Local.symbol(nu) });
    }
};

/// boundaryExtension declares the Neumann/Dirichlet free-boson boundary extension.
pub fn boundaryExtension(comptime cfg: FreeBosonBoundaryConfig) BoundaryExtension {
    const data = FreeBosonBoundaryData(cfg);
    return .{
        .kind = shared.extensionKind("free_boson_boundary"),
        .op = FreeBosonBoundaryOp,
        .wick_rules = data.disk_wick,
        .zero_modes = data.disk_zero_modes,
        .config_entries = data.config_entries,
    };
}
```

The boundary rules use projectors named by the correlator config. These are not fake runtime

labels: they point to configuration data such as the Neumann projector and the bulk-boundary doubling projector.

```
fn boundaryFreeBosonDxDx(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(alpha, .{ .projector_metric = .{
        .projector = .{ .config = neumann_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, holomorphicDifference(), -2, .{ .include_left = true, .include_right = true });
}

fn boundaryFreeBosonDxExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.neg(scalars.i(alpha)), .{ .projector_momentum_index
    ↪  = .{
        .projector = .{ .config = neumann_projector_ref },
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, holomorphicDifference(), -1, .{ .include_left = true });
}

fn boundaryFreeBosonDxProfile(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPole(scalars.neg(alpha), .none, holomorphicDifference(),
    ↪  -1, .{ .include_left = true }, &.{
        wick.projectedProfileDerivative(.{ .config = neumann_projector_ref },
        ↪  wick.label(.right, 0), wick.label(.left, 0)),
    });
}

fn boundaryFreeBosonExpXExpX(comptime alpha: RuleScalar) wick.Expr {
    return wick.greenExponential(alpha, .{ .projector_momentum_pair = .{
        .projector = .{ .config = neumann_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, holomorphicDifference());
}

fn mixedFreeBosonDxDxBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(alpha, 2), .{ .projector_metric = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, bulkToBoundaryHolomorphicDifference(), -2, .{ .include_left = true, .include_right =
    ↪  true });
}

fn mixedFreeBosonDxtDxBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(alpha, 2), .{ .projector_metric = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, bulkToBoundaryAntiholomorphicDifference(), -2, .{ .include_left = true,
    ↪  .include_right = true });
}

fn mixedFreeBosonDxExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(scalars.i(alpha)), 2), .{
    ↪  .projector_momentum_index = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, bulkToBoundaryHolomorphicDifference(), -1, .{ .include_left = true });
}

fn mixedFreeBosonDxProfileBoundary(comptime alpha: RuleScalar) wick.Expr {
```

```
    return wick.differentiatedActionPole(scalars.div(scalars.neg(alpha), 2), .none,
    ↪  bulkToBoundaryHolomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.projectedProfileDerivative(.{ .config = bulk_boundary_projector_ref },
        ↪  wick.label(.right, 0), wick.label(.left, 0)),
    });
}

fn mixedFreeBosonDxtExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedPole(scalars.div(scalars.neg(scalars.i(alpha)), 2), .{
    ↪  .projector_momentum_index = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .momentum = wick.label(.right, 0),
        .index = wick.label(.left, 0),
    } }, bulkToBoundaryAntiholomorphicDifference(), -1, .{ .include_left = true });
}

fn mixedFreeBosonDxtProfileBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.differentiatedActionPole(scalars.div(scalars.neg(alpha), 2), .none,
    ↪  bulkToBoundaryAntiholomorphicDifference(), -1, .{ .include_left = true }, &.{
        wick.projectedProfileDerivative(.{ .config = bulk_boundary_projector_ref },
        ↪  wick.label(.right, 0), wick.label(.left, 0)),
    });
}

fn mixedFreeBosonExpXExpXBoundary(comptime alpha: RuleScalar) wick.Expr {
    return wick.greenExponential(scalars.div(alpha, 2), .{ .projector_momentum_pair = .{
        .projector = .{ .config = bulk_boundary_projector_ref },
        .left = wick.label(.left, 0),
        .right = wick.label(.right, 0),
    } }, bulkToBoundaryHolomorphicDifference());
}

fn diskWickStorage(comptime alpha: RuleScalar) [11]wick.Rule {
    return [_]wick.Rule{
        wick.rule(dXBoundaryPattern(), dXBoundaryPattern(), boundaryFreeBosonDxDx(alpha)),
        wick.rule(dXBoundaryPattern(), expXBoundaryPattern(), boundaryFreeBosonDxExpX(alpha)),
        wick.rule(dXBoundaryPattern(), profileBoundaryPattern(),
        ↪  boundaryFreeBosonDxProfile(alpha)),
        wick.rule(expXBoundaryPattern(), expXBoundaryPattern(),
        ↪  boundaryFreeBosonExpXExpX(alpha)),
        wick.rule(dXPattern(), dXBoundaryPattern(), mixedFreeBosonDxDxBoundary(alpha)),
        wick.rule(dXtPattern(), dXBoundaryPattern(), mixedFreeBosonDxtDxBoundary(alpha)),
        wick.rule(dXPattern(), expXBoundaryPattern(), mixedFreeBosonDxExpXBoundary(alpha)),
        wick.rule(dXtPattern(), expXBoundaryPattern(), mixedFreeBosonDxtExpXBoundary(alpha)),
        wick.rule(dXPattern(), profileBoundaryPattern(),
        ↪  mixedFreeBosonDxProfileBoundary(alpha)),
        wick.rule(dXtPattern(), profileBoundaryPattern(),
        ↪  mixedFreeBosonDxtProfileBoundary(alpha)),
        wick.rule(expXPattern(), expXBoundaryPattern(),
        ↪  mixedFreeBosonExpXExpXBoundary(alpha)),
    };
}

fn diskZeroStorage(comptime cfg: FreeBosonBoundaryConfig) [1]zero_mode.Rule {
    return [_]zero_mode.Rule{
        zero_mode.rule(zeroSector(.boundary), .{ .free_boson_constant_mode = .{
            .integration_projector = neumann_projector_ref,
            .fixed_projector = dirichlet_projector_ref,
            .fixed_position = dirichlet_position_ref,
            .exp_kind_ids = &.{ kind(.exp_x), kind(.exp_x_boundary) },
            .profile_kind_ids = &.{ kind(.profile_x), kind(.profile_x_boundary) },
            .normalization = .{
                .scalar = cfg.zero_mode_normalization,
                .two_pi_power = .{ .projector_rank = neumann_projector_ref },
```

```
            },
        } }),
    };
}

fn FreeBosonBoundaryData(comptime cfg: FreeBosonBoundaryConfig) type {
    const alpha = scalars.atomScalar(cfg.alpha_prime);
    return struct {
        const disk_wick_storage = diskWickStorage(alpha);
        const disk_zero_storage = diskZeroStorage(cfg);
        const config_storage = if (cfg.chan_paton) |stack| [_]shared.ConfigEntry{
            .{ .id = neumann_projector_ref, .value = .{ .tensor_projector = cfg.neumann } },
            .{ .id = dirichlet_projector_ref, .value = .{ .tensor_projector = cfg.dirichlet }
            ↪ },
            .{ .id = bulk_boundary_projector_ref, .value = .{ .tensor_projector = cfg.neumann
            ↪ } },
            .{ .id = dirichlet_position_ref, .value = .{ .target_point =
            ↪  cfg.dirichlet_position } },
            .{ .id = chan_paton_ref, .value = .{ .boundary_stack = stack } },
        } else [_]shared.ConfigEntry{
            .{ .id = neumann_projector_ref, .value = .{ .tensor_projector = cfg.neumann } },
            .{ .id = dirichlet_projector_ref, .value = .{ .tensor_projector = cfg.dirichlet }
            ↪ },
            .{ .id = bulk_boundary_projector_ref, .value = .{ .tensor_projector = cfg.neumann
            ↪ } },
            .{ .id = dirichlet_position_ref, .value = .{ .target_point =
            ↪  cfg.dirichlet_position } },
        };

        /// disk_wick lists boundary and mixed free-boson Wick rules on the disk.
        pub const disk_wick: []const wick.Rule = &disk_wick_storage;
        /// disk_zero_modes lists free-boson constant-mode rules on the disk.
        pub const disk_zero_modes: []const zero_mode.Rule = &disk_zero_storage;
        /// config_entries lists typed Neumann, Dirichlet, and Chan-Paton boundary data.
        pub const config_entries: []const shared.ConfigEntry = &config_storage;
    };
}
```

3.5 bc Preset

Interface

Functions

`bcSphere(comptime cfg: BcSphereConfig) type`
bcSphere builds the generated preset type for the sphere bc ghost CFT.

`boundaryExtension(comptime cfg: BcDiskConfig) BoundaryExtension`
boundaryExtension declares boundary and mixed bulk-boundary bc rules on the disk.

Types

`BcSphereConfig (struct)`
BcSphereConfig declares the holomorphic bc system and optional antiholomorphic copy.

`BcDiskConfig (struct)`
BcDiskConfig declares the disk boundary extension for the bc ghost system.

This node defines the *bc* ghost preset on the sphere and its disk boundary extension. It is independent of the matter CFT and uses the shared operator/Wick vocabulary from Preset Shared Runtime.

The local OPE is

$$b(z)c(0) \sim \frac{1}{z},$$

with weights $h_b = 2$, $h_c = -1$. The sphere top form requires three c-ghost insertions in each chiral copy [2].

```
const std = @import("std");
const shared = @import("shared.zig");

const operators = @import("../expressions/operators.zig");
const kernel = @import("../kernel.zig");
const Handle = shared.Handle;
const CorrelatorConfig = shared.CorrelatorConfig;
const BoundaryExtension = shared.BoundaryExtension;
const Local = shared.Local;
const RuleScalar = shared.RuleScalar;
const wick = shared.wick;
const zero_mode = shared.zero_mode;
```

The ghost preset owns the b, c operator vocabulary. It uses a separate id block from the free boson and derives each concrete kind id from the enum tag, so this file names physics families without maintaining numeric ids per operator.

```
const namespace = "bc";

const Family = enum {
    b,
    c,
    bt,
    ct,
    b_boundary,
    c_boundary,
};

const ZeroSector = enum {
    sphere,
    disk,
};

fn kind(comptime family: Family) operators.OperatorKindId {
    return shared.familyKind(namespace, family);
}

fn zeroSector(comptime sector: ZeroSector) zero_mode.Sector {
    return shared.sectorId(namespace, sector);
}

fn rawToken(value: anytype) u32 {
    return @intFromEnum(value);
}

fn token(comptime T: type, id: u32) T {
    return @enumFromInt(if (id == 0) 1 else id);
}

fn coordVariable(coord: Handle.Coord) u32 {
    return rawToken(coord);
}

fn singleInsertion(z: Handle.Coord, derivatives: u8) operators.OperatorInsertion {
    return .{ .single = .{
        .position = coordVariable(z),
        .derivatives = derivatives,
```

```
    } };
}

fn boundaryCoord(y: Handle.BoundaryCoord) Handle.Coord {
    return token(Handle.Coord, rawToken(y));
}

fn bcPattern(comptime family: Family, comptime support: wick.Support) wick.Pattern {
    return wick.pattern(kind(family), support);
}

fn bPattern() wick.Pattern {
    return bcPattern(.b, .holomorphic);
}

fn cPattern() wick.Pattern {
    return bcPattern(.c, .holomorphic);
}

fn btPattern() wick.Pattern {
    return bcPattern(.bt, .antiholomorphic);
}

fn ctPattern() wick.Pattern {
    return bcPattern(.ct, .antiholomorphic);
}

fn bBoundaryPattern() wick.Pattern {
    return bcPattern(.b_boundary, .boundary);
}

fn cBoundaryPattern() wick.Pattern {
    return bcPattern(.c_boundary, .boundary);
}
```

3.5.1 Sphere Preset

The sphere preset exposes a holomorphic copy and, by default, an antiholomorphic copy. The config flag changes both the builder namespace and the emitted rule slice at compile time.

```
/// BcSphereConfig declares the holomorphic bc system and optional antiholomorphic copy.
pub const BcSphereConfig = struct {
    include_antiholomorphic_copy: bool = true,
    top_form_normalization: RuleScalar = .one,
};

/// bcSphere builds the generated preset type for the sphere bc ghost CFT.
pub fn bcSphere(comptime cfg: BcSphereConfig) type {
    return struct {
        /// is_bc_sphere_preset identifies this generated type for composition.
        pub const is_bc_sphere_preset = true;

        /// op exposes bc ghost local operator builders.
        pub const op = BcSphereOp(cfg.include_antiholomorphic_copy);
        /// config exposes named correlator configs for this preset.
        pub const config = BcSphereCorrelatorConfig(cfg);
        /// rules exposes the preset-authored Wick rule templates.
        pub const rules = BcSphereRules(cfg.include_antiholomorphic_copy);

        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) Local {
            return Local.init(allocator);
```

```
        }

        /// correlator streams rule matches for bc insertions.
        pub fn correlator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink:
        ↪  anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}
```

Derivative labels mean $\partial^n b$, $\partial^n c$, and their antiholomorphic copies. No ghost-number bookkeeping is performed here; that is part of theory data and later basis or correlator evaluation.

```
fn BcSphereOp(comptime include_antiholomorphic_copy: bool) type {
    const Holomorphic = struct {
        /// b builds a holomorphic b-ghost insertion.
        pub fn b(local: *Local, n: u8, z: Handle.Coord) !kernel.Call.LocalOp {
            return local.op(kind(.b), singleInsertion(z, n), &.{});
        }

        /// c builds a holomorphic c-ghost insertion.
        pub fn c(local: *Local, n: u8, z: Handle.Coord) !kernel.Call.LocalOp {
            return local.op(kind(.c), singleInsertion(z, n), &.{});
        }
    };

    if (!include_antiholomorphic_copy) return Holomorphic;

    return struct {
        /// b builds a holomorphic b-ghost insertion.
        pub const b = Holomorphic.b;
        /// c builds a holomorphic c-ghost insertion.
        pub const c = Holomorphic.c;

        /// bt builds an antiholomorphic b-ghost insertion.
        pub fn bt(local: *Local, n: u8, zbar: Handle.Coord) !kernel.Call.LocalOp {
            return local.op(kind(.bt), singleInsertion(zbar, n), &.{});
        }

        /// ct builds an antiholomorphic c-ghost insertion.
        pub fn ct(local: *Local, n: u8, zbar: Handle.Coord) !kernel.Call.LocalOp {
            return local.op(kind(.ct), singleInsertion(zbar, n), &.{});
        }
    };
}
```

3.5.2 Rules

The *bc* Wick rules have no tensor numerator. Their zero-mode rule records the top-form saturation separately from pairwise contractions.

```
fn holomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn antiholomorphicDifference() wick.CoordinateDifference {
    return wick.difference(wick.coord(.left, .position), wick.coord(.right, .position));
}

fn bcBC() wick.Expr {
    return wick.differentiatedPole(.one, .none, holomorphicDifference(), -1, .{ .include_left
    ↪  = true, .include_right = true });
}
```

```
fn bcBtCt() wick.Expr {
    return wick.differentiatedPole(.one, .none, antiholomorphicDifference(), -1, .{
    ↪  .include_left = true, .include_right = true });
}

const sphere_holomorphic_wick_storage = [_]wick.Rule{
    wick.rule(bPattern(), cPattern(), bcBC()),
};

const sphere_full_wick_storage = sphere_holomorphic_wick_storage ++ [_]wick.Rule{
    wick.rule(btPattern(), ctPattern(), bcBtCt()),
};

fn sphereZeroStorage(comptime cfg: BcSphereConfig) if (cfg.include_antiholomorphic_copy)
↪  [2]zero_mode.Rule else [1]zero_mode.Rule {
    const holomorphic = zero_mode.rule(zeroSector(.sphere), .{ .bc_top_form = .{
        .support = .sphere_holomorphic,
        .c_kind_ids = &.{kind(.c)},
        .normalization = cfg.top_form_normalization,
    } });
    if (!cfg.include_antiholomorphic_copy) return [_]zero_mode.Rule{holomorphic};
    return [_]zero_mode.Rule{
        holomorphic,
        zero_mode.rule(zeroSector(.sphere), .{ .bc_top_form = .{
            .support = .sphere_antiholomorphic,
            .c_kind_ids = &.{kind(.ct)},
            .normalization = cfg.top_form_normalization,
        } }),
    };
}

fn BcSphereRules(comptime include_antiholomorphic_copy: bool) type {
    const selected_wick = if (include_antiholomorphic_copy) sphere_full_wick_storage else
    ↪  sphere_holomorphic_wick_storage;
    return struct {
        /// sphere_wick lists the primitive bc Wick rules on the sphere.
        pub const sphere_wick: []const wick.Rule = &selected_wick;
    };
}

fn BcSphereCorrelatorConfig(comptime cfg: BcSphereConfig) type {
    const rules = BcSphereRules(cfg.include_antiholomorphic_copy);
    return struct {
        const sphere_zero_storage = sphereZeroStorage(cfg);
        const sphere_wick_index_storage = shared.wickRuleIndexStorage(rules.sphere_wick);

        /// sphere selects bc sphere Wick and zero-mode rules.
        pub const sphere = CorrelatorConfig{
            .wick_rules = rules.sphere_wick,
            .wick_rule_index = &sphere_wick_index_storage,
            .zero_modes = &sphere_zero_storage,
        };
    };
}
```

3.5.3 Disk Boundary

The disk extension adds boundary-local b, c operators and mixed bulk-boundary contractions. It is an extension chosen by a BCFT composition, not a modification of the bulk bc preset.

```
/// BcDiskConfig declares the disk boundary extension for the bc ghost system.
pub const BcDiskConfig = struct {
```

```
    include_mixed_bulk_boundary: bool = true,
    top_form_normalization: RuleScalar = .one,
};

const BcDiskBoundaryOp = struct {
    /// bBoundary builds a boundary b-ghost insertion.
    pub fn bBoundary(local: *Local, n: u8, y: Handle.BoundaryCoord) !kernel.Call.LocalOp {
        return local.op(kind(.b_boundary), singleInsertion(boundaryCoord(y), n), &.{});
    }

    /// cBoundary builds a boundary c-ghost insertion.
    pub fn cBoundary(local: *Local, n: u8, y: Handle.BoundaryCoord) !kernel.Call.LocalOp {
        return local.op(kind(.c_boundary), singleInsertion(boundaryCoord(y), n), &.{});
    }
};

/// boundaryExtension declares boundary and mixed bulk-boundary bc rules on the disk.
pub fn boundaryExtension(comptime cfg: BcDiskConfig) BoundaryExtension {
    const data = BcDiskData(cfg);
    return .{
        .kind = shared.extensionKind("bc_disk"),
        .op = BcDiskBoundaryOp,
        .wick_rules = if (cfg.include_mixed_bulk_boundary) &disk_full_wick_storage else
        ↪  &disk_boundary_wick_storage,
        .zero_modes = data.disk_zero_modes,
    };
}

const disk_boundary_wick_storage = [_]wick.Rule{
    wick.rule(bBoundaryPattern(), cBoundaryPattern(), bcBC()),
};

const disk_full_wick_storage = disk_boundary_wick_storage ++ [_]wick.Rule{
    wick.rule(bPattern(), cBoundaryPattern(), bcBC()),
    wick.rule(cPattern(), bBoundaryPattern(), bcBC()),
    wick.rule(btPattern(), cBoundaryPattern(), bcBtCt()),
    wick.rule(ctPattern(), bBoundaryPattern(), bcBtCt()),
};

fn diskZeroStorage(comptime cfg: BcDiskConfig) [1]zero_mode.Rule {
    return [_]zero_mode.Rule{
        zero_mode.rule(zeroSector(.disk), .{ .bc_top_form = .{
            .support = .disk_doubled,
            .c_kind_ids = &.{ kind(.c), kind(.ct), kind(.c_boundary) },
            .normalization = cfg.top_form_normalization,
        } }),
    };
}

fn BcDiskData(comptime cfg: BcDiskConfig) type {
    return struct {
        const disk_zero_storage = diskZeroStorage(cfg);

        /// disk_zero_modes lists doubled chiral bc zero-mode rules on the disk.
        pub const disk_zero_modes: []const zero_mode.Rule = &disk_zero_storage;
    };
}
```

3.6 Preset Composition

Interface

`product(comptime factors: anytype) type`
: product builds the generated preset type for a product of independent bulk presets.

`boundary(comptime Bulk: type, comptime extensions: anytype) type`
: boundary builds the generated preset type for a BCFT from a bulk preset and boundary extensions.

Composition turns bulk presets such as Free Boson Preset and bc Preset into products and BCFTs. It uses the shared runtime vocabulary from Preset Shared Runtime.

The important invariant is that tensor products do not allocate a new runtime operator representation. Product quantum numbers remain lists at theory level; runtime local operators are still just insertion data, a body kind, and a label span. Composition therefore joins operator namespaces and concatenates rule slices at compile time.

```
const std = @import("std");
const shared = @import("shared.zig");

const kernel = @import("../kernel.zig");
const CorrelatorConfig = shared.CorrelatorConfig;
const Local = shared.Local;
const wick = shared.wick;
const zero_mode = shared.zero_mode;
```

3.6.1 Product CFTs

A product preset forms its rules from the generated sphere config of each factor. Convenience operator aliases are intentionally explicit: for example `product(.{FreeBoson})` has `op.free_boson` and no `op.bc`. Future presets can compose through their configs immediately; adding a friendly `op` namespace is a separate user-facing alias decision.

```
const EmptyOp = struct {};

const KnownOpNamespace = enum {
    free_boson,
    bc_sphere,
};

fn hasPresetKind(comptime factors: anytype, comptime kind: KnownOpNamespace) bool {
    inline for (factors) |Factor| {
        switch (kind) {
            .free_boson => if (@hasDecl(Factor, "is_free_boson_preset")) return true,
            .bc_sphere => if (@hasDecl(Factor, "is_bc_sphere_preset")) return true,
        }
    }
    return false;
}

fn FactorWithKind(comptime factors: anytype, comptime kind: KnownOpNamespace) type {
    inline for (factors) |Factor| {
        switch (kind) {
            .free_boson => if (@hasDecl(Factor, "is_free_boson_preset")) return Factor,
            .bc_sphere => if (@hasDecl(Factor, "is_bc_sphere_preset")) return Factor,
        }
    }
    unreachable;
}

fn factorHasSphereConfig(comptime Factor: type) bool {
    return @hasDecl(Factor, "config") and @hasDecl(Factor.config, "sphere");
}
```

```
fn productSphereWickCount(comptime factors: anytype) usize {
    var count: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) count += Factor.config.sphere.wick_rules.len;
    }
    return count;
}

fn productSphereZeroCount(comptime factors: anytype) usize {
    var count: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) count += Factor.config.sphere.zero_modes.len;
    }
    return count;
}

fn productSphereConfigCount(comptime factors: anytype) usize {
    var count: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) count += Factor.config.sphere.config_entries.len;
    }
    return count;
}

fn productSphereWickStorage(comptime factors: anytype)
↪   [productSphereWickCount(factors)]wick.Rule {
    var storage: [productSphereWickCount(factors)]wick.Rule = undefined;
    var index: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) {
            inline for (Factor.config.sphere.wick_rules) |rule| {
                storage[index] = rule;
                index += 1;
            }
        }
    }
    return storage;
}

fn productSphereZeroStorage(comptime factors: anytype)
↪   [productSphereZeroCount(factors)]zero_mode.Rule {
    var storage: [productSphereZeroCount(factors)]zero_mode.Rule = undefined;
    var index: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) {
            inline for (Factor.config.sphere.zero_modes) |rule| {
                storage[index] = rule;
                index += 1;
            }
        }
    }
    return storage;
}

fn productSphereConfigStorage(comptime factors: anytype)
↪   [productSphereConfigCount(factors)]shared.ConfigEntry {
    var storage: [productSphereConfigCount(factors)]shared.ConfigEntry = undefined;
    var index: usize = 0;
    inline for (factors) |Factor| {
        if (factorHasSphereConfig(Factor)) {
            inline for (Factor.config.sphere.config_entries) |entry| {
                storage[index] = entry;
                index += 1;
```

```
            }
        }
    }
    return storage;
}

fn ProductRules(comptime factors: anytype) type {
    return struct {
        const sphere_wick_storage = productSphereWickStorage(factors);
        const sphere_wick_index_storage = shared.wickRuleIndexStorage(&sphere_wick_storage);
        const sphere_zero_storage = productSphereZeroStorage(factors);
        const sphere_config_storage = productSphereConfigStorage(factors);

        /// sphere_wick lists primitive Wick rules contributed by product factors.
        pub const sphere_wick: []const wick.Rule = &sphere_wick_storage;
        /// sphere_wick_index maps ordered operator-kind pairs to product sphere rules.
        pub const sphere_wick_index: []const shared.WickRuleIndexEntry =
        ↪  &sphere_wick_index_storage;
        /// sphere_zero_modes lists zero-mode rules contributed by product factors.
        pub const sphere_zero_modes: []const zero_mode.Rule = &sphere_zero_storage;
        /// sphere_config_entries lists typed config values contributed by product factors.
        pub const sphere_config_entries: []const shared.ConfigEntry = &sphere_config_storage;
    };
}

fn ProductCorrelatorConfig(comptime factors: anytype) type {
    const rules = ProductRules(factors);
    return struct {
        /// sphere selects composed product Wick and zero-mode rules.
        pub const sphere = CorrelatorConfig{
            .wick_rules = rules.sphere_wick,
            .wick_rule_index = rules.sphere_wick_index,
            .zero_modes = rules.sphere_zero_modes,
            .config_entries = rules.sphere_config_entries,
        };
    };
}

fn ProductOp(comptime factors: anytype) type {
    const has_free_boson = hasPresetKind(factors, .free_boson);
    const has_bc = hasPresetKind(factors, .bc_sphere);
    if (has_free_boson and has_bc) {
        return struct {
            /// free_boson exposes free-boson builders contributed by the product.
            pub const free_boson = FactorWithKind(factors, .free_boson).op;
            /// bc exposes bc-ghost builders contributed by the product.
            pub const bc = FactorWithKind(factors, .bc_sphere).op;
        };
    }
    if (has_free_boson) {
        return struct {
            /// free_boson exposes free-boson builders contributed by the product.
            pub const free_boson = FactorWithKind(factors, .free_boson).op;
        };
    }
    if (has_bc) {
        return struct {
            /// bc exposes bc-ghost builders contributed by the product.
            pub const bc = FactorWithKind(factors, .bc_sphere).op;
        };
    }
    return EmptyOp;
}
```

```
/// product builds the generated preset type for a product of independent bulk presets.
pub fn product(comptime factors: anytype) type {
    return struct {
        /// op exposes the joined operator builders of the product factors.
        pub const op = ProductOp(factors);
        /// config exposes named correlator configs for this product preset.
        pub const config = ProductCorrelatorConfig(factors);
        /// rules exposes the preset-authored product Wick rule templates.
        pub const rules = ProductRules(factors);

        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) Local {
            return Local.init(allocator);
        }

        /// correlator streams rule matches for product-theory insertions.
        pub fn correlator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink:
        ↪  anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}
```

3.6.2 Boundary Extensions

A BCFT is built from a bulk preset plus explicit boundary extensions. The bulk preset remains boundary-free; boundary data contributes new operator families, mixed Wick rules, zero-mode caveats, and optional Chan-Paton stack data.

```
const free_boson_boundary_kind = shared.extensionKind("free_boson_boundary");
const bc_disk_kind = shared.extensionKind("bc_disk");

fn hasBoundaryExtension(comptime extensions: anytype, comptime kind: u32) bool {
    inline for (extensions) |extension| {
        if (extension.kind == kind) {
            return true;
        }
    }
    return false;
}

fn ExtensionWithKind(comptime extensions: anytype, comptime kind: u32) type {
    inline for (extensions) |extension| {
        if (extension.kind == kind) {
            return extension.op;
        }
    }
    unreachable;
}
```

The disk config is formed from the bulk config payload plus the rule slices and config payloads selected by boundary extensions. Chan-Paton stacks enter through the same typed config stream as projectors and target positions.

```
fn boundaryWickCount(comptime extensions: anytype) usize {
    var count: usize = 0;
    inline for (extensions) |extension| {
        count += extension.wick_rules.len;
    }
    return count;
}
```

```
fn boundaryZeroCount(comptime extensions: anytype) usize {
    var count: usize = 0;
    inline for (extensions) |extension| {
        count += extension.zero_modes.len;
    }
    return count;
}

fn boundaryConfigCount(comptime Bulk: type, comptime extensions: anytype) usize {
    var count: usize = if (factorHasSphereConfig(Bulk)) Bulk.config.sphere.config_entries.len
    ↪  else 0;
    inline for (extensions) |extension| {
        count += extension.config_entries.len;
    }
    return count;
}

fn boundaryWickStorage(comptime extensions: anytype) [boundaryWickCount(extensions)]wick.Rule
↪  {
    var storage: [boundaryWickCount(extensions)]wick.Rule = undefined;
    var index: usize = 0;
    inline for (extensions) |extension| {
        inline for (extension.wick_rules) |rule| {
            storage[index] = rule;
            index += 1;
        }
    }
    return storage;
}

fn boundaryZeroStorage(comptime extensions: anytype)
↪  [boundaryZeroCount(extensions)]zero_mode.Rule {
    var storage: [boundaryZeroCount(extensions)]zero_mode.Rule = undefined;
    var index: usize = 0;
    inline for (extensions) |extension| {
        inline for (extension.zero_modes) |rule| {
            storage[index] = rule;
            index += 1;
        }
    }
    return storage;
}

fn boundaryConfigStorage(comptime Bulk: type, comptime extensions: anytype)
↪  [boundaryConfigCount(Bulk, extensions)]shared.ConfigEntry {
    var storage: [boundaryConfigCount(Bulk, extensions)]shared.ConfigEntry = undefined;
    var index: usize = 0;
    if (factorHasSphereConfig(Bulk)) {
        inline for (Bulk.config.sphere.config_entries) |entry| {
            storage[index] = entry;
            index += 1;
        }
    }
    inline for (extensions) |extension| {
        inline for (extension.config_entries) |entry| {
            storage[index] = entry;
            index += 1;
        }
    }
    return storage;
}

fn BoundaryRules(comptime Bulk: type, comptime extensions: anytype) type {
    return struct {
```

```
        const disk_wick_storage = boundaryWickStorage(extensions);
        const disk_wick_index_storage = shared.wickRuleIndexStorage(&disk_wick_storage);
        const disk_zero_storage = boundaryZeroStorage(extensions);
        const disk_config_storage = boundaryConfigStorage(Bulk, extensions);

        /// disk_wick lists primitive Wick rules contributed by boundary extensions.
        pub const disk_wick: []const wick.Rule = &disk_wick_storage;
        /// disk_wick_index maps ordered operator-kind pairs to composed disk rules.
        pub const disk_wick_index: []const shared.WickRuleIndexEntry =
        ↪  &disk_wick_index_storage;
        /// disk_zero_modes lists zero-mode rules contributed by boundary extensions.
        pub const disk_zero_modes: []const zero_mode.Rule = &disk_zero_storage;
        /// disk_config_entries lists typed config values contributed by the bulk and boundary
        ↪  extensions.
        pub const disk_config_entries: []const shared.ConfigEntry = &disk_config_storage;
    };
}

fn BoundaryCorrelatorConfig(comptime Bulk: type, comptime extensions: anytype) type {
    const rules = BoundaryRules(Bulk, extensions);
    return struct {
        /// disk selects composed boundary Wick and zero-mode rules.
        pub const disk = CorrelatorConfig{
            .wick_rules = rules.disk_wick,
            .wick_rule_index = rules.disk_wick_index,
            .zero_modes = rules.disk_zero_modes,
            .config_entries = rules.disk_config_entries,
        };
    };
}

fn BoundaryOp(comptime Bulk: type, comptime extensions: anytype) type {
    const has_free_boson_boundary = hasBoundaryExtension(extensions,
    ↪  free_boson_boundary_kind);
    const has_bc_boundary = hasBoundaryExtension(extensions, bc_disk_kind);
    if (has_free_boson_boundary and has_bc_boundary) {
        return struct {
            /// bulk exposes the bulk operator namespaces.
            pub const bulk = Bulk.op;
            /// free_boson_boundary exposes selected free-boson boundary builders.
            pub const free_boson_boundary = ExtensionWithKind(extensions,
            ↪  free_boson_boundary_kind);
            /// bc_boundary exposes selected bc boundary builders.
            pub const bc_boundary = ExtensionWithKind(extensions, bc_disk_kind);
        };
    }
    if (has_free_boson_boundary) {
        return struct {
            /// bulk exposes the bulk operator namespaces.
            pub const bulk = Bulk.op;
            /// free_boson_boundary exposes selected free-boson boundary builders.
            pub const free_boson_boundary = ExtensionWithKind(extensions,
            ↪  free_boson_boundary_kind);
        };
    }
    if (has_bc_boundary) {
        return struct {
            /// bulk exposes the bulk operator namespaces.
            pub const bulk = Bulk.op;
            /// bc_boundary exposes selected bc boundary builders.
            pub const bc_boundary = ExtensionWithKind(extensions, bc_disk_kind);
        };
    }
    return struct {
```

```
        /// bulk exposes the bulk operator namespaces.
        pub const bulk = Bulk.op;
    };
}

/// boundary builds the generated preset type for a BCFT from a bulk preset and boundary
↪  extensions.
pub fn boundary(comptime Bulk: type, comptime extensions: anytype) type {
    return struct {
        /// op exposes bulk and boundary operator builders.
        pub const op = BoundaryOp(Bulk, extensions);
        /// config exposes named correlator configs for this BCFT.
        pub const config = BoundaryCorrelatorConfig(Bulk, extensions);
        /// rules exposes the preset-authored boundary Wick rule templates.
        pub const rules = BoundaryRules(Bulk, extensions);

        /// local constructs a label-preserving local-operator builder.
        pub fn local(allocator: std.mem.Allocator) Local {
            return Local.init(allocator);
        }

        /// correlator streams rule matches for BCFT insertions.
        pub fn correlator(config_ptr: *const CorrelatorConfig, ops: kernel.Call.MultiOp, sink:
        ↪  anytype) !void {
            return shared.streamCorrelator(config_ptr, ops, sink);
        }
    };
}
```

4 Normal-Ordering

This module defines utilities related to normal-ordering of Operators.

5 Correlators

This module defines the notion of a correlator, which takes symbolic Expressions and computes their correlator (a Coefficient) via several possible tactics inferred from the definition of the Theory involved.

The base tactics are

- Abstract: the correlator remains unevaluated, only returns zero if selection rules as inferred through Tensor-Code from the quantum numbers of the inputs (they must intertwine to a singlet modulo possible anomalies, which can vary from one Riemann surface to another i.e. ghost number adding up to 3 on the sphere for bc system) are not met.

- Wick: the correlator is evaluated by Wick contractions until a base case is reached. Both the value of the Wick propagator and what the base case is can depend on the Riemann surface, but the underlying abstract algorithm remains the same. What we mean by base case is that the correlator dependence on zero-modes cannot quite be inferred by a Wick contraction logic (e.g. why the base case <ccc> is not zero for c-ghosts of bc system on a sphere, note that e.g. the base case <c ∂ c c> follows). The zero mode data is specified by the user, just as the Wick contractions are surface-by-surface, yet the underlying algorithm remains fixed.

- Bosonization: the correlator is evaluated by writing out all tensor structures of Tensor-Code that can appear in the result and matching their coefficients by plugging in particular values of group-theoretic indices to both sides of the correlator, evaluating the tensor structures numerically and evaluating the correlator via bosonization rules and the Wick tactic on the bosonized form of the operators.

6 OPE

This module defines the notion of an OPE, which takes symbolic Expressions and computes their OPE (again Expressions) via several possible tactics inferred from the definition of the Theory involved.

Since the OPE is a local expression, we evaluate it via the definition of the theory on the Riemann sphere. When the user defines a non-degenerate pairing (for a unitary CFT, can be the usual BPZ pairing) on the basis of Operators, we can evaluate the OPE by writing out all Operators that can appear in the OPE (we know the quantum numbers of the inputs so we know the quantum numbers of the output, being in their tensor product) via Basis-Generation and compute their coefficients by combining the pairing with Correlators. This way, we can immediately transfer optimizations of Correlators and Basis-Generation to an optimized OPE.

Tensor-Code

API

Types

`SimpleLieAlgebra = symmetry.SimpleLieAlgebra`
SimpleLieAlgebra stores the simple family and rank.

`LieFamily = symmetry.LieFamily`
LieFamily names the supported simple Lie families.

`QuantumNumbers = symmetry.QuantumNumbers`
QuantumNumbers is the tensor-product list of representation ids.

`RepresentationId = symmetry.RepresentationId`
RepresentationId names a representation in a representation store.

`TensorStructure = symmetry.TensorStructure`
TensorStructure stores simple built-in tensor structures.

`AlgebraHandle = store.AlgebraHandle`
AlgebraHandle names a registered Lie algebra in a tensor context.

`IrrepHandle = store.IrrepHandle`
IrrepHandle names an abstract irreducible representation.

`RealizationHandle = realization.RealizationHandle`
RealizationHandle names a concrete index realization of an irrep.

`RealizationChannel = realization.RealizationChannel`
RealizationChannel selects one irrep copy inside a realization product.

`BasisHandle = coupling.BasisHandle`
BasisHandle names a generated invariant basis.

`InvariantHandle = coupling.InvariantHandle`
InvariantHandle names one invariant inside a basis.

`Context = context.Context`
Context is an opaque handle to tensor-code stores and caches.

`ContextOptions = context.ContextOptions`
ContextOptions configures algebra and rendering defaults.

`AlgebraConventions = root_data.AlgebraConventions`
AlgebraConventions selects configurable exact-algebra normalizations.

`AlgebraSpec = store.AlgebraSpec`

AlgebraSpec describes a requested algebra before it is interned.

`IrrepSpec = store.IrrepSpec`
IrrepSpec describes a requested representation before it is interned.

`ExternalLeg = realization.ExternalLeg`
ExternalLeg describes a public invariant leg with named free indices.

`RealizationSpec = realization.RealizationSpec`
RealizationSpec describes a primitive or projected external index model.

`InvariantBasisRequest = coupling.InvariantBasisRequest`
InvariantBasisRequest is the CFT-facing request for invariant tensors.

`TensorExpansionTerm = coupling.TensorExpansionTerm`
TensorExpansionTerm stores a CFT coefficient attached to one invariant id.

`RenderOptions = rendering.RenderOptions`
RenderOptions controls streamed invariant rendering.

`EvalOptions = rendering.EvalOptions`
EvalOptions controls component evaluation of one invariant.

`SymbolicTerm = rendering.SymbolicTerm`
SymbolicTerm stores one streamed coefficient times symbolic atoms.

`SymbolicAtom = rendering.SymbolicAtom`
SymbolicAtom stores one symbolic invariant contraction atom.

`TensorExprId = kernel.TensorExprId`
TensorExprId names a tensor expression stored by tensor-code.

This module handles finite-dimensional representation data for the symmetry algebras used by the other modules. For a Lie algebra $\mathfrak{g}$, the central problem is to construct bases of

$$\mathrm{Hom}_{\mathfrak{g}}\left(T,\ R_1 \otimes \cdots \otimes R_k\right),$$

usually with $T = \mathbf{1}$. These bases are Clebsch-Gordan data and invariant tensors. Concrete index notation, such as gamma matrices, Young symmetrizers, metrics, epsilons, or structure constants, is a rendering layer over this representation-theoretic basis.

The public module exposes named declarations directly. Source files such as Symmetry and Tensor Kernel organize implementation and prose, but clients should import this node and use the individual public declarations below. The caller-facing object is `Context`: it registers algebras, irreps, and realizations, then returns compact handles for invariant bases and rendered/evaluated output streams.

```
const context = @import("context.zig");
const coupling = @import("coupling.zig");
const kernel = @import("kernel.zig");
const realization = @import("realization.zig");
const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");

/// SimpleLieAlgebra stores the simple family and rank.
pub const SimpleLieAlgebra = symmetry.SimpleLieAlgebra;
/// LieFamily names the supported simple Lie families.
pub const LieFamily = symmetry.LieFamily;
/// QuantumNumbers is the tensor-product list of representation ids.
pub const QuantumNumbers = symmetry.QuantumNumbers;
/// RepresentationId names a representation in a representation store.
pub const RepresentationId = symmetry.RepresentationId;
/// TensorStructure stores simple built-in tensor structures.
pub const TensorStructure = symmetry.TensorStructure;
```

```
/// AlgebraHandle names a registered Lie algebra in a tensor context.
pub const AlgebraHandle = store.AlgebraHandle;
/// IrrepHandle names an abstract irreducible representation.
pub const IrrepHandle = store.IrrepHandle;
/// RealizationHandle names a concrete index realization of an irrep.
pub const RealizationHandle = realization.RealizationHandle;
/// RealizationChannel selects one irrep copy inside a realization product.
pub const RealizationChannel = realization.RealizationChannel;
/// BasisHandle names a generated invariant basis.
pub const BasisHandle = coupling.BasisHandle;
/// InvariantHandle names one invariant inside a basis.
pub const InvariantHandle = coupling.InvariantHandle;
/// Context is an opaque handle to tensor-code stores and caches.
pub const Context = context.Context;
/// ContextOptions configures algebra and rendering defaults.
pub const ContextOptions = context.ContextOptions;
/// AlgebraConventions selects configurable exact-algebra normalizations.
pub const AlgebraConventions = root_data.AlgebraConventions;
/// AlgebraSpec describes a requested algebra before it is interned.
pub const AlgebraSpec = store.AlgebraSpec;
/// IrrepSpec describes a requested representation before it is interned.
pub const IrrepSpec = store.IrrepSpec;
/// ExternalLeg describes a public invariant leg with named free indices.
pub const ExternalLeg = realization.ExternalLeg;
/// RealizationSpec describes a primitive or projected external index model.
pub const RealizationSpec = realization.RealizationSpec;
/// InvariantBasisRequest is the CFT-facing request for invariant tensors.
pub const InvariantBasisRequest = coupling.InvariantBasisRequest;
/// TensorExpansionTerm stores a CFT coefficient attached to one invariant id.
pub const TensorExpansionTerm = coupling.TensorExpansionTerm;
/// RenderOptions controls streamed invariant rendering.
pub const RenderOptions = rendering.RenderOptions;
/// EvalOptions controls component evaluation of one invariant.
pub const EvalOptions = rendering.EvalOptions;
/// SymbolicTerm stores one streamed coefficient times symbolic atoms.
pub const SymbolicTerm = rendering.SymbolicTerm;
/// SymbolicAtom stores one symbolic invariant contraction atom.
pub const SymbolicAtom = rendering.SymbolicAtom;

/// TensorExprId names a tensor expression stored by tensor-code.
pub const TensorExprId = kernel.TensorExprId;
```

This root block is the public tensor-code API for now. The compatibility exports `QuantumNumbers`, `RepresentationId`, and `TensorStructure` come from Symmetry, while `TensorExprId` comes from Tensor Kernel. They exist because CFT data structures currently store compact tensor and representation ids.

The central workflow is:

1. register an algebra with `Context.registerAlgebra`;
2. register abstract irreps by Dynkin labels with `Context.registerIrrep`;
3. optionally register concrete index models with `Context.registerRealization`;
4. request a basis with `Context.invariantBasis`;
5. stream a concrete convention with `Context.renderInvariant` or `Context.evaluateInvariant`.

The constructors are deliberately on the public data records, so callers do not need to import implementation helpers:

```
var ctx = try tensor.Context.init(allocator);
defer ctx.deinit();
const algebra = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
const spinor = try ctx.registerIrrep(algebra, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
const leg = tensor.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = algebra,
    .external_legs = &.{leg},
});
```

Conventions are configurable at context construction:

```
var ctx = try tensor.Context.initWithOptions(allocator, .{
    .algebra_conventions = .{
        .quadratic_casimir_scale = .{ .numerator = 1, .denominator = 1 },
    },
    .default_signature = .complex,
});
defer ctx.deinit();
```

Helper enums inside requests and render options are intentionally not root exports; callers can use inferred literals such as `.singlet`, `.auto`, `.vector`, and `.complex`. Cache handles, decomposition handles, channel handles, stream sources, schema records, and backend capability tables stay behind this boundary until a caller needs them.

1 Public API Glossary

- Lie algebra $\mathfrak{g}$: the symmetry algebra, such as A_n, D_n, or E_8. Tensor-code uses $\mathfrak{g}$ to decide which tensor structures are invariant.
- Root datum: the Cartan matrix, simple roots, coroots, and weight lattice conventions used internally for $\mathfrak{g}$. CFT code should not handle this directly.
- Irrep: an irreducible representation of $\mathfrak{g}$. For simple Lie algebras this is identified by Dynkin labels, not by dimension.
- Dynkin label: the integer highest-weight label of an irrep. It is the stable representation id used by the mathematics layer.
- Tensor product decomposition: the expansion

$$R \otimes S = \bigoplus_T m_T T$$

  into irreps T with multiplicities m_T. Tensor-code caches this compactly; it does not expand component tensors during decomposition.
- Multiplicity: the number of independent copies of the same irrep inside a product. A dimension is never enough to distinguish these copies.
- Channel: one copy of an irrep inside a tensor product. If $R \otimes S$ contains two copies of T, those two copies have the same irrep but different channel copy labels.
- Realization: a concrete index model for an irrep. A realization may be primitive, such as one spinor index, or constrained, such as a traceless or projected tensor product.
- Backend: the internal family-specific capability table described in Tensor Backend. It supplies decomposition, local projector, primitive invariant, and rendering routines for a Lie family without changing the public request flow.

- Primitive realization: a representation whose public indices are already the natural storage indices, such as a vector index, spinor index, fundamental index, or adjoint index.
- Constrained realization: a representation embedded in a larger ambient tensor product, such as the $144 \subset 10 \otimes 16$ gamma-traceless channel. Tensor-code records the boundary projector once and then treats the leg as the irrep, not as the full ambient product at every internal step.
- Index: a caller-named free slot in a realization. An index name is not a representation by itself.
- External leg: one field/operator tensor slot supplied by a caller. It has an irrep, a realization, and named free indices.
- Intertwiner: a $\mathfrak{g}$-equivariant map between representations. Clebsch-Gordan coefficients are component formulas for such maps.
- Invariant: a basis vector in
$$\mathrm{Hom}_{\mathfrak{g}}\left(T,\ R_1 \otimes \cdots \otimes R_k\right).$$
Usually $T = \mathbf{1}$, so the invariant is a singlet tensor.
- Coupling path: the compact internal description of one invariant as a tree of local Clebsch-Gordan maps.
- Projector: an idempotent intertwiner selecting one channel or constrained realization from an ambient product. Projectors may be rendered as named operators, block formulas, or fully expanded symbolic terms.
- Renderer: the procedure that turns an invariant id into concrete notation, such as δ's, ϵ's, Young symmetrizers, gamma matrices, or structure constants.
- Signature convention: the metric and Clifford-algebra convention used when rendering or evaluating components. Complex, Euclidean, and Lorentzian signatures are conventions on the same abstract representation data.
- Tensor expansion: the object CFT code carries: rational-function ids times tensor invariant ids. It does not require expanded tensor formulas.

2 Symmetry

Interface

Types

`SimpleLieAlgebra (struct)`
SimpleLieAlgebra stores the simple family and rank.

`LieFamily (enum)`
LieFamily names the supported simple Lie families.

`QuantumNumbers = []const RepresentationId`
QuantumNumbers is the tensor-product list of representation ids.

`SymmetryId = u8`
SymmetryId names a symmetry in a symmetry store.

`RepresentationId = u8`
RepresentationId names a representation in a representation store.

`U1ChargeRational (struct)`
: U1ChargeRational stores a rational U(1) charge.

`TensorIndexId = u32`
: TensorIndexId names a tensor index in a tensor expression.

`TensorStructure (union)`
: TensorStructure stores simple built-in tensor structures.

A symmetry is labeled by its name and a Lie Algebra, which can be u(1) or a simple Lie algebra.

```
/// Symmetry labels a named symmetry algebra.
const Symmetry = struct {
    name: []const u8,
    algebra: LieAlgebra,
};

/// LieAlgebra distinguishes U(1) from simple Lie algebras.
const LieAlgebra = union(enum) {
    u1,
    simple: SimpleLieAlgebra,
};

/// SimpleLieAlgebra stores the simple family and rank.
pub const SimpleLieAlgebra = struct {
    family: LieFamily,
    rank: u8,
};

/// LieFamily names the supported simple Lie families.
pub const LieFamily = enum {
    a,
    b,
    c,
    d,
    e6,
    e7,
    e8,
    f4,
    g2,
};
```

A quantum number is labeled by the symmetry algebra and its representation.

```
/// QuantumNumbers is the tensor-product list of representation ids.
pub const QuantumNumbers = []const RepresentationId;

/// SymmetryId names a symmetry in a symmetry store.
pub const SymmetryId = u8;
/// RepresentationId names a representation in a representation store.
pub const RepresentationId = u8;

/// Representation stores a symmetry id and its label.
const Representation = struct {
    symmetry: SymmetryId,
    label: RepresentationLabel,
};

/// RepresentationLabel stores either a rational charge or Dynkin labels.
const RepresentationLabel = union(enum) {
    u1_charge: U1ChargeRational,
    dynkin: []const i8,
```

```
};

/// U1ChargeRational stores a rational U(1) charge.
pub const U1ChargeRational = struct {
    numerator: i8,
    denominator: i8,
};
```

An abstract tensor index carries a label and a particular representation.

```
/// TensorIndex stores a representation id and concrete index label.
const TensorIndex = struct {
    representation: RepresentationId,
    label: u32,
};

/// TensorIndexId names a tensor index in a tensor expression.
pub const TensorIndexId = u32;
```

An example of a concrete tensor structures - should only appear for SO-type groups, todo.

```
/// TensorStructure stores simple built-in tensor structures.
pub const TensorStructure = union(enum) {
    none,
    kronecker_delta: struct {
        left: TensorIndexId,
        right: TensorIndexId,
    },
};
```

3 Tensor Kernel

Tensor-code supplies compact handles for tensor structures that appear in CFT coefficients. The expensive work is generation and selection of invariant tensors, so those procedures should stream candidates rather than building a large expression tree inside the kernel.

An invariant source represents basis elements in

$$\mathrm{Hom}_{\mathfrak{g}}(T, \otimes_i R_i)$$

by handles. Rendering into a concrete notation is backend-dependent: A_n may render through Young symmetrizers and ϵ/δ tensors, D_n through metric, epsilon, and spinorial gamma data, and exceptional algebras through their available primitive invariant tensors.

```
const coupling = @import("coupling.zig");
const realization = @import("realization.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");
```

Interface

Types

`TensorExprId = u32`

TensorExprId names a tensor expression stored by tensor-code.

The basic ids are deliberately small. They name entries in stores owned by a preset, a tensor generator, or a client.

```
/// TensorExprId names a tensor expression stored by tensor-code.
pub const TensorExprId = u32;
```

```
/// TensorTermId names one term inside a tensor expression store.
const TensorTermId = u32;
/// BasisPathId names one compact coupling path in an invariant basis.
const BasisPathId = u32;
/// RenderedTermId names one streamed symbolic tensor term.
const RenderedTermId = u32;
/// ReductionTermId names one coefficient in a target invariant basis.
const ReductionTermId = u32;
/// InvariantFamilyId names a family of invariant tensor candidates.
const InvariantFamilyId = u16;
/// AlgebraSchemaId names a Lie or superalgebra schema.
const AlgebraSchemaId = u16;
/// GradingId names a grading attached to a tensor algebra schema.
const GradingId = u16;
```

Invariant generation should be pull-based. A caller can feed the source to an independent-basis selector without materializing every candidate first.

```
/// TensorExprSource streams tensor expression ids from a generator.
const TensorExprSource = struct {
    next_fn: *const fn (*anyopaque) ?TensorExprId,
    state: *anyopaque,

    /// next returns the next generated tensor expression id.
    pub fn next(self: *TensorExprSource) ?TensorExprId {
        return self.next_fn(self.state);
    }
};
```

Basis paths and rendered terms are streamed separately. This lets CFT code attach invariant ids to rational functions without asking tensor-code to expand concrete tensor expressions during ordinary simplification.

```
/// BasisPathSource streams invariant ids from a generated basis.
const BasisPathSource = struct {
    next_fn: *const fn (*anyopaque) ?coupling.InvariantHandle,
    state: *anyopaque,

    /// next returns the next invariant in the basis.
    pub fn next(self: *BasisPathSource) ?coupling.InvariantHandle {
        return self.next_fn(self.state);
    }
};

/// RenderedTermSource streams symbolic terms for one invariant rendering.
const RenderedTermSource = struct {
    next_fn: *const fn (*anyopaque) ?RenderedTermId,
    state: *anyopaque,

    /// next returns the next rendered symbolic term.
    pub fn next(self: *RenderedTermSource) ?RenderedTermId {
        return self.next_fn(self.state);
    }
};

/// ReductionTermSource streams coefficients in a chosen invariant basis.
const ReductionTermSource = struct {
    next_fn: *const fn (*anyopaque) ?ReductionTermId,
    state: *anyopaque,

    /// next returns the next reduction coefficient term.
    pub fn next(self: *ReductionTermSource) ?ReductionTermId {
        return self.next_fn(self.state);
```

```
    }
};
```

An invariant family says which algebra and external representations define the candidate space. The actual candidates are produced by a generator.

```
/// InvariantFamily declares the tensor-invariant problem to generate.
const InvariantFamily = struct {
    name: []const u8,
    algebra: AlgebraSchemaId,
    external_representations: []const symmetry.RepresentationId,
};
```

The new invariant request keeps external leg names and realizations. The old family type remains for compatibility with early kernel sketches.

```
/// InvariantRequest records a named public invariant-basis request.
const InvariantRequest = struct {
    name: []const u8,
    algebra: store.AlgebraHandle,
    external_legs: []const realization.ExternalLeg,
    target: coupling.TargetIrrep = .singlet,
    tree_policy: coupling.TreePolicy = .auto,
    basis_policy: coupling.BasisPolicy = .default,
};
```

The algebra schema stores only compact references to core tensors such as a metric or structure constants. This is enough for CFT coefficient code to refer to a symmetry algebra without owning its full tensor simplifier.

```
/// AlgebraSchema records the compact tensor data of a symmetry algebra.
const AlgebraSchema = struct {
    name: []const u8,
    symmetry: symmetry.SymmetryId,
    grading: ?GradingId = null,
    metric: ?TensorExprId = null,
    structure_constants: ?TensorExprId = null,
};
```

4 Clebsch-Gordan

This node is the handoff for implementing the tensor-code backend that will eventually generate Clebsch-Gordan data and invariant tensor bases. The first backend milestone is not explicit CG coefficients. It is a reliable, generic, cached decomposition engine:

$$R \otimes S = \bigoplus_T m_T T.$$

The CG generator will consume these decompositions to enumerate coupling paths to a target representation, usually the singlet $\mathbf{1}$.

4.1 Goals

The implementation must follow AGENTS.md:

- write simple explicit procedural Zig;
- design lean data structures first;
- avoid large intermediate symbolic expressions;

- keep data flows parallelizable;
- expose the bare minimum public API;
- keep the CFT-facing API in Tensor-Code;
- keep internal backend/decomposition/projector details out of the public root unless a caller actually needs them;
- document the implementation in org-roam nodes and keep prose concise.

The mathematical and product requirements are:

- support A_n for $SU(N)$;
- support B_n, D_n for $SO(N)$ and $Spin(N)$;
- support E_6, E_7, E_8, with E_8 a key target;
- do not overfocus the architecture on $SO(10)$, even though $Spin(10)$ is the main stress test;
- identify irreps by Dynkin labels, not dimensions;
- treat multiplicities as first-class channel copies;
- distinguish abstract irreps from concrete index realizations;
- support constrained realizations such as $144 \subset 10 \otimes 16$ without carrying the full ambient tensor product through every internal step;
- represent large invariant bases as compact coupling paths, not dense tensor components;
- render explicit gamma/projector formulas only on demand and by streaming terms to a sink;
- make every projector appearing in a returned result fully expandable.

The goals are concrete:

1. Abstract representation theory must be correct first. Irreps are Dynkin labels; decompositions have explicit multiplicities; channels are not identified by dimension.
2. Large invariant bases must be stored as compact coupling paths. A basis element is a recipe: external legs, intermediate channels, and local projector/CG kernel ids.
3. Projectors in returned results must not be opaque tokens. A projector id may be the compact storage form, but it must carry enough derivation data to expand into concrete symbolic terms later.
4. Expansion must be streamed. Fully expandable does not mean eagerly expanded. It means the renderer can enumerate the terms without inventing missing identities or asking the user for hidden Fierz relations.
5. The runnable target is not merely obtaining 73744 abstract basis expressions. The runnable target is also being able to expand each selected basis element through a caller-supplied filter, and for the resulting stream to be consumed at high throughput by CFT/correlator code.

6. Filters are part of the execution model. They may discard terms by index support, operator kind, realization leg, component assignment, symmetry, coefficient class, or target downstream contraction. The renderer must apply filters while streaming, not after materializing the full expansion. Partial filtering must be sound: a partial product may be rejected only when no completion can survive. Otherwise the filter must return `undecided` and the expansion continues.

7. Coupling-tree policy is part of the basis convention. Build one canonical tree first, and certify independence for that tree. Multiple trees are different bases unless a basis-change or reduction map is explicitly constructed.

8. Fierz and projector identities must be derived from the backend construction: channel decomposition, idempotent splitting, highest-weight solvers, Casimir separation where valid, and explicit residual linear algebra only when necessary. They are not user-supplied tables.

9. Compute estimates must include projector expansion cost. Counting coupling paths alone is insufficient if each path contains a projector whose expansion is large.

The motivating stress target remains:

$$\dim \mathrm{Hom}_{Spin(10)}(\mathbf{1}, 16^{12}) = 73744.$$

Those 73744 objects should be records describing coupling paths and local projector ids. They should not be expanded into component tensors during basis enumeration. However, every projector id in those paths must be traceable to a stored expansion procedure. When the caller asks for concrete notation, rendering should stream terms built from metrics, epsilons, named operators, gamma-like backend operators, or projector blocks. The stream must be filterable as it is produced. A successful implementation is one where a downstream consumer can iterate through selected expanded terms without waiting for the full expansion of all 73744 invariants to be allocated.

4.2 Non-Solutions

The following are explicitly not acceptable:

- Returning only abstract representation-theory counts. Counts are useful for feasibility, but CFT code needs actual invariant handles and eventually streamable tensor expressions.
- Returning opaque named projectors with no derivation recipe. A compact projector id is acceptable only if the backend can expand it.
- Returning compact coupling paths with no efficient expansion path. A list of 73744 abstract expressions is not enough.
- Expanding all 73744 basis elements into memory before filtering. Filters must be pushed into the streaming expansion loop.
- Treating rendering as a slow debug printer. Rendering is part of the runnable computational path and must feed downstream correlator code at a fast pace.
- Treating $144 \subset 10 \otimes 16$ as $10 \otimes 16$ throughout all internal coupling steps and projecting only at the very end. The realization projector belongs at the boundary.
- Relying on hand-entered Fierz identities. Identities must follow from the algebraic construction and verified projector/idempotent relations.

- Building dense tensors or dense component spaces such as 16^{12}.
- Designing an $SO(10)$-only system. $Spin(10)$ is a stress test, not the architecture.
- Making gamma matrices part of the generic public ADE API. Gamma-like objects are backend-rendered named operators or internal SO/Spin atoms.
- Optimizing by hiding mathematical ambiguity. Multiplicity copies, realization channels, basis conventions, and signature/rendering conventions must be explicit.
- Rejecting a partial expanded term because the current prefix does not yet match a filter. Prefix filters must distinguish `reject` from `undecided`.
- Concatenating paths from several coupling trees and calling the union a larger independent basis. This is allowed only after constructing and applying a basis-change or reduction map.

4.3 Weasel Modes And Required Evidence

An implementation is not accepted because it can print plausible data. It must produce the required records and expose enough evidence that the records are real. The following failure modes are common and must be rejected during review:

- Claiming a "fast structure generator" that only returns the count of invariants. Required evidence: stored basis handles with inspectable path records, one record per basis element.
- Returning a list of abstract labels without local channel data. Required evidence: each path stores the coupling tree, intermediate channel handles, multiplicity-copy ids, and local projector/CG kernel ids.
- Returning projectors as names such as `P_144` with no expansion route. Required evidence: every projector id reports its derivation kind and an expansion source, or returns a typed `NotExpandable` error before it appears in a returned basis.
- Using an overcomplete gamma-monomial list and calling it a basis. Required evidence: basis construction happens in representation/multiplicity space before rendering; gamma terms are a rendering of already-independent basis elements.
- Running a dense component-space linear solve and hiding the cost behind a small output. Required evidence: logs/counters for maximum allocated weight multiset size, path count, projector count, and no allocations proportional to 16^{12} or another full tensor component space.
- Filtering after materializing all expanded terms. Required evidence: filters are called during expansion, and a reject-all filter allocates no large output buffer.
- Failing to distinguish multiplicity copies. Required evidence: decomposition terms include multiplicity, and local kernels/projectors carry the selected copy.
- Treating projected realizations as ambient products until the final output. Required evidence: external legs using constrained realizations carry a boundary projector once, and internal coupling uses the target irrep.
- Claiming independence from successful numerical spot checks only. Required evidence: independence follows from the decomposition path construction, and numerical/rendered checks are secondary audits.

- Hardcoding known decompositions for the acceptance tests. Required evidence: tests include at least one generated decomposition not present as a literal special case, and the backend exposes counters showing which generic procedure produced it.

Every major stage must produce a compact audit record:

- algebra and convention ids;
- input irreps;
- number of weights generated;
- total weight multiplicity;
- product multiset size;
- decomposition channel count;
- sum of channel dimensions times multiplicities;
- number of basis paths;
- number of distinct projector ids;
- maximum known projector expansion size, if available;
- filter accept/reject/undecided counts during streamed expansion.

These audits are mandatory internal milestone evidence, not optional debug output. They are not user-facing API by default, but each backend milestone must produce enough audit data to prove that it is doing the promised computation.

4.4 Independence By Construction

The basis must be independent before rendering. The construction should be:

1. For each binary product $R \otimes S$, decompose into irreps:

$$R \otimes S = \bigoplus_T M_{RS}^T \otimes T.$$

   Here M_{RS}^T is the finite multiplicity space. If $m_T = \dim M_{RS}^T$, choose a basis vector e_i for each copy $i = 0, \ldots, m_T - 1$.

2. A local CG kernel is an intertwiner selecting one basis vector in M_{RS}^T. Distinct multiplicity-copy ids are distinct basis vectors in that multiplicity space.

3. For a fixed canonical coupling tree, a global path is a product of local choices: at every internal node, choose the output channel and multiplicity-copy basis vector.

4. The set of all paths ending in the requested target T, usually $\mathbf{1}$, is a basis of the multiplicity space

$$\mathrm{Hom}_{\mathfrak{g}}\left(T,\ R_1 \otimes \cdots \otimes R_n\right)$$

   for that tree and basis convention.

Independence follows for that fixed tree because the decomposition is a direct sum of irreducible channels and explicit multiplicity spaces. We are not selecting linearly independent gamma expressions from an overcomplete list. We are selecting basis vectors in multiplicity spaces and composing intertwiners along one tree convention.

Different tree policies give different coordinate systems on the same multiplicity space. They must not be concatenated. If the implementation later supports several trees, each tree must separately produce the target multiplicity. The combined count across trees is not an independence certificate; it is redundant data until a basis-change or reduction map is available.

Rendered formulas may look dependent because gamma matrices, epsilons, metrics, and projector blocks satisfy identities. Those identities are part of the rendering backend. They must not be used to define the abstract basis. Rendering must be a faithful expansion of the already-selected path basis in a chosen convention. If a renderer cannot prove or verify faithfulness for a convention, it must report a typed error or mark the expansion as an audit target; it must not silently collapse or add basis elements.

Required independence checks:

- The number of emitted basis paths equals the target multiplicity computed by dynamic programming over decompositions.
- For every binary decomposition, the sum

$$\sum_T m_T \dim T$$

  equals $\dim R \dim S$.
- Local projector records for a product channel must satisfy the abstract projector laws in their construction record: idempotency and channel separation. When explicit formulas exist, audits should verify $P_i P_j = \delta_{ij} P_i$ in the chosen representation of the local product.
- Boundary realization projectors must satisfy the same rule: e.g. $144 \subset 10 \otimes 16$ is represented by an idempotent whose image has dimension 144, and the complementary channel is accounted for.
- Rendered term streams are audits of the basis, not the source of the basis. Component evaluation or filtered numerical checks can catch renderer bugs but cannot replace the multiplicity-space construction.

4.5 Implementation Instructions

Proceed in this order. Do not skip ahead to rendered gamma formulas or high-level invariant counts.

1. Implement the backend registry privately in `Context`. It must select a backend by algebra family and return typed unsupported-capability errors.
2. Implement flat weight storage and a weight-system cache. Store signed weight coordinates in one buffer and weight records as offsets. Add audit counters for weight count and total multiplicity.
3. Implement the generic highest-weight weight-system generator. Validate it on known dimensions before any tensor-product code is accepted.
4. Implement tensor-product character multiplication and subtraction. The result must be flat product terms with multiplicities. Store the result in Tensor Decomposition.

5. Add decomposition acceptance tests: $SU(2) : 2 \otimes 2 = 1 \oplus 3$, $Spin(10) : 10 \otimes 16 = 16 \oplus 144$, $Spin(10) : 16 \otimes 16 = 10 \oplus 120 \oplus 126$, and $E_8 : 248 \otimes 248$ with the known channels.

6. Implement dynamic-programming path counting to a target using cached binary decompositions. This is still counting, but it must use the same channel records that path enumeration will use.

7. Implement path enumeration for one canonical coupling tree. Each emitted path must store its local channel choices and multiplicity-copy ids. The path count must equal the dynamic count. Do not enumerate several trees until basis-change or reduction maps exist.

8. Implement projector identity creation. Each projector id must record its role, convention, derivation kind, and expansion obligation.

9. Implement a minimal expansion interface that can walk one path and call a sink with streamed symbolic terms. At first it may return typed `NotImplemented` for missing local formulas, but it must not return opaque successful placeholders.

10. Implement filters inside the expansion loop. The filter decision is tri-state: `accept`, `reject`, or `undecided`. A partial term can be rejected only when rejection is final under all completions. Prove with a reject-all filter test that no large output buffer is allocated.

11. Only after the generic path and projector infrastructure is correct, add specialized SO/Spin gamma/projector renderers, SU tableau/Gelfand-Tsetlin optimizations, or E-type special renderers.

At every step, prefer the smallest working data structure. Add a new abstraction only when it removes real duplication or separates a hot data path. Do not add public exports for backend internals unless CFT code has a concrete call site that needs them.

4.6 Current State

The public API is rooted at Tensor-Code. Public callers create a context, register an algebra, register irreps, optionally register concrete realizations, and request an invariant basis:

```
var ctx = try tensor.Context.init(allocator);
defer ctx.deinit();

const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
const leg = tensor.ExternalLeg.primitive(spinor, &.{
    .init(.spinor, "a"),
});
const basis = try ctx.invariantBasis(.{
    .algebra = so10,
    .external_legs = &.{leg},
});
```

Conventions are configurable at context construction:

```
var ctx = try tensor.Context.initWithOptions(allocator, .{
    .algebra_conventions = .{
        .quadratic_casimir_scale = .{ .numerator = 1, .denominator = 1 },
    },
    .default_signature = .complex,
});
defer ctx.deinit();
```

The current substrate is:

- Tensor Root Data validates supported simple algebras, stores Cartan data, computes positive roots, dual Dynkin labels, Weyl dimensions, and quadratic Casimirs.
- Tensor Representation Store interns algebras and irreps. It computes metadata from Dynkin labels and provides explicit `dualIrrep` lookup without recursively closing registration under duals.
- Tensor Realization records concrete external index shapes. Projected realizations store a selected channel, so a constrained leg such as $144 \subset 10 \otimes 16$ is a first-class boundary realization.
- Tensor Decomposition owns the binary product decomposition cache. It stores flat product terms and has a hash index keyed by the pair of irrep handles.
- Tensor Projector owns projector identities: product-channel projectors, boundary-realization projectors, and basis-change projectors.
- Tensor Backend defines the internal backend capability contract.
- Tensor Rendering defines streamed symbolic terms. `SymbolicTerm` carries borrowed slices valid only until the sink returns. The public root does not expose a gamma-specific type; SO/Spin renderers can emit backend operators through generic symbolic atoms.

Build-checked metadata cases currently include:

- B_4: vector 9, spinor 16;
- D_5: vector 10, spinor 16, vector-spinor 144;
- E_6: fundamental 27;
- E_7: fundamental 56;
- E_8: adjoint 248.

The following are still intentionally not implemented:

- binary decomposition algorithms;
- weight-system generation;
- channel-path enumeration;
- local CG/projector construction;
- explicit gamma or projector expansion execution;
- expansion filters and downstream stream consumers;
- `Context.renderInvariant`;
- `Context.evaluateInvariant`.

4.7 Backend Goal

The first backend should be a generic highest-weight backend. It should be a correctness backend and cache builder, not yet a family-specific optimized renderer. Its first job is to answer:

1. is this algebra supported?
2. what is the weight system of this irrep?
3. what is the decomposition of $R \otimes S$?
4. what stable projector identity corresponds to one product channel?

The backend must not allocate dense tensor-product component spaces. It may construct compact weight multisets and flat cache records.

4.8 Backend Build Order

4.8.1 1. Backend registry

Add an internal backend registry owned by the private `Context` implementation. It maps `LieFamily` to a backend record and selects a backend from an algebra family. It should not be exported from Tensor-Code.

Start with one backend implementation:

- file: `generic-highest-weight-backend.zig`;
- supports $A_n, B_n, D_n, E_6, E_7, E_8$;
- returns typed missing-capability errors for unimplemented operations.

The registry should make no CFT-facing API changes. CFT code still asks the context for invariant bases.

4.8.2 2. Root-data helpers

Use Tensor Root Data as the source of algebra conventions. Do not duplicate Cartan matrices or exceptional diagrams in the backend.

The backend will need internal helpers for:

- rank;
- Cartan entries;
- simple-root reflection on weights;
- dominant-weight test;
- highest-weight validation;
- exact rational inner products where Freudenthal or Casimir formulas need them.

Keep these helper APIs internal unless a later caller clearly needs them.

4.8.3 3. Weight storage

Weights are not public API. Store them in flat buffers:

```
const WeightRef = struct {
    dynkin_offset: u32,
};

const WeightMultiplicity = struct {
    weight: WeightRef,
    multiplicity: u32,
};
```

The actual Dynkin-coordinate arrays should live in one flat signed buffer, probably `i32` once tensor products become large enough. Signed coordinates are necessary because intermediate weights and reflected weights need not be dominant.

Store rank once on the weight-system span or get it from the algebra. Do not store rank in every weight record. Do not allocate one heap object per weight.

4.8.4 4. Weight-system generator

Implement a cached weight-system generator for one highest-weight irrep. Freudenthal multiplicity recursion is the generic first choice because it works uniformly for the supported simple families.

Cache key:

```
const WeightSystemKey = struct {
    algebra: store.AlgebraHandle,
    irrep: store.IrrepHandle,
};
```

Cache value:

```
const WeightSystem = struct {
    weights_offset: u32,
    weights_len: u32,
    rank: u8,
    dynkin_coords_offset: u32,
    dynkin_coords_len: u32,
};
```

Acceptance checks:

- D_5 spinor has total weight multiplicity 16;
- D_5 vector has total weight multiplicity 10;
- B_4 spinor has total weight multiplicity 16;
- E_8 adjoint has total weight multiplicity 248.

4.8.5 5. Tensor-product decomposition

Use character subtraction first. It is generic and easier to audit than a family-specific closed formula.

Algorithm:

1. get cached weight systems for R and S;
2. add all pairs of weights to build the weight multiset for $R \otimes S$;
3. find a maximal dominant weight still present, using one deterministic dominance-order rule;
4. its remaining multiplicity is the multiplicity of the next channel;

5. subtract that many copies of the cached character of that highest weight;
6. repeat until the multiset is empty.

The dominance selection rule must be explicit and stable. A practical first rule is:

1. scan only weights with positive residual multiplicity;
2. discard non-dominant weights;
3. choose a dominant weight not strictly below another remaining dominant weight in the root partial order;
4. break ties by lexicographic Dynkin labels.

After every subtraction, no residual multiplicity may become negative. A negative residual is a backend bug, not a recoverable ambiguity. The selected channel multiplicity is the residual multiplicity of the selected highest weight before subtraction.

The audit invariant after each subtraction is:

$$\chi_{\text{remaining}} + \sum_i m_i \chi_{T_i} = \chi_R \chi_S.$$

The final residual character must be zero. This catches lost multiplicities, wrong dominant-weight selection, and incorrect subtraction signs.

This is not the final fastest algorithm. It is the first trusted backend. It will become the oracle against which specialized $SO/Spin$, SU, and exceptional backends are checked.

Acceptance checks:

- $SU(2) : 2 \otimes 2 = 1 \oplus 3$;
- $Spin(10) : 10 \otimes 16 = 16 \oplus 144$;
- $Spin(10) : 16 \otimes 16 = 10 \oplus 120 \oplus 126$;
- $E_8 : 248 \otimes 248$ matches the known decomposition, including multiplicities and dimensions.

4.8.6 6. Decomposition cache integration

The backend writes decompositions into Tensor Decomposition. The cache stores:

- product key R, S;
- flat product terms (T, m_T);
- stable handle for repeated requests.

The future internal context method should look like:

```
fn decomposeProduct(ctx: *Context, left: store.IrrepHandle, right: store.IrrepHandle)
↪  !decomposition.ProductDecompositionHandle
```

Do not expose this method publicly unless CFT code needs it.

4.8.7 7. Projector identities and expansion obligations

For each product term, the backend may create a product-channel projector identity in Tensor Projector. The first decomposition backend does not need to expand projector formulas immediately, but a successful basis result must not contain projector ids that have no expansion route. Internal blocked records are allowed only if they fail before being returned as successful invariant data. Each projector record needs enough information to recover an expansion path.

The projector identity records:

- left irrep;
- right irrep;
- target channel;
- convention id;
- derivation kind;
- expansion source, such as Casimir polynomial, idempotent split, highest-weight solver output, or backend-specific verified construction.

Do not store expandability as a boolean. The backend needs enough state to distinguish several useful stages:

```
const ExpansionStatus = enum {
    not_requested,
    expandable_named,
    expandable_blocks,
    expandable_terms,
    blocked_missing_formula,
    blocked_unverified_identity,
};
```

`expandable_named` is only a rendering mode. It is not proof that the projector satisfies the fully-expandable requirement. A projector appearing in a successful invariant basis must have a derivation route that can reach `expandable_terms` in the requested convention, even when the caller chooses to render it as a compact name.

Acceptance checks:

- repeated requests return the same projector id;
- multiplicity copies are explicit;
- no symbolic projector expansion is allocated during decomposition;
- every returned projector reports an `ExpansionStatus` in the requested convention;
- blocked projector statuses are typed errors before successful basis return, not silently named placeholders;
- a basis result rendered in named mode still audits that each local projector has a term-expansion route.

4.8.8 8. Backend capability implementation

Update Tensor Backend only as needed to support the generic backend. The expected internal procedures are:

- `validateAlgebra`;
- `irrepMetadata`, if a backend-specific metadata path is needed;
- `weightSystem`;
- `decomposeProduct`;
- `localProjectorId`.

Rendering remains `NotImplemented`.

4.8.9 9. Expansion stream and filters

Before explicit projector formulas are implemented, define the execution boundary they must satisfy. The expansion API should be a streaming pipeline:

```
fn renderInvariantExpanded(
    ctx: *Context,
    basis: coupling.BasisHandle,
    invariant: coupling.InvariantHandle,
    options: rendering.RenderOptions,
    filter: ExpansionFilter,
    sink: anytype,
) !void
```

The exact public shape may differ, but the semantics should not:

```
const FilterDecision = enum {
    accept,
    reject,
    undecided,
};
```

- the renderer walks the compact coupling path;
- local projector expansions are pulled lazily;
- each local expansion term is combined into the current partial term;
- the filter is applied as early as possible to borrowed scratch terms;
- accepted terms are emitted immediately to the sink;
- rejected terms are not stored;
- undecided partial terms may continue, but the implementation must track their frontier size;
- repeated local projector expansions are cached by projector id and convention when doing so reduces work.

Partial terms can be rejected only when the predicate is final and sound under all later factors. Otherwise the filter must return `undecided`. This is part of correctness, not just an optimization detail.

Filter callbacks and sinks must not retain borrowed slices from streamed terms. If a caller needs to keep a term, it must copy it into caller-owned storage. This keeps the renderer free to reuse scratch buffers and prevents a hidden materialized expansion from growing behind the streaming interface.

The filter is not a post-processing pass over a giant vector. It is an execution hook that keeps the streamed expansion small enough to be useful.

Possible filter predicates:

- term contains or avoids a given external index block;
- term uses only selected symbolic operator kinds;
- term is compatible with a component assignment;
- term can contribute to a downstream contraction;
- term has a coefficient/operator class accepted by the target CFT routine.

Acceptance checks:

- expanding one invariant can stream terms without allocating the full expansion;
- expanding many invariants can reuse local projector expansions;
- a filter that rejects all terms performs no large output allocation;
- a filter that accepts a sparse subset scales with the accepted frontier plus unavoidable local projector traversal, not with a materialized full basis.

4.9 Compute Model

The first backend has four different compute scales:

1. Decomposition compute: weight-system generation, tensor-product weight multisets, and character subtraction.
2. Path enumeration compute: dynamic programming over cached binary decompositions to enumerate coupling paths to a target.
3. Expansion compute: streaming the local projector/CG expansions appearing in selected paths.
4. Filter compute: deciding early which partial or complete expansion terms can be discarded before they reach downstream CFT code.

The first milestone implements the first scale and prepares records for the third and fourth. It must not pretend expansion is free. A 73744-path result can be small as stored paths while still being expensive to render if each path contains large expandable projectors. Therefore every feasibility estimate must track:

- number of basis paths;
- number of distinct local projector ids used by those paths;

- maximum and total expected expansion terms per projector when known;
- whether expansion can be shared across repeated projector ids;
- whether expansion is requested as named, block-expanded, or term-expanded;
- expected filter rejection rate;
- whether filters can be applied to partial products or only complete terms;
- number of undecided partial terms retained at each expansion layer;
- sink throughput and backpressure behavior.

4.10 Data-Oriented Constraints

The expensive objects are weight systems, tensor-product weight multisets, path records, and projector expansions. Therefore:

- cache weight systems by irrep handle;
- store weights in flat buffers;
- sort or hash compact weight keys, not heap-allocated slices;
- store product decompositions as flat terms;
- keep projector ids as small records;
- cache projector expansions by projector id and convention when expansion is requested repeatedly;
- push filters into expansion loops;
- emit accepted terms immediately;
- stream rendered terms into sinks;
- never build dense component tensors.

The 16^{12} target should produce coupling paths, not components. A single dense tensor in 16^{12} has 16^{12} components, while the desired answer has 73744 basis paths. The algorithm must scale with the number of channels and paths, not the full component space.

4.11 Explicit Non-Goals For First Backend

Do not implement the following in the first backend:

- gamma-matrix rendering;
- bulk explicit projector expansion execution;
- explicit CG coefficient tables;
- higher-Casimir projector splitting;
- sparse nullspace solvers;
- parallel execution;

- $SU(N)$ Gelfand-Tsetlin or tableau optimized CGs;
- $SO/Spin$ Clifford-specialized renderers;
- E_8 adjoint-specialized renderers.

Those belong after binary decomposition is correct, cached, and tested. The data model must still be designed so these operations can be added without changing the meaning of existing projector ids.

4.12 Definition Of Done

The backend handoff is complete when:

- `zig build --global-cache-dir /tmp/zig-global-cache` passes;
- all acceptance decompositions above pass;
- weight systems are cached and flat;
- product decompositions are cached by key;
- no dense tensor component arrays are allocated;
- `Context.invariantBasis` can internally request product decompositions;
- projector records created by the backend have explicit expansion obligations;
- expanded invariants can be streamed through a filter without materializing their full term list;
- repeated projector expansions can be cached and reused;
- a downstream sink can consume accepted terms incrementally;
- render/evaluate remain explicitly unimplemented until projector formulas and term streaming are ready.

5 Tensor Context

The context module is the procedural boundary of tensor-code. Public callers get an opaque state pointer and methods; the stores remain in a private implementation record so callers cannot bypass validation.

```
const std = @import("std");
const coupling = @import("coupling.zig");
const decomposition = @import("decomposition.zig");
const projector = @import("projector.zig");
const realization = @import("realization.zig");
const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");

/// ContextId names an initialized tensor-code context or preset.
pub const ContextId = struct {
    value: u32,

    /// init constructs a context id from a store index.
    pub fn init(value: u32) ContextId {
        return .{ .value = value };
```

```
    }
};

/// ContextOptions configures algebra and rendering defaults.
pub const ContextOptions = struct {
    algebra_conventions: root_data.AlgebraConventions = .{},
    default_signature: rendering.SignatureConvention = .complex,
};
```

Registration validates cross-store handles before interning. Metadata and duals are queried through methods, not by reaching into stores.

```
/// Context is an opaque handle to tensor-code stores and caches.
pub const Context = struct {
    state: *anyopaque,
    allocator: std.mem.Allocator,

    /// init constructs an empty tensor-code context.
    pub fn init(allocator: std.mem.Allocator) !Context {
        return Context.initWithOptions(allocator, .{});
    }

    /// initWithOptions constructs a context with explicit conventions.
    pub fn initWithOptions(allocator: std.mem.Allocator, options: ContextOptions) !Context {
        const state = try allocator.create(Impl);
        errdefer allocator.destroy(state);
        state.* = Impl.init(allocator, options);
        return .{
            .state = state,
            .allocator = allocator,
        };
    }

    /// deinit releases memory owned by this context.
    pub fn deinit(self: *Context) void {
        const state = self.impl();
        state.deinit();
        self.allocator.destroy(state);
        self.* = undefined;
    }

    /// registerAlgebra interns a Lie algebra and returns its handle.
    pub fn registerAlgebra(self: *Context, spec: store.AlgebraSpec) !store.AlgebraHandle {
        return self.impl().representations.internAlgebra(spec);
    }

    /// registerIrrep interns an abstract irreducible representation.
    pub fn registerIrrep(self: *Context, algebra: store.AlgebraHandle, spec: store.IrrepSpec)
    ↪   !store.IrrepHandle {
        return self.impl().representations.internIrrep(algebra, spec);
    }

    /// dualIrrep returns the registered dual representation.
    pub fn dualIrrep(self: *Context, irrep: store.IrrepHandle) !store.IrrepHandle {
        return self.impl().representations.dualIrrep(irrep);
    }

    /// irrepMetadata returns derived metadata for a registered irrep.
    pub fn irrepMetadata(self: Context, irrep: store.IrrepHandle) ?store.IrrepMetadata {
        return self.impl().representations.irrepMetadata(irrep);
    }

    /// registerRealization interns a concrete index realization of an irrep.
```

```
    pub fn registerRealization(self: *Context, spec: realization.RealizationSpec)
    ↪  !realization.RealizationHandle {
        try self.validateRealizationSpec(spec);
        return self.impl().realizations.internRealization(spec);
    }

    /// invariantBasis generates or retrieves a compact invariant basis.
    pub fn invariantBasis(self: *Context, request: coupling.InvariantBasisRequest)
    ↪  !coupling.BasisHandle {
        try self.validateInvariantBasisRequest(request);
        return self.impl().couplings.internBasisRequest(request);
    }

    /// renderInvariant streams one invariant in the requested convention.
    pub fn renderInvariant(self: *Context, basis: coupling.BasisHandle, invariant:
    ↪  coupling.InvariantHandle, options: rendering.RenderOptions, sink: anytype) !void {
        _ = self;
        _ = basis;
        _ = invariant;
        _ = options;
        _ = sink;
        return error.NotImplemented;
    }

    /// evaluateInvariant streams component-evaluation data for one invariant.
    pub fn evaluateInvariant(self: *Context, invariant: coupling.InvariantHandle, options:
    ↪  rendering.EvalOptions, sink: anytype) !void {
        _ = self;
        _ = invariant;
        _ = options;
        _ = sink;
        return error.NotImplemented;
    }

    fn validateRealizationSpec(self: Context, spec: realization.RealizationSpec) !void {
        const target_algebra = self.impl().representations.irrepAlgebra(spec.channel.irrep)
        ↪  orelse return error.UnknownIrrep;
        for (spec.ambient) |ambient| {
            const ambient_algebra = self.impl().representations.irrepAlgebra(ambient) orelse
            ↪  return error.UnknownIrrep;
            if (ambient_algebra.value != target_algebra.value) return
            ↪  error.MixedRealizationAlgebras;
        }
    }

    fn validateInvariantBasisRequest(self: Context, request: coupling.InvariantBasisRequest)
    ↪  !void {
        if (!self.impl().representations.containsAlgebra(request.algebra)) return
        ↪  error.UnknownAlgebra;
        for (request.external_legs) |leg| {
            const leg_algebra = try self.externalLegAlgebra(leg);
            if (leg_algebra) |algebra| {
                if (algebra.value != request.algebra.value) return
                ↪  error.MixedInvariantAlgebras;
            }
        }
    }

    fn externalLegAlgebra(self: Context, leg: realization.ExternalLeg) !?store.AlgebraHandle {
        return switch (leg.realization) {
            .primitive_irrep => |irrep| self.impl().representations.irrepAlgebra(irrep) orelse
            ↪  return error.UnknownIrrep,
            .handle => |handle| {
```

```
                const irrep = self.impl().realizations.targetIrrep(handle) orelse return
                ↪  error.UnknownRealization;
                return self.impl().representations.irrepAlgebra(irrep) orelse return
                ↪  error.UnknownIrrep;
            },
            .registered_name => null,
        };
    }

    fn impl(self: Context) *Impl {
        return @ptrCast(@alignCast(self.state));
    }
};
```

The private implementation record owns the flat stores. It is not imported by the root API.

```
const Impl = struct {
    id: ContextId,
    allocator: std.mem.Allocator,
    options: ContextOptions,
    representations: store.Store,
    realizations: realization.Store,
    decompositions: decomposition.Store,
    projectors: projector.Store,
    couplings: coupling.Store,

    fn init(allocator: std.mem.Allocator, options: ContextOptions) Impl {
        return .{
            .id = ContextId.init(0),
            .allocator = allocator,
            .options = options,
            .representations = store.Store.init(allocator, options.algebra_conventions),
            .realizations = realization.Store.init(allocator),
            .decompositions = decomposition.Store.init(allocator),
            .projectors = projector.Store.init(allocator),
            .couplings = coupling.Store.init(allocator),
        };
    }

    fn deinit(self: *Impl) void {
        self.couplings.deinit();
        self.projectors.deinit();
        self.decompositions.deinit();
        self.realizations.deinit();
        self.representations.deinit();
        self.* = Impl.init(self.allocator, self.options);
    }
};
```

6 Tensor Backend

Backends are internal family-specific providers. The public workflow should not depend on an ADE, gamma-matrix, Young-symmetrizer, or exceptional-algebra implementation detail. A backend validates support, supplies representation metadata, decomposes binary products, constructs local projector ids, and streams concrete symbolic terms when rendering is requested.

```
const decomposition = @import("decomposition.zig");
const projector = @import("projector.zig");
const rendering = @import("rendering.zig");
const root_data = @import("root-data.zig");
const store = @import("representation-store.zig");
const symmetry = @import("symmetry.zig");
```

```
/// BackendId names one registered Lie-family backend.
pub const BackendId = u16;

/// PrimitiveInvariantSpan names backend-owned primitive invariant descriptors.
pub const PrimitiveInvariantSpan = struct {
    offset: u32,
    len: u32,
};

/// RenderConvention records the concrete rendering convention passed to a backend.
pub const RenderConvention = struct {
    options: rendering.RenderOptions,
};

/// Capabilities is the internal contract every Lie-family backend must satisfy.
pub const Capabilities = struct {
    validate_algebra: *const fn (*anyopaque, symmetry.SimpleLieAlgebra) anyerror!void,
    irrep_metadata: *const fn (*anyopaque, store.AlgebraHandle, store.IrrepHandle)
    ↪  anyerror!store.IrrepMetadata,
    decompose_product: *const fn (*anyopaque, decomposition.ProductKey, *anyopaque)
    ↪  anyerror!void,
    local_projector: *const fn (*anyopaque, projector.ProjectorKey)
    ↪  anyerror!projector.ProjectorId,
    primitive_invariants: *const fn (*anyopaque, []const store.IrrepHandle)
    ↪  anyerror!PrimitiveInvariantSpan,
    render_projector: *const fn (*anyopaque, projector.ProjectorId, RenderConvention,
    ↪  *anyopaque) anyerror!void,
};

/// Record stores one backend state pointer and capability table.
pub const Record = struct {
    id: BackendId,
    state: *anyopaque,
    family: symmetry.LieFamily,
    root_conventions: root_data.AlgebraConventions,
    capabilities: Capabilities,
};
```

7 Tensor Representation Store

This module owns the abstract representation data. A Lie algebra and an irrep are interned once, then every downstream system carries small typed handles. That keeps tensor generation independent of how a representation is eventually rendered as indices, matrices, or symbolic projectors.

```
const std = @import("std");
const root_data = @import("root-data.zig");
const symmetry = @import("symmetry.zig");
```

Handles are public because other modules need stable references. The records behind them stay private.

```
/// AlgebraHandle names a registered Lie algebra in a tensor context.
pub const AlgebraHandle = struct {
    value: u32,

    /// init constructs an algebra handle from a store index.
    pub fn init(value: u32) AlgebraHandle {
        return .{ .value = value };
    }
};
```

```
/// IrrepHandle names an abstract irreducible representation.
pub const IrrepHandle = struct {
    value: u32,

    /// init constructs an irrep handle from a store index.
    pub fn init(value: u32) IrrepHandle {
        return .{ .value = value };
    }
};

/// ChannelHandle names one irrep copy inside a local product.
pub const ChannelHandle = struct {
    value: u32,

    /// init constructs a channel handle from a store index.
    pub fn init(value: u32) ChannelHandle {
        return .{ .value = value };
    }
};
```

Specs are caller-owned construction data. Interning clones only the slices we must retain.

```
/// AlgebraSpec describes a requested algebra before it is interned.
pub const AlgebraSpec = union(enum) {
    u1,
    simple: symmetry.SimpleLieAlgebra,
};

/// IrrepSpec describes a requested representation before it is interned.
pub const IrrepSpec = union(enum) {
    u1_charge: symmetry.U1ChargeRational,
    dynkin: []const i16,
};

/// IrrepMetadata stores derived representation data.
pub const IrrepMetadata = struct {
    dimension: ?u128 = null,
    dual: ?IrrepHandle = null,
    quadratic_casimir: ?root_data.Rational = null,
};
```

The store is intentionally simple: linear lookup first, then append. The first implementation has few registered algebras and irreps compared with the later coupling search, so this avoids prematurely adding hash table machinery. If profiles show this store in the hot path, the record layout can stay fixed while we add an index.

```
/// Store owns interned algebra and irrep records.
pub const Store = struct {
    allocator: std.mem.Allocator,
    conventions: root_data.AlgebraConventions,
    algebras: std.ArrayList(AlgebraRecord) = .empty,
    irreps: std.ArrayList(IrrepRecord) = .empty,

    /// init constructs an empty representation store.
    pub fn init(allocator: std.mem.Allocator, conventions: root_data.AlgebraConventions) Store
    ↪ {
        return .{ .allocator = allocator, .conventions = conventions };
    }

    /// deinit releases all representation-store memory.
    pub fn deinit(self: *Store) void {
        for (self.irreps.items) |record| {
            switch (record.label) {
```

```
                .dynkin => |label| self.allocator.free(label),
                .u1_charge => {},
            }
        }
        self.irreps.deinit(self.allocator);
        self.algebras.deinit(self.allocator);
        self.* = Store.init(self.allocator, self.conventions);
    }

    /// internAlgebra returns the stable handle for an algebra spec.
    pub fn internAlgebra(self: *Store, spec: AlgebraSpec) !AlgebraHandle {
        try validateAlgebraSpec(spec);
        for (self.algebras.items, 0..) |record, index| {
            if (algebraSpecEql(record.spec, spec)) {
                return AlgebraHandle.init(@intCast(index));
            }
        }

        try self.algebras.append(self.allocator, .{
            .spec = spec,
            .root_data = try makeRootDatum(spec, self.conventions),
        });
        return AlgebraHandle.init(@intCast(self.algebras.items.len - 1));
    }

    /// containsAlgebra returns whether an algebra handle belongs to this store.
    pub fn containsAlgebra(self: Store, handle: AlgebraHandle) bool {
        return self.algebraRecord(handle) != null;
    }

    /// internIrrep returns the stable handle for an irrep spec in an algebra.
    pub fn internIrrep(self: *Store, algebra: AlgebraHandle, spec: IrrepSpec)
    ↪   anyerror!IrrepHandle {
        const algebra_record = self.algebraRecord(algebra) orelse return error.UnknownAlgebra;
        try validateIrrepSpec(algebra_record.spec, spec);

        for (self.irreps.items, 0..) |record, index| {
            if (handleEql(record.algebra, algebra) and irrepLabelSpecEql(record.label, spec))
            ↪   {
                return IrrepHandle.init(@intCast(index));
            }
        }

        const label = try cloneIrrepSpec(self.allocator, spec);
        errdefer freeIrrepLabel(self.allocator, label);

        try self.irreps.append(self.allocator, .{
            .algebra = algebra,
            .label = label,
            .metadata = .{},
        });
        errdefer _ = self.irreps.pop();
        const handle = IrrepHandle.init(@intCast(self.irreps.items.len - 1));
        try self.fillIrrepMetadata(handle);
        return handle;
    }

    /// dualIrrep returns the handle for the dual irrep, interning it if requested.
    pub fn dualIrrep(self: *Store, handle: IrrepHandle) anyerror!IrrepHandle {
        const index: usize = @intCast(handle.value);
        if (index >= self.irreps.items.len) return error.UnknownIrrep;
        if (self.irreps.items[index].metadata.dual) |dual| return dual;

        const record = self.irreps.items[index];
```

```
        const dual = switch (record.label) {
            .u1_charge => |charge| try self.internIrrep(record.algebra, .{ .u1_charge = .{
                .numerator = -charge.numerator,
                .denominator = charge.denominator,
            } }),
            .dynkin => |label| dual: {
                const algebra = self.algebraRecord(record.algebra) orelse return
                ↪  error.UnknownAlgebra;
                const dual_label = try root_data.dualDynkin(self.allocator,
                ↪  algebra.spec.simple, label);
                defer self.allocator.free(dual_label);
                break :dual try self.internIrrep(record.algebra, .{ .dynkin = dual_label });
            },
        };

        self.irreps.items[index].metadata.dual = dual;
        const dual_index: usize = @intCast(dual.value);
        if (dual_index < self.irreps.items.len) self.irreps.items[dual_index].metadata.dual =
        ↪  handle;
        return dual;
    }

    /// containsIrrep returns whether an irrep handle belongs to this store.
    pub fn containsIrrep(self: Store, handle: IrrepHandle) bool {
        return self.irrepRecord(handle) != null;
    }

    /// irrepMetadata returns derived metadata for an irrep handle.
    pub fn irrepMetadata(self: Store, handle: IrrepHandle) ?IrrepMetadata {
        const record = self.irrepRecord(handle) orelse return null;
        return record.metadata;
    }

    /// irrepAlgebra returns the algebra that owns an irrep handle.
    pub fn irrepAlgebra(self: Store, handle: IrrepHandle) ?AlgebraHandle {
        const record = self.irrepRecord(handle) orelse return null;
        return record.algebra;
    }

    fn algebraRecord(self: Store, handle: AlgebraHandle) ?AlgebraRecord {
        const index: usize = @intCast(handle.value);
        if (index >= self.algebras.items.len) return null;
        return self.algebras.items[index];
    }

    fn irrepRecord(self: Store, handle: IrrepHandle) ?IrrepRecord {
        const index: usize = @intCast(handle.value);
        if (index >= self.irreps.items.len) return null;
        return self.irreps.items[index];
    }

    fn fillIrrepMetadata(self: *Store, handle: IrrepHandle) anyerror!void {
        const index: usize = @intCast(handle.value);
        const record = self.irreps.items[index];
        const algebra = self.algebraRecord(record.algebra) orelse return error.UnknownAlgebra;

        switch (record.label) {
            .u1_charge => |charge| {
                self.irreps.items[index].metadata.dimension = 1;
                self.irreps.items[index].metadata.quadratic_casimir = try
                ↪  u1QuadraticCasimir(charge);
                if (charge.numerator == 0) self.irreps.items[index].metadata.dual = handle;
            },
            .dynkin => |label| {
```

```
                const simple = algebra.spec.simple;
                self.irreps.items[index].metadata.dimension = try
                ↪ root_data.dimension(self.allocator, simple, label);
                self.irreps.items[index].metadata.quadratic_casimir = try
                ↪ root_data.quadraticCasimir(self.allocator, simple, label,
                ↪ self.conventions);
                const dual_label = try root_data.dualDynkin(self.allocator, simple, label);
                defer self.allocator.free(dual_label);
                if (std.mem.eql(i16, label, dual_label)) {
                    self.irreps.items[index].metadata.dual = handle;
                } else if (self.findIrrep(record.algebra, .{ .dynkin = dual_label })) |dual| {
                    self.irreps.items[index].metadata.dual = dual;
                }
            },
        }
    }

    fn findIrrep(self: Store, algebra: AlgebraHandle, spec: IrrepSpec) ?IrrepHandle {
        for (self.irreps.items, 0..) |record, index| {
            if (handleEql(record.algebra, algebra) and irrepLabelSpecEql(record.label, spec))
            ↪ {
                return IrrepHandle.init(@intCast(index));
            }
        }
        return null;
    }
};
```

Private records carry derived metadata, root data, and future product-channel occurrences. They are not exported because callers should not depend on the storage layout.

```
const AlgebraRecord = struct {
    spec: AlgebraSpec,
    root_data: ?root_data.RootDatum,
};

const IrrepRecord = struct {
    algebra: AlgebraHandle,
    label: IrrepLabel,
    metadata: IrrepMetadata,
};

const IrrepLabel = union(enum) {
    u1_charge: symmetry.U1ChargeRational,
    dynkin: []const i16,
};

const ChannelRecord = struct {
    irrep: IrrepHandle,
    multiplicity_copy: u16,
};

const OccurrenceKind = enum {
    external_leg,
    product_node,
    realization,
};

const OccurrenceRecord = struct {
    channel: ChannelHandle,
    kind: OccurrenceKind,
    source_id: u32,
    realization: ?u32 = null,
};
```

Interning equality is structural and allocation-free. Validation happens before interning: a simple-algebra irrep must have one non-negative Dynkin label per rank, and a U(1) irrep must be a rational charge. Unknown dimension and dual data is represented by `null`, never by a fake handle.

```
fn handleEql(left: AlgebraHandle, right: AlgebraHandle) bool {
    return left.value == right.value;
}

fn validateAlgebraSpec(spec: AlgebraSpec) !void {
    switch (spec) {
        .u1 => {},
        .simple => |simple| try root_data.validateSimple(simple),
    }
}

fn makeRootDatum(spec: AlgebraSpec, conventions: root_data.AlgebraConventions)
↪   !?root_data.RootDatum {
    return switch (spec) {
        .u1 => null,
        .simple => |simple| try root_data.RootDatum.init(simple, conventions),
    };
}

fn algebraSpecEql(left: AlgebraSpec, right: AlgebraSpec) bool {
    return switch (left) {
        .u1 => switch (right) {
            .u1 => true,
            .simple => false,
        },
        .simple => |left_simple| switch (right) {
            .u1 => false,
            .simple => |right_simple| left_simple.family == right_simple.family and
            ↪   left_simple.rank == right_simple.rank,
        },
    };
}

fn validateIrrepSpec(algebra: AlgebraSpec, spec: IrrepSpec) !void {
    switch (algebra) {
        .u1 => switch (spec) {
            .u1_charge => |charge| {
                if (charge.denominator == 0) return error.InvalidU1Charge;
            },
            .dynkin => return error.InvalidIrrepForAlgebra,
        },
        .simple => |simple| switch (spec) {
            .u1_charge => return error.InvalidIrrepForAlgebra,
            .dynkin => |label| {
                if (label.len != simple.rank) return error.InvalidDynkinRank;
                for (label) |entry| {
                    if (entry < 0) return error.InvalidDynkinLabel;
                }
            },
        },
    }
}

fn u1QuadraticCasimir(charge: symmetry.U1ChargeRational) !root_data.Rational {
    const numerator = @as(i128, charge.numerator) * @as(i128, charge.numerator);
    const denominator = @as(i128, charge.denominator) * @as(i128, charge.denominator);
    return root_data.Rational.init(numerator, denominator);
}
```

```
fn irrepLabelSpecEql(left: IrrepLabel, right: IrrepSpec) bool {
    return switch (left) {
        .u1_charge => |left_charge| switch (right) {
            .u1_charge => |right_charge| left_charge.numerator == right_charge.numerator and
            ↪  left_charge.denominator == right_charge.denominator,
            .dynkin => false,
        },
        .dynkin => |left_dynkin| switch (right) {
            .u1_charge => false,
            .dynkin => |right_dynkin| std.mem.eql(i16, left_dynkin, right_dynkin),
        },
    };
}

fn cloneIrrepSpec(allocator: std.mem.Allocator, spec: IrrepSpec) !IrrepLabel {
    return switch (spec) {
        .u1_charge => |charge| .{ .u1_charge = charge },
        .dynkin => |label| .{ .dynkin = try allocator.dupe(i16, label) },
    };
}

fn freeIrrepLabel(allocator: std.mem.Allocator, label: IrrepLabel) void {
    switch (label) {
        .dynkin => |dynkin| allocator.free(dynkin),
        .u1_charge => {},
    }
}
```

8 Tensor Decomposition

A tensor-product decomposition is the compact statement

$$R \otimes S = \bigoplus_{T} m_T T.$$

This module stores that statement as flat product terms. It does not compute the decomposition; the family backend will do that and intern the result here.

```
const std = @import("std");
const coupling = @import("coupling.zig");
const store = @import("representation-store.zig");

/// ProductDecompositionHandle names a cached binary decomposition.
pub const ProductDecompositionHandle = struct {
    value: u32,

    /// init constructs a product-decomposition handle.
    pub fn init(value: u32) ProductDecompositionHandle {
        return .{ .value = value };
    }
};

/// ProductKey identifies a cached tensor-product decomposition.
pub const ProductKey = struct {
    left: store.IrrepHandle,
    right: store.IrrepHandle,
};

/// ProductTerm stores one channel and its multiplicity in a product.
pub const ProductTerm = struct {
    irrep: store.IrrepHandle,
    multiplicity: u16,
};
```

```
/// ProductTermSpan stores a flat range of product terms.
pub const ProductTermSpan = struct {
    offset: u32,
    len: u32,
};

/// Store owns cached binary tensor-product decomposition records.
pub const Store = struct {
    allocator: std.mem.Allocator,
    decompositions: std.ArrayList(ProductDecomposition) = .empty,
    terms: std.ArrayList(ProductTerm) = .empty,
    product_index: std.AutoHashMap(u64, ProductDecompositionHandle),

    /// init constructs an empty decomposition store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{
            .allocator = allocator,
            .product_index = std.AutoHashMap(u64, ProductDecompositionHandle).init(allocator),
        };
    }

    /// deinit releases decomposition-store memory.
    pub fn deinit(self: *Store) void {
        self.product_index.deinit();
        self.terms.deinit(self.allocator);
        self.decompositions.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// findProduct returns a cached product handle if present.
    pub fn findProduct(self: Store, key: ProductKey) ?ProductDecompositionHandle {
        return self.product_index.get(productKeyBits(key));
    }

    /// internProduct stores a binary product decomposition.
    pub fn internProduct(self: *Store, key: ProductKey, terms: []const ProductTerm)
    ↪   !ProductDecompositionHandle {
        if (self.findProduct(key)) |handle| return handle;

        const offset: u32 = @intCast(self.terms.items.len);
        try self.terms.appendSlice(self.allocator, terms);
        errdefer self.terms.shrinkRetainingCapacity(offset);

        const handle =
        ↪   ProductDecompositionHandle.init(@intCast(self.decompositions.items.len));
        try self.decompositions.append(self.allocator, .{
            .key = key,
            .terms = .{ .offset = offset, .len = @intCast(terms.len) },
        });
        errdefer _ = self.decompositions.pop();

        try self.product_index.put(productKeyBits(key), handle);
        return handle;
    }

    /// productTerms returns the cached term span for a product handle.
    pub fn productTerms(self: Store, handle: ProductDecompositionHandle) ?[]const ProductTerm
    ↪ {
        const index: usize = @intCast(handle.value);
        if (index >= self.decompositions.items.len) return null;
        const span = self.decompositions.items[index].terms;
        const start: usize = @intCast(span.offset);
        const end = start + span.len;
```

```
        return self.terms.items[start..end];
    }
};

/// ProductDecomposition stores a flat range of product terms.
const ProductDecomposition = struct {
    key: ProductKey,
    terms: ProductTermSpan,
};

/// SingletCountRequest records a generic invariant-counting problem.
const SingletCountRequest = struct {
    algebra: store.AlgebraHandle,
    factors: []const store.IrrepHandle,
    target: coupling.TargetIrrep = .singlet,
};

fn productKeyBits(key: ProductKey) u64 {
    return (@as(u64, key.left.value) << 32) | @as(u64, key.right.value);
}
```

9 Tensor Realization

Realizations map abstract irreps to concrete index layouts. This is where a field acquires caller-visible slots such as vector, spinor, fundamental, or adjoint indices. The coupling search still reasons about the target irrep, so projected realizations like an irrep inside $V \otimes S$ do not force every later step to carry the full ambient product.

```
const std = @import("std");
const store = @import("representation-store.zig");
```

Public handles identify registered concrete realizations and external legs.

```
/// RealizationHandle names a concrete index realization of an irrep.
pub const RealizationHandle = struct {
    value: u32,

    /// init constructs a realization handle from a store index.
    pub fn init(value: u32) RealizationHandle {
        return .{ .value = value };
    }
};

/// ExternalLegHandle names one external leg in an invariant request.
pub const ExternalLegHandle = struct {
    value: u32,

    /// init constructs an external-leg handle from a store index.
    pub fn init(value: u32) ExternalLegHandle {
        return .{ .value = value };
    }
};
```

Construction records are public because CFT code must be able to describe its free indices. The arrays are caller-owned until they are registered in the context.

```
/// RealizationRef identifies a primitive or registered external tensor shape.
pub const RealizationRef = union(enum) {
    primitive_irrep: store.IrrepHandle,
    handle: RealizationHandle,
    registered_name: []const u8,
};
```

```
/// IndexKind classifies the visible slot type of an external index.
pub const IndexKind = enum {
    vector,
    spinor,
    conjugate_spinor,
    fundamental,
    anti_fundamental,
    adjoint,
    form,
    custom,
};

/// NamedIndex records one caller-supplied free index name.
pub const NamedIndex = struct {
    kind: IndexKind,
    name: []const u8,

    /// init constructs one caller-named index slot.
    pub fn init(kind: IndexKind, name: []const u8) NamedIndex {
        return .{ .kind = kind, .name = name };
    }
};

/// RealizationChannel selects one irrep copy inside a realization product.
pub const RealizationChannel = struct {
    irrep: store.IrrepHandle,
    copy: u16 = 0,

    /// init constructs a selected irrep copy.
    pub fn init(irrep: store.IrrepHandle, copy: u16) RealizationChannel {
        return .{ .irrep = irrep, .copy = copy };
    }

    /// first selects the first copy of an irrep.
    pub fn first(irrep: store.IrrepHandle) RealizationChannel {
        return .{ .irrep = irrep };
    }
};

/// ExternalLeg describes a public invariant leg with named free indices.
pub const ExternalLeg = struct {
    realization: RealizationRef,
    indices: []const NamedIndex,
    label: ?[]const u8 = null,

    /// primitive constructs a leg whose realization is the irrep itself.
    pub fn primitive(irrep: store.IrrepHandle, indices: []const NamedIndex) ExternalLeg {
        return .{
            .realization = .{ .primitive_irrep = irrep },
            .indices = indices,
        };
    }

    /// realized constructs a leg from a registered realization handle.
    pub fn realized(realization: RealizationHandle, indices: []const NamedIndex) ExternalLeg {
        return .{
            .realization = .{ .handle = realization },
            .indices = indices,
        };
    }

    /// registered constructs a leg from a preset realization name.
    pub fn registered(name: []const u8, indices: []const NamedIndex) ExternalLeg {
```

```
        return .{
            .realization = .{ .registered_name = name },
            .indices = indices,
        };
    }

    /// named returns the leg with a caller-facing label attached.
    pub fn named(self: ExternalLeg, label: []const u8) ExternalLeg {
        var out = self;
        out.label = label;
        return out;
    }
};

/// RealizationSpec describes a primitive or projected external index model.
pub const RealizationSpec = struct {
    channel: RealizationChannel,
    ambient: []const store.IrrepHandle = &.{},
    indices: []const NamedIndex,
    name: ?[]const u8 = null,

    /// primitive constructs a direct index realization of an irrep.
    pub fn primitive(target: store.IrrepHandle, indices: []const NamedIndex) RealizationSpec {
        return .{
            .channel = RealizationChannel.first(target),
            .indices = indices,
        };
    }

    /// projected constructs an irrep realization inside an ambient product.
    pub fn projected(target: store.IrrepHandle, ambient: []const store.IrrepHandle, indices:
    ↪  []const NamedIndex, copy: u16) RealizationSpec {
        return RealizationSpec.projectedChannel(RealizationChannel.init(target, copy),
        ↪  ambient, indices);
    }

    /// projectedChannel constructs a realization from an explicit channel.
    pub fn projectedChannel(channel: RealizationChannel, ambient: []const store.IrrepHandle,
    ↪  indices: []const NamedIndex) RealizationSpec {
        return .{
            .channel = channel,
            .ambient = ambient,
            .indices = indices,
        };
    }

    /// named returns the realization spec with a preset-facing name attached.
    pub fn named(self: RealizationSpec, name: []const u8) RealizationSpec {
        var out = self;
        out.name = name;
        return out;
    }

    /// targetIrrep returns the abstract irrep realized by this index model.
    pub fn targetIrrep(self: RealizationSpec) store.IrrepHandle {
        return self.channel.irrep;
    }
};
```

The store owns cloned realization specs and interns repeated requests.

```
/// Store owns interned concrete realization records.
pub const Store = struct {
    allocator: std.mem.Allocator,
```

```
    specs: std.ArrayList(RealizationSpec) = .empty,

    /// init constructs an empty realization store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases all realization-store memory.
    pub fn deinit(self: *Store) void {
        for (self.specs.items) |spec| {
            freeRealizationSpec(self.allocator, spec);
        }
        self.specs.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// internRealization returns the stable handle for a realization spec.
    pub fn internRealization(self: *Store, spec: RealizationSpec) !RealizationHandle {
        for (self.specs.items, 0..) |stored, index| {
            if (realizationSpecEql(stored, spec)) {
                return RealizationHandle.init(@intCast(index));
            }
        }

        const owned = try cloneRealizationSpec(self.allocator, spec);
        errdefer freeRealizationSpec(self.allocator, owned);
        try self.specs.append(self.allocator, owned);
        return RealizationHandle.init(@intCast(self.specs.items.len - 1));
    }

    /// containsRealization returns whether a realization handle belongs to this store.
    pub fn containsRealization(self: Store, handle: RealizationHandle) bool {
        return self.realizationSpec(handle) != null;
    }

    /// targetIrrep returns the irrep realized by a stored realization handle.
    pub fn targetIrrep(self: Store, handle: RealizationHandle) ?store.IrrepHandle {
        const spec = self.realizationSpec(handle) orelse return null;
        return spec.targetIrrep();
    }

    fn realizationSpec(self: Store, handle: RealizationHandle) ?RealizationSpec {
        const index: usize = @intCast(handle.value);
        if (index >= self.specs.items.len) return null;
        return self.specs.items[index];
    }
};
```

These private records describe the future projector derivation path for a realization. The selected channel is a single value, so future embedding conventions have one natural place to attach.

```
/// IndexSlotSpec declares one slot in a realization layout.
const IndexSlotSpec = struct {
    kind: IndexKind,
    default_name: ?[]const u8 = null,
};

/// RealizationRequest declares an irrep as a projected ambient tensor shape.
const RealizationRequest = struct {
    channel: RealizationChannel,
    ambient: []const store.IrrepHandle,
    index_layout: []const IndexSlotSpec,
};
```

```
/// RealizationKind distinguishes primitive from projected realizations.
const RealizationKind = enum {
    primitive,
    projected_channel,
};

/// RealizationRecord stores one concrete index layout for an irrep.
const RealizationRecord = struct {
    channel: RealizationChannel,
    kind: RealizationKind,
    ambient_offset: u32,
    ambient_len: u16,
    index_layout_offset: u32,
    index_layout_len: u16,
    projector: ?RealizationProjectorId = null,
};

/// RealizationProjectorId names a derived boundary realization projector.
const RealizationProjectorId = u32;

/// AmbientFactor records one irrep in an ambient realization product.
const AmbientFactor = struct {
    irrep: store.IrrepHandle,
};

/// RealizationRule records the construction request for one realization.
const RealizationRule = struct {
    request: RealizationRequest,
    projector: ?RealizationProjectorId = null,
};
```

Structural equality and clone/free helpers are private implementation details. They keep public construction records lightweight while making registered realizations stable after caller buffers go out of scope.

```
fn realizationSpecEql(left: RealizationSpec, right: RealizationSpec) bool {
    return realizationChannelEql(left.channel, right.channel) and
        optionalStringEql(left.name, right.name) and
        irrepSliceEql(left.ambient, right.ambient) and
        namedIndexSliceEql(left.indices, right.indices);
}

fn realizationChannelEql(left: RealizationChannel, right: RealizationChannel) bool {
    return left.irrep.value == right.irrep.value and left.copy == right.copy;
}

fn irrepSliceEql(left: []const store.IrrepHandle, right: []const store.IrrepHandle) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_item, right_item| {
        if (left_item.value != right_item.value) return false;
    }
    return true;
}

fn namedIndexSliceEql(left: []const NamedIndex, right: []const NamedIndex) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_item, right_item| {
        if (left_item.kind != right_item.kind) return false;
        if (!std.mem.eql(u8, left_item.name, right_item.name)) return false;
    }
    return true;
}
```

```
fn optionalStringEql(left: ?[]const u8, right: ?[]const u8) bool {
    if (left == null or right == null) return left == null and right == null;
    return std.mem.eql(u8, left.?, right.?);
}

fn cloneRealizationSpec(allocator: std.mem.Allocator, spec: RealizationSpec) !RealizationSpec
↪ {
    const ambient = try allocator.dupe(store.IrrepHandle, spec.ambient);
    errdefer allocator.free(ambient);

    const indices = try cloneNamedIndices(allocator, spec.indices);
    errdefer freeNamedIndices(allocator, indices);

    const name = if (spec.name) |value| try allocator.dupe(u8, value) else null;
    errdefer if (name) |value| allocator.free(value);

    return .{
        .channel = spec.channel,
        .ambient = ambient,
        .indices = indices,
        .name = name,
    };
}

fn freeRealizationSpec(allocator: std.mem.Allocator, spec: RealizationSpec) void {
    allocator.free(spec.ambient);
    freeNamedIndices(allocator, spec.indices);
    if (spec.name) |name| allocator.free(name);
}

fn cloneNamedIndices(allocator: std.mem.Allocator, indices: []const NamedIndex) ![]NamedIndex
↪ {
    const owned = try allocator.alloc(NamedIndex, indices.len);
    var initialized: usize = 0;
    errdefer {
        for (owned[0..initialized]) |previous| allocator.free(previous.name);
        allocator.free(owned);
    }

    for (indices, owned[0..]) |source, *target| {
        const name = try allocator.dupe(u8, source.name);
        target.* = .{ .kind = source.kind, .name = name };
        initialized += 1;
    }

    return owned;
}

fn freeNamedIndices(allocator: std.mem.Allocator, indices: []const NamedIndex) void {
    for (indices) |index| {
        allocator.free(index.name);
    }
    allocator.free(indices);
}
```

10 Tensor Projector

A projector is an idempotent intertwiner. It can select a channel in $R \otimes S$, impose a boundary realization such as a gamma-traceless subspace, or change between invariant-basis conventions. This module stores projector identities and derivation labels; it does not construct formulas.

```zig
const std = @import("std");
const realization = @import("realization.zig");
const rendering = @import("rendering.zig");
const store = @import("representation-store.zig");

/// ProjectorId names one local projector or Clebsch-Gordan kernel.
pub const ProjectorId = u32;

/// ProjectorConvention records configurable projector-rendering choices.
pub const ProjectorConvention = struct {
    render_mode: rendering.ProjectorRenderMode = .named,
    signature: rendering.SignatureConvention = .complex,
    normalization_id: u32 = 0,
};

/// ProjectorRole records what mathematical object a projector selects.
pub const ProjectorRole = union(enum) {
    product_channel: ProductChannelKey,
    boundary_realization: realization.RealizationHandle,
    basis_change: BasisChangeKey,
};

/// ProductChannelKey identifies one target channel inside a binary product.
pub const ProductChannelKey = struct {
    left: store.IrrepHandle,
    right: store.IrrepHandle,
    target: store.ChannelHandle,
};

/// BasisChangeKey identifies a change of invariant-basis convention.
pub const BasisChangeKey = struct {
    source_basis: u32,
    target_basis: u32,
};

/// Store owns projector identities and compact emitted-term spans.
pub const Store = struct {
    allocator: std.mem.Allocator,
    records: std.ArrayList(ProjectorRecord) = .empty,

    /// init constructs an empty projector store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases projector-store memory.
    pub fn deinit(self: *Store) void {
        self.records.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// internProjector returns a stable id for a projector identity.
    pub fn internProjector(self: *Store, key: ProjectorKey, kind: ProjectorKind) !ProjectorId
    ↪ {
        for (self.records.items, 0..) |record, index| {
            if (projectorKeyEql(record.key, key)) return @intCast(index);
        }
        try self.records.append(self.allocator, .{
            .key = key,
            .kind = kind,
            .term_offset = 0,
            .term_len = 0,
        });
        return @intCast(self.records.items.len - 1);
```

```
    }
};

/// ProjectorKey identifies one projector in one convention.
pub const ProjectorKey = struct {
    role: ProjectorRole,
    convention: ProjectorConvention,
};

/// ProjectorKind records how a local projector was derived.
pub const ProjectorKind = enum {
    identity,
    quadratic_casimir,
    higher_casimir_signature,
    local_idempotent_split,
    highest_weight_solver,
    backend_specific_verified,
};

/// ProjectorRecord stores one local kernel and its derivation kind.
const ProjectorRecord = struct {
    key: ProjectorKey,
    kind: ProjectorKind,
    term_offset: u32,
    term_len: u32,
};

/// ProjectorAudit stores local splitting diagnostics for feasibility studies.
const ProjectorAudit = struct {
    product: ProductChannelKey,
    channel_count: u32,
    quadratic_collision_count: u32,
    true_multiplicity_count: u32,
    residual_block_max: u32,
};

fn projectorKeyEql(left: ProjectorKey, right: ProjectorKey) bool {
    return projectorRoleEql(left.role, right.role) and
        left.convention.render_mode == right.convention.render_mode and
        left.convention.signature == right.convention.signature and
        left.convention.normalization_id == right.convention.normalization_id;
}

fn projectorRoleEql(left: ProjectorRole, right: ProjectorRole) bool {
    return switch (left) {
        .product_channel => |left_key| switch (right) {
            .product_channel => |right_key| left_key.left.value == right_key.left.value and
                left_key.right.value == right_key.right.value and
                left_key.target.value == right_key.target.value,
            else => false,
        },
        .boundary_realization => |left_handle| switch (right) {
            .boundary_realization => |right_handle| left_handle.value == right_handle.value,
            else => false,
        },
        .basis_change => |left_key| switch (right) {
            .basis_change => |right_key| left_key.source_basis == right_key.source_basis and
            ↪  left_key.target_basis == right_key.target_basis,
            else => false,
        },
    };
}
```

11 Tensor Coupling

Coupling owns invariant-basis requests and, later, the compact paths that span those bases. It does not own dense tensor values. A basis element should be a small recipe: which external legs are coupled, which intermediate channels are used, and which local projector kernels render the final invariant if a caller asks for explicit components.

```
const std = @import("std");
const projector = @import("projector.zig");
const realization = @import("realization.zig");
const store = @import("representation-store.zig");
```

Basis and invariant handles are public ids. The path records behind them stay private because we want freedom to change the solver layout.

```
/// BasisHandle names a generated invariant basis.
pub const BasisHandle = struct {
    value: u32,

    /// init constructs a basis handle from a store index.
    pub fn init(value: u32) BasisHandle {
        return .{ .value = value };
    }
};

/// InvariantHandle names one invariant inside a basis.
pub const InvariantHandle = struct {
    value: u32,

    /// init constructs an invariant handle from a store index.
    pub fn init(value: u32) InvariantHandle {
        return .{ .value = value };
    }
};
```

The public request is deliberately small: algebra, external legs, target, and basis policy. The default target is the singlet because that is the CFT workflow for invariant tensor structures.

```
/// TargetIrrep selects the target representation, usually the singlet.
pub const TargetIrrep = union(enum) {
    singlet,
    irrep: store.IrrepHandle,
};

/// TreePolicy controls the coupling tree used for a basis.
pub const TreePolicy = union(enum) {
    auto,
    fixed: []const u16,
};

/// BasisPolicy controls the public basis convention.
pub const BasisPolicy = enum {
    default,
    coupling_paths,
    channel_adapted,
};

/// InvariantBasisRequest is the CFT-facing request for invariant tensors.
pub const InvariantBasisRequest = struct {
    algebra: store.AlgebraHandle,
    external_legs: []const realization.ExternalLeg,
    target: TargetIrrep = .singlet,
    tree_policy: TreePolicy = .auto,
```

```
    basis_policy: BasisPolicy = .default,
};

/// TensorExpansionTerm stores a CFT coefficient attached to one invariant id.
pub const TensorExpansionTerm = struct {
    rational_function_id: u32,
    invariant: InvariantHandle,
    scalar_id: u32,
};
```

The store interns basis requests. This is not invariant generation yet; it is the stable ownership layer the generator will consume. The important invariant is that caller-owned slices can disappear after registration.

```
/// Store owns interned invariant-basis requests.
pub const Store = struct {
    allocator: std.mem.Allocator,
    requests: std.ArrayList(InvariantBasisRequest) = .empty,

    /// init constructs an empty coupling store.
    pub fn init(allocator: std.mem.Allocator) Store {
        return .{ .allocator = allocator };
    }

    /// deinit releases all coupling-store memory.
    pub fn deinit(self: *Store) void {
        for (self.requests.items) |request| {
            freeRequest(self.allocator, request);
        }
        self.requests.deinit(self.allocator);
        self.* = Store.init(self.allocator);
    }

    /// internBasisRequest returns a stable basis handle for a request.
    pub fn internBasisRequest(self: *Store, request: InvariantBasisRequest) !BasisHandle {
        for (self.requests.items, 0..) |stored, index| {
            if (requestEql(stored, request)) {
                return BasisHandle.init(@intCast(index));
            }
        }

        const owned = try cloneRequest(self.allocator, request);
        errdefer freeRequest(self.allocator, owned);
        try self.requests.append(self.allocator, owned);
        return BasisHandle.init(@intCast(self.requests.items.len - 1));
    }
};
```

Internal coupling records use offsets and lengths into flat stores. This is the shape the future dynamic-programming solver should fill without allocating large symbolic expressions.

```
/// CouplingInput names either an external leg or a previous coupling node.
const CouplingInput = union(enum) {
    external: realization.ExternalLegHandle,
    node: u32,
};

/// CouplingNodeRecord stores one binary coupling step.
const CouplingNodeRecord = struct {
    left: CouplingInput,
    right: CouplingInput,
    output: store.ChannelHandle,
    kernel: projector.ProjectorId,
```

```
};

/// CouplingPathRecord stores a compact invariant basis element.
const CouplingPathRecord = struct {
    external_offset: u32,
    external_len: u16,
    node_offset: u32,
    node_len: u16,
};

/// BasisRecord stores one generated invariant basis.
const BasisRecord = struct {
    request: InvariantBasisRequest,
    path_offset: u32,
    path_len: u32,
};

/// ReachabilityRecord stores pruning data for one coupling tree node.
const ReachabilityRecord = struct {
    node: u32,
    irrep_offset: u32,
    irrep_len: u32,
};
```

Request equality is structural. This makes repeated high-level calls cheap and deterministic before the expensive solver exists.

```
fn requestEql(left: InvariantBasisRequest, right: InvariantBasisRequest) bool {
    return left.algebra.value == right.algebra.value and
        targetEql(left.target, right.target) and
        treePolicyEql(left.tree_policy, right.tree_policy) and
        left.basis_policy == right.basis_policy and
        externalLegsEql(left.external_legs, right.external_legs);
}

fn targetEql(left: TargetIrrep, right: TargetIrrep) bool {
    return switch (left) {
        .singlet => switch (right) {
            .singlet => true,
            .irrep => false,
        },
        .irrep => |left_irrep| switch (right) {
            .singlet => false,
            .irrep => |right_irrep| left_irrep.value == right_irrep.value,
        },
    };
}

fn treePolicyEql(left: TreePolicy, right: TreePolicy) bool {
    return switch (left) {
        .auto => switch (right) {
            .auto => true,
            .fixed => false,
        },
        .fixed => |left_fixed| switch (right) {
            .auto => false,
            .fixed => |right_fixed| std.mem.eql(u16, left_fixed, right_fixed),
        },
    };
}

fn externalLegsEql(left: []const realization.ExternalLeg, right: []const
↪  realization.ExternalLeg) bool {
    if (left.len != right.len) return false;
```

```
    for (left, right) |left_leg, right_leg| {
        if (!externalLegEql(left_leg, right_leg)) return false;
    }
    return true;
}

fn externalLegEql(left: realization.ExternalLeg, right: realization.ExternalLeg) bool {
    return realizationRefEql(left.realization, right.realization) and
        namedIndicesEql(left.indices, right.indices) and
        optionalStringEql(left.label, right.label);
}

fn realizationRefEql(left: realization.RealizationRef, right: realization.RealizationRef) bool
↪  {
    return switch (left) {
        .primitive_irrep => |left_irrep| switch (right) {
            .primitive_irrep => |right_irrep| left_irrep.value == right_irrep.value,
            else => false,
        },
        .handle => |left_handle| switch (right) {
            .handle => |right_handle| left_handle.value == right_handle.value,
            else => false,
        },
        .registered_name => |left_name| switch (right) {
            .registered_name => |right_name| std.mem.eql(u8, left_name, right_name),
            else => false,
        },
    };
}

fn namedIndicesEql(left: []const realization.NamedIndex, right: []const
↪  realization.NamedIndex) bool {
    if (left.len != right.len) return false;
    for (left, right) |left_index, right_index| {
        if (left_index.kind != right_index.kind) return false;
        if (!std.mem.eql(u8, left_index.name, right_index.name)) return false;
    }
    return true;
}

fn optionalStringEql(left: ?[]const u8, right: ?[]const u8) bool {
    if (left == null or right == null) return left == null and right == null;
    return std.mem.eql(u8, left.?, right.?);
}
```

The clone/free helpers keep ownership local to this module. Later, generated path data can be added beside requests without changing the public request shape.

```
fn cloneRequest(allocator: std.mem.Allocator, request: InvariantBasisRequest)
↪  !InvariantBasisRequest {
    const external_legs = try cloneExternalLegs(allocator, request.external_legs);
    errdefer freeExternalLegs(allocator, external_legs);

    return .{
        .algebra = request.algebra,
        .external_legs = external_legs,
        .target = request.target,
        .tree_policy = try cloneTreePolicy(allocator, request.tree_policy),
        .basis_policy = request.basis_policy,
    };
}

fn freeRequest(allocator: std.mem.Allocator, request: InvariantBasisRequest) void {
    freeExternalLegs(allocator, request.external_legs);
```

```
    switch (request.tree_policy) {
        .fixed => |fixed| allocator.free(fixed),
        .auto => {},
    }
}

fn cloneTreePolicy(allocator: std.mem.Allocator, policy: TreePolicy) !TreePolicy {
    return switch (policy) {
        .auto => .auto,
        .fixed => |fixed| .{ .fixed = try allocator.dupe(u16, fixed) },
    };
}

fn cloneExternalLegs(allocator: std.mem.Allocator, legs: []const realization.ExternalLeg)
↪  ![]realization.ExternalLeg {
    const owned = try allocator.alloc(realization.ExternalLeg, legs.len);
    var initialized: usize = 0;
    errdefer {
        for (owned[0..initialized]) |leg| freeExternalLeg(allocator, leg);
        allocator.free(owned);
    }

    for (legs, owned[0..]) |source, *target| {
        target.* = try cloneExternalLeg(allocator, source);
        initialized += 1;
    }
    return owned;
}

fn freeExternalLegs(allocator: std.mem.Allocator, legs: []const realization.ExternalLeg) void
↪  {
    for (legs) |leg| freeExternalLeg(allocator, leg);
    allocator.free(legs);
}

fn cloneExternalLeg(allocator: std.mem.Allocator, leg: realization.ExternalLeg)
↪  !realization.ExternalLeg {
    const indices = try cloneNamedIndices(allocator, leg.indices);
    errdefer freeNamedIndices(allocator, indices);

    const label = if (leg.label) |value| try allocator.dupe(u8, value) else null;
    errdefer if (label) |value| allocator.free(value);

    const ref = try cloneRealizationRef(allocator, leg.realization);
    errdefer freeRealizationRef(allocator, ref);

    return .{ .realization = ref, .indices = indices, .label = label };
}

fn freeExternalLeg(allocator: std.mem.Allocator, leg: realization.ExternalLeg) void {
    freeRealizationRef(allocator, leg.realization);
    freeNamedIndices(allocator, leg.indices);
    if (leg.label) |label| allocator.free(label);
}

fn cloneRealizationRef(allocator: std.mem.Allocator, ref: realization.RealizationRef)
↪  !realization.RealizationRef {
    return switch (ref) {
        .primitive_irrep => |irrep| .{ .primitive_irrep = irrep },
        .handle => |handle| .{ .handle = handle },
        .registered_name => |name| .{ .registered_name = try allocator.dupe(u8, name) },
    };
}
```

```
fn freeRealizationRef(allocator: std.mem.Allocator, ref: realization.RealizationRef) void {
    switch (ref) {
        .registered_name => |name| allocator.free(name),
        .primitive_irrep, .handle => {},
    }
}

fn cloneNamedIndices(allocator: std.mem.Allocator, indices: []const realization.NamedIndex)
↪   ![]realization.NamedIndex {
    const owned = try allocator.alloc(realization.NamedIndex, indices.len);
    var initialized: usize = 0;
    errdefer {
        for (owned[0..initialized]) |index| allocator.free(index.name);
        allocator.free(owned);
    }

    for (indices, owned[0..]) |source, *target| {
        target.* = .{
            .kind = source.kind,
            .name = try allocator.dupe(u8, source.name),
        };
        initialized += 1;
    }
    return owned;
}

fn freeNamedIndices(allocator: std.mem.Allocator, indices: []const realization.NamedIndex)
↪   void {
    for (indices) |index| allocator.free(index.name);
    allocator.free(indices);
}
```

12 Tensor Rendering

Rendering turns compact invariant ids into concrete notation. Signature is a rendering/evaluation convention, not a different abstract representation. The symbolic output is streamed one term at a time so a large expanded projector does not become one large allocation.

```
const coupling = @import("coupling.zig");

/// SignatureConvention selects the metric convention for rendering/evaluation.
pub const SignatureConvention = enum {
    complex,
    euclidean,
    lorentzian_mostly_plus,
    lorentzian_mostly_minus,
};

/// ProjectorRenderMode controls how much derived projector data is expanded.
pub const ProjectorRenderMode = enum {
    named,
    expanded_blocks,
    expanded_terms,
};

/// RenderFormat selects the output syntax level.
pub const RenderFormat = enum {
    symbolic_terms,
    text,
};

/// RenderOptions controls streamed invariant rendering.
pub const RenderOptions = struct {
```

```
    format: RenderFormat = .symbolic_terms,
    projectors: ProjectorRenderMode = .named,
    signature: SignatureConvention = .complex,
    preserve_caller_index_names: bool = true,
};

/// EvalOptions controls component evaluation of one invariant.
pub const EvalOptions = struct {
    signature: SignatureConvention = .complex,
};

/// IndexRef names one symbolic index in a rendered term.
pub const IndexRef = u32;

/// IndexBlockId names a compact antisymmetric or ordered index block.
pub const IndexBlockId = u32;

/// RationalId names an exact rational coefficient in the scalar store.
pub const RationalId = u32;

/// GammaBlock stores one internal antisymmetrized gamma factor.
const GammaBlock = struct {
    spinor_left: IndexRef,
    spinor_right: IndexRef,
    vector_block: IndexBlockId,
    rank: u8,
    chirality: u8,
    duality: DualityTag = .none,
};

/// DualityTag records middle-form self-duality data.
pub const DualityTag = enum {
    none,
    self_dual,
    anti_self_dual,
};

/// SymbolicAtom stores one symbolic invariant contraction atom.
pub const SymbolicAtom = union(enum) {
    metric_pair: struct { left: IndexRef, right: IndexRef },
    generalized_delta: struct { upper: IndexBlockId, lower: IndexBlockId },
    epsilon: struct { block: IndexBlockId },
    hodge_star: struct { input: IndexBlockId, output: IndexBlockId },
    antisymmetrizer: struct { block: IndexBlockId },
    named_operator: struct { operator_id: u32, first_index: u32, index_len: u16 },
};

/// SymbolicTerm stores one streamed coefficient times symbolic atoms.
pub const SymbolicTerm = struct {
    coefficient: RationalId,
    /// atoms is borrowed and valid only until the sink returns.
    atoms: []const SymbolicAtom,
};

/// emitTerm sends one symbolic term to a caller-provided sink.
pub fn emitTerm(sink: anytype, term: SymbolicTerm) !void {
    try sink.emitTerm(term);
}

/// RenderedTermSpan stores a compact range of already emitted terms.
const RenderedTermSpan = struct {
    offset: u32,
    len: u32,
};
```

```
/// ReductionTerm stores one coefficient of an invariant in a target basis.
const ReductionTerm = struct {
    invariant: coupling.InvariantHandle,
    coefficient: RationalId,
};
```

On-Shell-Code

This code consumes CFT-Code and handles on-shell string theory computations. That is, given a set of vertex Operators, it computes their string amplitudes from their Correlators, via prescriptions appropriate for the given string theory. We shall support all five superstrings: heterotic $SO(32)$, $E_8 \times E_8$, Type IIA, Type IIB and Type I, and bosonic string theory. In both open, closed and open-closed sectors. For free fields, we can derive the open, open-closed sector data from that of the closed sector given a boundary condition on the fundamental fields (doubling trick). Note that each requires us to define the appropriate ghost system Theory sectors. The superstring theories each come with the a distinct notion of a GSO projection.

SFT-Code

This module defines string field theory (SFT) brackets and vertices for open, closed and open-closed string fields theories (for all five superstrings and the bosonic string). As such, it applies conformal transforms defined by consistent local coordinate data on Operators and follows up by computing their OPE (in case of brackets) or Correlators in the case of vertices, integrated over for now abstract chains in the moduli space, with appropriate PCO insertions in the superstring case.

NLSM-Code

This is an overview of NLSM-Code

Pure-Spinor-Code

This module computes OPE and Correlators in the CFT defined by the appropriately regularized pure spinor NLSM on $PSU(2,2|4)$.

Tests

A String-Code

A.1 API

```
test "top-level API exposes tensor and cft boundaries" {
    const testing = @import("std").testing;
    try testing.expect(@hasDecl(@This(), "tensor"));
    try testing.expect(@hasDecl(@This(), "cft"));
    try testing.expect(@hasDecl(cft, "FreeBoson"));
    try testing.expect(@hasDecl(cft, "Bc"));
    try testing.expect(@hasDecl(cft, "product"));
    try testing.expect(@hasDecl(cft, "boundary"));
    try testing.expect(@hasDecl(cft.FreeBoson, "make"));
```

```
    try testing.expect(@hasDecl(cft.FreeBoson, "boundary"));
    try testing.expect(@hasDecl(cft.Bc, "sphere"));
    try testing.expect(@hasDecl(cft.Bc, "diskBoundary"));
    try testing.expect(!@hasDecl(cft, "freeBoson"));
    try testing.expect(!@hasDecl(cft, "bcSphere"));
    try testing.expect(!@hasDecl(cft, "freeBosonBoundary"));
    try testing.expect(!@hasDecl(cft, "bcDiskBoundary"));
    try testing.expect(!@hasDecl(cft, "presets"));
    try testing.expect(!@hasDecl(cft, "kernel"));
    try testing.expect(!@hasDecl(cft, "theory"));
    try testing.expect(!@hasDecl(cft, "expressions"));
}

test "tensor context interns projected realization requests without expansion" {
    const testing = @import("std").testing;

    var ctx = try tensor.Context.init(testing.allocator);
    defer ctx.deinit();

    const so10 = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    const so10_again = try ctx.registerAlgebra(.{ .simple = .{ .family = .d, .rank = 5 } });
    try testing.expectEqual(so10.value, so10_again.value);

    const vector = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 0 } });
    const spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 0, 0, 0, 0, 1 } });
    const vector_spinor = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 } });
    const vector_spinor_again = try ctx.registerIrrep(so10, .{ .dynkin = &.{ 1, 0, 0, 0, 1 }
    ↪  });
    try testing.expectEqual(vector_spinor.value, vector_spinor_again.value);
    try testing.expectEqual(@as(u128, 10), ctx.irrepMetadata(vector).?.dimension.?);
    try testing.expectEqual(@as(u128, 16), ctx.irrepMetadata(spinor).?.dimension.?);
    try testing.expectEqual(@as(u128, 144), ctx.irrepMetadata(vector_spinor).?.dimension.?);
    const conjugate_spinor = try ctx.dualIrrep(spinor);
    try testing.expect(conjugate_spinor.value != spinor.value);
    try testing.expectEqual(spinor.value, (try ctx.dualIrrep(conjugate_spinor)).value);
    try testing.expectError(error.InvalidDynkinRank, ctx.registerIrrep(so10, .{ .dynkin = &.{
    ↪  1, 0 } }));

    const su2 = try ctx.registerAlgebra(.{ .simple = .{ .family = .a, .rank = 1 } });
    const fundamental = try ctx.registerIrrep(su2, .{ .dynkin = &.{1} });
    try testing.expectError(error.MixedRealizationAlgebras,
    ↪  ctx.registerRealization(tensor.RealizationSpec.projected(vector_spinor,
    ↪  &.{fundamental}, &.{
        .init(.vector, "m"),
        .init(.spinor, "a"),
    }, 0)));

    const channel = tensor.RealizationChannel.first(vector_spinor);
    const realization = try
    ↪  ctx.registerRealization(tensor.RealizationSpec.projectedChannel(channel, &.{ vector,
    ↪  spinor }, &.{
        .init(.vector, "m"),
        .init(.spinor, "a"),
    }).named("SO10.vector_spinor"));
    const realization_again = try
    ↪  ctx.registerRealization(tensor.RealizationSpec.projectedChannel(channel, &.{ vector,
    ↪  spinor }, &.{
        .init(.vector, "m"),
        .init(.spinor, "a"),
    }).named("SO10.vector_spinor"));
    try testing.expectEqual(realization.value, realization_again.value);

    const leg = tensor.ExternalLeg.realized(realization, &.{
        .init(.vector, "m1"),
```

```
        .init(.spinor, "a1"),
    });
    const basis = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ leg, leg, leg, leg, leg, leg },
    });
    const basis_again = try ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{ leg, leg, leg, leg, leg, leg },
    });
    try testing.expectEqual(basis.value, basis_again.value);

    try testing.expectError(error.MixedInvariantAlgebras, ctx.invariantBasis(.{
        .algebra = so10,
        .external_legs = &.{tensor.ExternalLeg.primitive(fundamental, &.{
            .init(.fundamental, "i"),
        })},
    }));

    const so9 = try ctx.registerAlgebra(.{ .simple = .{ .family = .b, .rank = 4 } });
    const so9_vector = try ctx.registerIrrep(so9, .{ .dynkin = &.{ 1, 0, 0, 0 } });
    const so9_spinor = try ctx.registerIrrep(so9, .{ .dynkin = &.{ 0, 0, 0, 1 } });
    try testing.expectEqual(@as(u128, 9), ctx.irrepMetadata(so9_vector).?.dimension.?);
    try testing.expectEqual(@as(u128, 16), ctx.irrepMetadata(so9_spinor).?.dimension.?);

    const e6 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e6, .rank = 6 } });
    const e7 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e7, .rank = 7 } });
    const e8 = try ctx.registerAlgebra(.{ .simple = .{ .family = .e8, .rank = 8 } });
    const e6_27 = try ctx.registerIrrep(e6, .{ .dynkin = &.{ 1, 0, 0, 0, 0, 0 } });
    const e7_56 = try ctx.registerIrrep(e7, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 1, 0 } });
    const e8_248 = try ctx.registerIrrep(e8, .{ .dynkin = &.{ 0, 0, 0, 0, 0, 0, 1, 0 } });
    try testing.expectEqual(@as(u128, 27), ctx.irrepMetadata(e6_27).?.dimension.?);
    try testing.expectEqual(@as(u128, 56), ctx.irrepMetadata(e7_56).?.dimension.?);
    try testing.expectEqual(@as(u128, 248), ctx.irrepMetadata(e8_248).?.dimension.?);
}
```

B CFT-Code

B.1 API

```
test "root CFT API exposes selected constructors without implementation namespaces" {
    const testing = @import("std").testing;
    try testing.expect(@hasDecl(@This(), "FreeBoson"));
    try testing.expect(@hasDecl(@This(), "Bc"));
    try testing.expect(@hasDecl(@This(), "product"));
    try testing.expect(@hasDecl(@This(), "boundary"));
    try testing.expect(@hasDecl(FreeBoson, "make"));
    try testing.expect(@hasDecl(FreeBoson, "boundary"));
    try testing.expect(@hasDecl(Bc, "sphere"));
    try testing.expect(@hasDecl(Bc, "diskBoundary"));
    try testing.expect(!@hasDecl(@This(), "freeBoson"));
    try testing.expect(!@hasDecl(@This(), "bcSphere"));
    try testing.expect(!@hasDecl(@This(), "freeBosonBoundary"));
    try testing.expect(!@hasDecl(@This(), "bcDiskBoundary"));
    try testing.expect(!@hasDecl(@This(), "presets"));
    try testing.expect(!@hasDecl(@This(), "kernel"));
    try testing.expect(!@hasDecl(@This(), "theory"));
    try testing.expect(!@hasDecl(@This(), "expressions"));
}
```

B.2 CFT Kernel

B.2.1 Generated Kernels

The first test protects the runtime call shape: a `MultiOp` carries the local operators and the label store needed to interpret them, without receiving a separate context.

```
test "MultiOp carries local operators and labels" {
    const testing = @import("std").testing;

    const labels = Call.LabelStore{
        .values = &.{ .{ .integer = 7 } },
    };
    const ops = [_]Call.LocalOp{
        .{
            .insertion = .{ .single = .{ .position = 1, .derivatives = 0 } },
            .kind = 2,
            .labels = 0,
        },
    };
    const input = Call.MultiOp{
        .operators = &ops,
        .labels = &labels,
    };

    try testing.expectEqual(@as(usize, 1), input.operators.len);
    try testing.expectEqual(@as(operators.OperatorKindId, 2), input.operators[0].kind);
    switch (input.labels.values[0]) {
        .integer => |value| try testing.expectEqual(@as(i64, 7), value),
        else => return error.UnexpectedLabelKind,
    }
}
```

The second test checks the kernel/theory boundary that matters now: a BCFT generated kernel extends the body schema without mutating the bulk generated kernel.

```
test "BCFT kernel extends bulk schema without mutating bulk schema" {
    const testing = @import("std").testing;

    const bulk_spec = theory.Spec.CFT{
        .sectors = &.{},
        .bulk_operator_kinds = &.{},
        .conformal_structure = 0,
        .conformal_structures = &.{},
        .quantum_names = &.{},
        .label_schemas = &.{},
        .conventions = &.{},
        .weight_rules = &.{},
        .quantum_rules = &.{},
        .statistics_rules = &.{},
        .strategy_support_rules = &.{},
        .wick_rule_sets = &.{},
        .pairing_data = &.{},
        .zero_mode_rule_sets = &.{},
        .correlator_configs = &.{},
        .local_ope_configs = &.{},
        .cocycle_tables = &.{},
        .presentations = &.{},
        .finite_projections = &.{},
        .lattices = &.{},
        .mode_algebras = &.{},
        .mode_action_rules = &.{},
    };
```

```
    const bcft_spec = theory.Spec.BCFT{
        .name = "empty boundary extension",
        .bulk = 0,
        .boundary_conditions = &.{},
        .boundary_stacks = &.{},
        .boundary_stress_tensor = .{ .kind = 0, .labels = 0 },
        .boundary_spin = &.{},
        .boundary_operator_kinds = &.{ 1, 2 },
        .boundary_changing_kinds = &.{},
        .wick_rule_sets = &.{},
        .pairing_data = &.{},
        .zero_mode_rule_sets = &.{},
        .correlator_configs = &.{},
        .local_ope_configs = &.{},
    };

    const Bulk = CFTKernel(bulk_spec);
    const BoundaryKernel = BCFTKernel(Bulk, bcft_spec);

    try testing.expectEqual(@as(usize, 0), Bulk.Bodies.operator_schema.boundary_kind_ids.len);
    try testing.expectEqual(@as(usize, 2),
    ↪  BoundaryKernel.Bodies.operator_schema.boundary_kind_ids.len);
    try testing.expectEqual(@as(operators.OperatorKindId, 1),
    ↪  BoundaryKernel.Bodies.operator_schema.boundary_kind_ids[0]);
}
```

B.3 Presets

B.3.1 Preset Families

The tests here are intentionally behavioral. They check concepts that should survive later implementation work: composition exposes only selected namespaces, rule slices compose from the supplied factors, labels remain out-of-line, and the current correlator boundary streams primitive Wick terms rather than allocating a final expression.

```
test "presets compose operator builders without exposing theory tables" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 26 });
    const BosonicSphere = product(.{ X, bcSphere(.{}) });
    const BosonicDisk = boundary(BosonicSphere, .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
            .chan_paton = X.target.boundaryStack("stack"),
        }),
        bcDiskBoundary(.{}),
    });

    var local = BosonicDisk.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const y = try local.boundaryCoord("y");
    const x = try BosonicDisk.op.free_boson_boundary.expXBoundary(&local, k, y);
    const c = try BosonicDisk.op.bc_boundary.cBoundary(&local, 0, y);
    const ops = try local.ops(.{ x, c });

    try testing.expectEqual(@as(usize, 16), BosonicDisk.config.disk.wick_rules.len);
    try testing.expectEqual(@as(usize, 30), BosonicDisk.config.disk.wick_rule_index.len);
```

```
    try testing.expectEqual(@as(usize, 2), BosonicDisk.config.disk.zero_modes.len);
    try testing.expectEqual(@as(usize, 6), BosonicDisk.config.disk.config_entries.len);
    switch (BosonicDisk.config.disk.config_entries[0].value) {
        .target_dimension => |dimension| try testing.expectEqual(@as(u16, 26), dimension),
        else => return error.UnexpectedBulkConfig,
    }
    switch (BosonicDisk.config.disk.config_entries[1].value) {
        .tensor_projector => |projector| try
        ↪  testing.expectEqual(@intFromEnum(X.target.subspace(&.{ 0, 1, 2, 3 })),
        ↪  @intFromEnum(projector)),
        else => return error.UnexpectedBoundaryConfig,
    }
    switch (BosonicDisk.config.disk.config_entries[5].value) {
        .boundary_stack => |stack| try
        ↪  testing.expectEqual(@intFromEnum(X.target.boundaryStack("stack")),
        ↪  @intFromEnum(stack)),
        else => return error.UnexpectedBoundaryStack,
    }
    try testing.expectEqual(@as(usize, 1), X.config.sphere.config_entries.len);
    switch (X.config.sphere.config_entries[0].value) {
        .target_dimension => |dimension| try testing.expectEqual(@as(u16, 26), dimension),
        else => return error.UnexpectedBulkConfig,
    }
    try testing.expectEqual(@as(usize, 2), ops.operators.len);
    switch (ops.labels.values[@intCast(ops.operators[0].labels)]) {
        .symbol => |value| try testing.expectEqual(@intFromEnum(k), value),
        else => return error.UnexpectedLabelKind,
    }
}

test "preset rules and namespaces compose from supplied factors" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const Ghost = bcSphere(.{});
    const MatterOnly = product(.{X});
    const GhostOnly = product(.{Ghost});
    const MatterGhost = product(.{ X, Ghost });

    try testing.expect(@hasDecl(MatterOnly.op, "free_boson"));
    try testing.expect(!@hasDecl(MatterOnly.op, "bc"));
    try testing.expect(@hasDecl(GhostOnly.op, "bc"));
    try testing.expect(!@hasDecl(GhostOnly.op, "free_boson"));
    try testing.expectEqual(@as(usize, 7), MatterOnly.config.sphere.wick_rules.len);
    try testing.expectEqual(@as(usize, 2), GhostOnly.config.sphere.wick_rules.len);
    try testing.expectEqual(@as(usize, 9), MatterGhost.config.sphere.wick_rules.len);
    try testing.expectEqual(@as(usize, 15), MatterGhost.config.sphere.wick_rule_index.len);

    const DiskMatter = boundary(MatterGhost, .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
        }),
    });
    const DiskMatterGhost = boundary(MatterGhost, .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
        }),
        bcDiskBoundary(.{}),
    });
```

```
    try testing.expect(@hasDecl(DiskMatter.op, "bulk"));
    try testing.expect(@hasDecl(DiskMatter.op, "free_boson_boundary"));
    try testing.expect(!@hasDecl(DiskMatter.op, "bc_boundary"));
    try testing.expect(@hasDecl(DiskMatterGhost.op, "bc_boundary"));
    try testing.expectEqual(@as(usize, 11), DiskMatter.config.disk.wick_rules.len);
    try testing.expectEqual(@as(usize, 16), DiskMatterGhost.config.disk.wick_rules.len);
}

test "preset compile-time flags change generated rule surfaces" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const HolomorphicBc = bcSphere(.{ .include_antiholomorphic_copy = false });
    const MatterGhost = product(.{ X, HolomorphicBc });
    const BoundaryOnlyGhost = boundary(HolomorphicBc, .{
        bcDiskBoundary(.{ .include_mixed_bulk_boundary = false }),
    });

    try testing.expect(@hasDecl(HolomorphicBc.op, "b"));
    try testing.expect(!@hasDecl(HolomorphicBc.op, "bt"));
    try testing.expectEqual(@as(usize, 1), HolomorphicBc.config.sphere.wick_rules.len);
    try testing.expectEqual(@as(usize, 8), MatterGhost.config.sphere.wick_rules.len);
    try testing.expectEqual(@as(usize, 1), BoundaryOnlyGhost.config.disk.wick_rules.len);
}

test "bc zero modes consume c jets by top-form saturation" {
    const testing = @import("std").testing;

    const Ghost = bcSphere(.{ .include_antiholomorphic_copy = false });

    var local = Ghost.local(testing.allocator);
    defer local.deinit();

    const z1 = try local.coord("z1");
    const z2 = try local.coord("z2");
    const z3 = try local.coord("z3");
    const ops = try local.ops(.{
        try Ghost.op.c(&local, 0, z1),
        try Ghost.op.c(&local, 1, z2),
        try Ghost.op.c(&local, 2, z3),
    });

    const Sink = struct {
        factor_count: usize = 0,
        base_end_count: usize = 0,
        derivative_sum: u16 = 0,

        /// emitZeroModeFactor records the top-form factor emitted by the base case.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            self.factor_count += 1;
            switch (factor) {
                .bc_top_form => |top| {
                    if (top.support != .sphere_holomorphic) return error.UnexpectedBcSupport;
                    self.derivative_sum = @as(u16, top.jets[0].derivative_order) +
                    ↪  top.jets[1].derivative_order + top.jets[2].derivative_order;
                },
                else => return error.UnexpectedZeroModeFactor,
            }
        }

        /// emitZeroModeBaseEnd records that every residual insertion was consumed.
        pub fn emitZeroModeBaseEnd(self: *@This()) !void {
            self.base_end_count += 1;
        }
```

```
    };

    var sink = Sink{};
    try testing.expect(try shared.emitZeroModeBaseCase(&Ghost.config.sphere, ops, null,
    ↪  &sink));
    try testing.expectEqual(@as(usize, 1), sink.factor_count);
    try testing.expectEqual(@as(usize, 1), sink.base_end_count);
    try testing.expectEqual(@as(u16, 3), sink.derivative_sum);
}

test "disk bc zero modes use one doubled chiral top form" {
    const testing = @import("std").testing;

    const Ghost = bcSphere(.{});
    const Disk = boundary(Ghost, .{bcDiskBoundary(.{})});

    var local = Disk.local(testing.allocator);
    defer local.deinit();

    const z = try local.coord("z");
    const zbar = try local.coord("zbar");
    const y = try local.boundaryCoord("y");
    const ops = try local.ops(.{
        try Disk.op.bulk.c(&local, 0, z),
        try Disk.op.bulk.ct(&local, 0, zbar),
        try Disk.op.bc_boundary.cBoundary(&local, 0, y),
    });

    const Sink = struct {
        saw_disk_doubled: bool = false,

        /// emitZeroModeFactor records whether the doubled disk top form was emitted.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            switch (factor) {
                .bc_top_form => |top| self.saw_disk_doubled = top.support == .disk_doubled,
                else => return error.UnexpectedZeroModeFactor,
            }
        }

        /// emitZeroModeBaseEnd accepts the successful zero-mode base case.
        pub fn emitZeroModeBaseEnd(self: *@This()) !void {
            _ = self;
        }
    };

    var sink = Sink{};
    try testing.expect(try shared.emitZeroModeBaseCase(&Disk.config.disk, ops, null, &sink));
    try testing.expect(sink.saw_disk_doubled);
}

test "free boson zero modes emit momentum delta and profile presentations" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });

    var local = X.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const z = try local.coord("z");
    const zbar = try local.coord("zbar");
    const f0 = try local.targetFunction("f");
    const fhat = try local.fourierTransform("fhat");
    const coeff = try local.profileCoefficient("a");
```

```
    const point = try local.targetPoint("x0");
    const position_profile = X.profile.positionSpace(f0);
    const fourier_profile = X.profile.fourier(fhat);
    const polynomial_profile = X.profile.polynomialRnc(point, &.{.{ .coefficient = coeff,
    ↪  .power = 2 }});
    const ops = try local.ops(.{
        try X.op.expX(&local, k, z, zbar),
        try X.op.profile(&local, position_profile, z, zbar),
        try X.op.profile(&local, fourier_profile, z, zbar),
        try X.op.profile(&local, polynomial_profile, z, zbar),
    });

    const Sink = struct {
        delta_count: usize = 0,
        gaussian_count: usize = 0,
        fourier_count: usize = 0,
        polynomial_count: usize = 0,
        base_end_count: usize = 0,
        two_pi_power: u16 = 0,

        /// emitZeroModeFactor records each free-boson zero-mode factor by presentation.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            switch (factor) {
                .momentum_delta => |delta| {
                    self.delta_count += 1;
                    self.two_pi_power = delta.two_pi_power;
                    if (delta.projector != null or delta.momenta.len != 1) return
                    ↪  error.UnexpectedMomentumDelta;
                },
                .profile_gaussian_differential => self.gaussian_count += 1,
                .profile_fourier_integral => self.fourier_count += 1,
                .profile_polynomial_integral => self.polynomial_count += 1,
                else => return error.UnexpectedZeroModeFactor,
            }
        }

        /// emitZeroModeBaseEnd records that all profile and plane-wave residuals were
        ↪  consumed.
        pub fn emitZeroModeBaseEnd(self: *@This()) !void {
            self.base_end_count += 1;
        }
    };

    var sink = Sink{};
    try testing.expect(try shared.emitZeroModeBaseCase(&X.config.sphere, ops, null, &sink));
    try testing.expectEqual(@as(usize, 1), sink.delta_count);
    try testing.expectEqual(@as(u16, 10), sink.two_pi_power);
    try testing.expectEqual(@as(usize, 1), sink.gaussian_count);
    try testing.expectEqual(@as(usize, 1), sink.fourier_count);
    try testing.expectEqual(@as(usize, 1), sink.polynomial_count);
    try testing.expectEqual(@as(usize, 1), sink.base_end_count);
}

test "disk free boson zero modes emit Neumann delta and Dirichlet phase" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const Disk = boundary(product(.{X}), .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
        }),
    });
```

```
    var local = Disk.local(testing.allocator);
    defer local.deinit();

    const k = try local.momentum("k");
    const y = try local.boundaryCoord("y");
    const ops = try local.ops(.{
        try Disk.op.free_boson_boundary.expXBoundary(&local, k, y),
    });

    const Sink = struct {
        delta_count: usize = 0,
        phase_count: usize = 0,
        base_end_count: usize = 0,
        two_pi_power: u16 = 0,
        saw_disk_scalar: bool = false,
        saw_projector: bool = false,
        saw_position: bool = false,

        /// emitZeroModeFactor records Neumann momentum conservation and Dirichlet phase data.
        pub fn emitZeroModeFactor(self: *@This(), factor: anytype) !void {
            switch (factor) {
                .momentum_delta => |delta| {
                    self.delta_count += 1;
                    self.two_pi_power = delta.two_pi_power;
                    self.saw_projector = delta.projector != null;
                    switch (delta.scalar) {
                        .monomial => |monomial| self.saw_disk_scalar = monomial.atom != null
                        ↪  and monomial.atom_power == 1,
                        else => return error.UnexpectedDiskNormalization,
                    }
                    if (delta.momenta.len != 1) return error.UnexpectedMomentumDelta;
                },
                .dirichlet_phase => |phase| {
                    self.phase_count += 1;
                    self.saw_position = phase.momenta.len == 1;
                    switch (phase.position) {
                        .target_point => {},
                        else => return error.UnexpectedDirichletPosition,
                    }
                },
                else => return error.UnexpectedZeroModeFactor,
            }
        }

        /// emitZeroModeBaseEnd records that the disk constant mode consumed the insertion.
        pub fn emitZeroModeBaseEnd(self: *@This()) !void {
            self.base_end_count += 1;
        }
    };

    var sink = Sink{};
    try testing.expect(try shared.emitZeroModeBaseCase(&Disk.config.disk, ops, null, &sink));
    try testing.expectEqual(@as(usize, 1), sink.delta_count);
    try testing.expectEqual(@as(usize, 1), sink.phase_count);
    try testing.expectEqual(@as(u16, 4), sink.two_pi_power);
    try testing.expect(sink.saw_disk_scalar);
    try testing.expect(sink.saw_projector);
    try testing.expect(sink.saw_position);
    try testing.expectEqual(@as(usize, 1), sink.base_end_count);
}

test "unconsumed free boson derivative kills the zero-mode base case" {
    const testing = @import("std").testing;
```

```
    const X = freeBoson(.{ .dimension = 10 });

    var local = X.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const z = try local.coord("z");
    const ops = try local.ops(.{
        try X.op.dX(&local, mu, 0, z),
    });

    const Sink = struct {
        /// emitZeroModeFactor fails if an unconsumed derivative field emits a zero-mode
        ↪  factor.
        pub fn emitZeroModeFactor(_: *@This(), _: anytype) !void {
            return error.UnexpectedZeroModeFactor;
        }

        /// emitZeroModeBaseEnd fails because the derivative residual should not be consumed.
        pub fn emitZeroModeBaseEnd(_: *@This()) !void {
            return error.UnexpectedZeroModeBaseEnd;
        }
    };

    var sink = Sink{};
    try testing.expect(!try shared.emitZeroModeBaseCase(&X.config.sphere, ops, null, &sink));
}

test "presets expose physics-named Wick rule templates" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10, .alpha_prime = scalars.atom("test_free_boson",
    ↪  "custom_alpha_prime") });
    const custom_alpha = scalars.atom("test_free_boson", "custom_alpha_prime");
    const Ghost = bcSphere(.{});
    const Disk = boundary(product(.{ X, Ghost }), .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
        }),
        bcDiskBoundary(.{}),
    });

    switch (scalars.rational(2, 4)) {
        .rational => |value| {
            try testing.expectEqual(@as(i64, 1), value.numerator);
            try testing.expectEqual(@as(i64, 2), value.denominator);
        },
        else => return error.UnexpectedScalarShape,
    }

    try testing.expectEqual(@as(usize, 7), X.rules.sphere_wick.len);
    try testing.expectEqual(@as(usize, 1), X.rules.sphere_wick[0].expr.terms.len);
    switch (X.rules.sphere_wick[0].expr.terms[0].coordinates[0]) {
        .difference_power => |factor| {
            try testing.expectEqual(@as(i16, -2), factor.exponent);
            try testing.expectEqual(.left, factor.coordinate.left.side);
            try testing.expect(factor.derivatives.include_left);
            try testing.expect(factor.derivatives.include_right);
        },
        else => return error.UnexpectedCoordinateFactor,
    }
```

```
    switch (X.rules.sphere_wick[0].expr.terms[0].scalars[0]) {
        .value => |scalar| switch (scalar) {
            .monomial => |monomial| {
                try testing.expectEqual(@as(i64, 1), monomial.rational.numerator);
                try testing.expectEqual(@as(i64, 2), monomial.rational.denominator);
                try testing.expectEqual(@as(u2, 0), monomial.imaginary_power);
                try testing.expectEqual(@as(i8, 1), monomial.atom_power);
                try testing.expectEqual(custom_alpha, monomial.atom.?);
            },
            else => return error.UnexpectedScalarShape,
        },
        else => return error.UnexpectedScalarFactor,
    }
    switch (Ghost.rules.sphere_wick[0].expr.terms[0].coordinates[0]) {
        .difference_power => |factor| {
            try testing.expectEqual(@as(i16, -1), factor.exponent);
            try testing.expect(factor.derivatives.include_left);
            try testing.expect(factor.derivatives.include_right);
        },
        else => return error.UnexpectedCoordinateFactor,
    }
    try testing.expectEqual(@as(usize, 2), Ghost.rules.sphere_wick.len);
    try testing.expectEqual(@as(usize, 16), Disk.rules.disk_wick.len);
}

test "pair Wick resolves derivative action on a vector profile" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });

    var local = X.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const z = try local.coord("z");
    const w = try local.coord("w");
    const wbar = try local.coord("wbar");
    const f0 = try local.targetFunction("f");
    const f = X.profile.positionSpace(f0);

    const dx = try X.op.dX(&local, mu, 0, z);
    const profile = try X.op.profileVector(&local, f, nu, w, wbar);
    const ops = try local.ops(.{ dx, profile });

    const Sink = struct {
        term_count: usize = 0,
        scalar_count: usize = 0,
        coordinate_count: usize = 0,
        tensor_count: usize = 0,
        action_count: usize = 0,
        saw_scalar: bool = false,
        saw_coordinate: bool = false,
        saw_tensor_none: bool = false,
        saw_action: bool = false,
        expected_z: u32,
        expected_w: u32,
        expected_profile: u32,
        expected_index: u32,

        /// emitWickTermStart records the single primitive profile Wick term.
        pub fn emitWickTermStart(self: *@This(), event: anytype) !void {
            if (event.left_index != 0 or event.right_index != 1 or event.term_index != 0)
            ↪  return error.UnexpectedWickTermEvent;
```

```
        self.term_count += 1;
    }

    /// emitWickScalar checks the expected free-boson profile scalar.
    pub fn emitWickScalar(self: *@This(), factor: anytype) !void {
        self.scalar_count += 1;
        switch (factor) {
            .value => |scalar| switch (scalar) {
                .monomial => |monomial| {
                    if (monomial.rational.numerator != -1 or monomial.rational.denominator
                    ↪  != 2 or monomial.atom == null) return
                    ↪  error.UnexpectedScalarFactor;
                    self.saw_scalar = true;
                },
                else => return error.UnexpectedScalarFactor,
            },
            else => return error.UnexpectedScalarFactor,
        }
    }

    /// emitWickCoordinate checks the resolved profile pole.
    pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
        self.coordinate_count += 1;
        if (factor.scalar.numerator != 1 or factor.scalar.denominator != 1) return
        ↪  error.UnexpectedCoordinateScalar;
        switch (factor.kernel) {
            .difference_power => |power| {
                if (power.coordinate.left != self.expected_z or power.coordinate.right !=
                ↪  self.expected_w or power.exponent != -1) return
                ↪  error.UnexpectedCoordinateFactor;
                self.saw_coordinate = true;
            },
            else => return error.UnexpectedCoordinateFactor,
        }
    }

    /// emitWickTensor checks that no tensor numerator is emitted.
    pub fn emitWickTensor(self: *@This(), factor: anytype) !void {
        self.tensor_count += 1;
        switch (factor) {
            .none => self.saw_tensor_none = true,
            else => return error.UnexpectedTensorFactor,
        }
    }

    /// emitWickAction checks the profile-derivative action.
    pub fn emitWickAction(self: *@This(), factor: anytype) !void {
        self.action_count += 1;
        switch (factor) {
            .profile_derivative => |action| {
                switch (action.profile) {
                    .symbol => |value| if (value != self.expected_profile) return
                    ↪  error.UnexpectedProfileLabel,
                    else => return error.UnexpectedProfileLabel,
                }
                switch (action.index) {
                    .symbol => |value| if (value != self.expected_index) return
                    ↪  error.UnexpectedIndexLabel,
                    else => return error.UnexpectedIndexLabel,
                }
                self.saw_action = true;
            },
            else => return error.UnexpectedProfileAction,
        }
```

```
        }

        /// emitWickTermEnd accepts the completed primitive term.
        pub fn emitWickTermEnd(self: *@This()) !void {
            _ = self;
        }
    };

    var sink = Sink{
        .expected_z = @intFromEnum(z),
        .expected_w = @intFromEnum(w),
        .expected_profile = @intFromEnum(f),
        .expected_index = @intFromEnum(mu),
    };
    try testing.expectEqual(@as(usize, 1), try shared.emitWickPairTerms(&X.config.sphere, ops,
    ↪  0, 1, &sink));
    try testing.expectEqual(@as(usize, 1), sink.term_count);
    try testing.expectEqual(@as(usize, 1), sink.scalar_count);
    try testing.expectEqual(@as(usize, 1), sink.coordinate_count);
    try testing.expectEqual(@as(usize, 1), sink.tensor_count);
    try testing.expectEqual(@as(usize, 1), sink.action_count);
    try testing.expect(sink.saw_scalar);
    try testing.expect(sink.saw_coordinate);
    try testing.expect(sink.saw_tensor_none);
    try testing.expect(sink.saw_action);

    const coeff = try local.profileCoefficient("a");
    const x0 = try local.targetPoint("x0");
    const polynomial_linear = X.profile.polynomialRnc(x0, &.{.{ .coefficient = coeff, .power =
    ↪  1 }});
    const polynomial_quadratic = X.profile.polynomialRnc(x0, &.{.{ .coefficient = coeff,
    ↪  .power = 2 }});
    try testing.expect(@intFromEnum(polynomial_linear) != @intFromEnum(polynomial_quadratic));
}

test "pair Wick resolves config-backed boundary projectors" {
    const testing = @import("std").testing;

    const X = freeBoson(.{ .dimension = 10 });
    const Disk = boundary(product(.{X}), .{
        freeBosonBoundary(.{
            .neumann = X.target.subspace(&.{ 0, 1, 2, 3 }),
            .dirichlet = X.target.complement(&.{ 0, 1, 2, 3 }),
            .dirichlet_position = X.target.point("x0"),
        }),
    });
    const neumann = X.target.subspace(&.{ 0, 1, 2, 3 });

    var local = Disk.local(testing.allocator);
    defer local.deinit();

    const mu = try local.index("mu");
    const nu = try local.index("nu");
    const y = try local.boundaryCoord("y");
    const y2 = try local.boundaryCoord("y2");
    const left = try Disk.op.free_boson_boundary.dXBoundary(&local, mu, 0, y);
    const right = try Disk.op.free_boson_boundary.dXBoundary(&local, nu, 1, y2);
    const ops = try local.ops(.{ left, right });

    const Sink = struct {
        term_count: usize = 0,
        scalar_count: usize = 0,
        coordinate_count: usize = 0,
        tensor_count: usize = 0,
```

```
        action_count: usize = 0,
        saw_scalar: bool = false,
        saw_coordinate: bool = false,
        saw_projector: bool = false,
        expected_y: u32,
        expected_y2: u32,
        expected_neumann: u32,

        /// emitWickTermStart records the single boundary Wick term.
        pub fn emitWickTermStart(self: *@This(), event: anytype) !void {
            if (event.left_index != 0 or event.right_index != 1 or event.term_index != 0)
            ↪  return error.UnexpectedWickTermEvent;
            self.term_count += 1;
        }

        /// emitWickScalar checks the boundary free-boson scalar factor.
        pub fn emitWickScalar(self: *@This(), factor: anytype) !void {
            self.scalar_count += 1;
            switch (factor) {
                .value => |scalar| switch (scalar) {
                    .monomial => |monomial| {
                        if (monomial.rational.numerator != 1 or monomial.rational.denominator
                        ↪  != 1 or monomial.atom == null) return
                        ↪  error.UnexpectedScalarFactor;
                        self.saw_scalar = true;
                    },
                    else => return error.UnexpectedScalarFactor,
                },
                else => return error.UnexpectedScalarFactor,
            }
        }

        /// emitWickCoordinate checks the differentiated boundary pole.
        pub fn emitWickCoordinate(self: *@This(), factor: anytype) !void {
            self.coordinate_count += 1;
            if (factor.scalar.numerator != 2 or factor.scalar.denominator != 1) return
            ↪  error.UnexpectedCoordinateScalar;
            switch (factor.kernel) {
                .difference_power => |power| {
                    if (power.coordinate.left != self.expected_y or power.coordinate.right !=
                    ↪  self.expected_y2 or power.exponent != -3) return
                    ↪  error.UnexpectedCoordinateFactor;
                    self.saw_coordinate = true;
                },
                else => return error.UnexpectedCoordinateFactor,
            }
        }

        /// emitWickTensor checks that the Neumann projector resolves from config.
        pub fn emitWickTensor(self: *@This(), factor: anytype) !void {
            self.tensor_count += 1;
            switch (factor) {
                .projector_metric => |projector_metric| switch (projector_metric.projector) {
                    .config => |value| switch (value) {
                        .tensor_projector => |projector| {
                            if (@intFromEnum(projector) != self.expected_neumann) return
                            ↪  error.UnexpectedProjectorConfig;
                            self.saw_projector = true;
                        },
                        else => return error.UnexpectedProjectorConfig,
                    },
                    else => return error.UnexpectedProjectorSource,
                },
                else => return error.UnexpectedTensorFactor,
```

```
            }
        }

        /// emitWickAction counts unexpected profile actions.
        pub fn emitWickAction(self: *@This(), factor: anytype) !void {
            _ = factor;
            self.action_count += 1;
        }

        /// emitWickTermEnd accepts the completed boundary term.
        pub fn emitWickTermEnd(self: *@This()) !void {
            _ = self;
        }
    };

    var sink = Sink{
        .expected_y = @intFromEnum(y),
        .expected_y2 = @intFromEnum(y2),
        .expected_neumann = @intFromEnum(neumann),
    };
    try testing.expectEqual(@as(usize, 1), try shared.emitWickPairTerms(&Disk.config.disk,
    ↪  ops, 0, 1, &sink));
    try testing.expectEqual(@as(usize, 1), sink.term_count);
    try testing.expectEqual(@as(usize, 1), sink.scalar_count);
    try testing.expectEqual(@as(usize, 1), sink.coordinate_count);
    try testing.expectEqual(@as(usize, 1), sink.tensor_count);
    try testing.expectEqual(@as(usize, 0), sink.action_count);
    try testing.expect(sink.saw_scalar);
    try testing.expect(sink.saw_coordinate);
    try testing.expect(sink.saw_projector);
}
```

References

Bibliography

[1] Joseph J. Atick and Ashoke Sen. “Covariant One-Loop Fermion Emission Amplitudes in Closed String Theories”. In: *Nuclear Physics B* 293 (1987), pp. 317–347. DOI: 10.1016/0550-3213(87)90074-4.

[2] Xi Yin. *String Theory*. URL: https://github.com/xiyin137/stringbook (visited on 06/06/2026).